Indian Institute of Technology Roorkee

Department of Computer Science and Engineering

HVAC Service Planner

Using Machine Learning for HVAC System Providers such as Aire Serv

Demand forecasting, failure prediction and customer retention

Project Report

Course: Machine Learning (CSC-505)

Group 1

Group member	Enrollment number
Kriti Vajpayee	25931008
Dhruv Aggarwal	25911019
Ankit Kumar	26911002
Animesh Basak	26911001
Sourav Kumar Boro	26535026

Under the guidance of

Dr. Durga Toshniwal

Professor, Department of Computer Science and Engineering, IIT Roorkee

October 2026

Contribution of the group members

Each member wrote the report chapters for two lifecycle stages and built the part of the project that those stages describe. The page numbers are the first and last page of each member's chapters.

Member	Enrollment	Lifecycle stages	Contribution	Pages
Kriti Vajpayee	25931008	1. Problem definition; 2. Data collection	Framed the problem and the product (stage 1). Searched for, downloaded and documented the three datasets and their sources (stage 2).	10 to 20
Dhruv Aggarwal	25911019	3. Data cleaning and preprocessing; 4. Exploratory data analysis	Cleaned the data, built the churn table, designed the train and test splits and the leakage tests (stage 3). Explored the data and drew the plots that guided the features (stage 4).	21 to 34
Ankit Kumar	26911002	5. Feature engineering; 6. Model selection	Designed and tested the engineered features (stage 5). Chose the candidate models and the selection rule, and ran the model contest (stage 6).	35 to 47
Animesh Basak	26911001	7. Model training; 8. Model evaluation and tuning	Trained all models under one protocol and measured learning curves and sensitivity (stage 7). Ran the held-out evaluation, error analysis and calibration checks (stage 8).	48 to 61
Sourav Kumar Boro	26535026	9. Model deployment; 10. Monitoring and maintenance	Built and tested the dashboard and the heatwave safeguard (stage 9). Wrote the monitoring checks, alert rules and maintenance plan (stage 10).	62 to 73

Shared work. All five members took part in choosing the topic and the product, in reviewing each other's chapters, and in preparing the presentation. The report chapters 11 to 13 (results, conclusion and recommendation, limitations and future work) were written jointly.

Where to find the other parts. Results and discussion, conclusion with the recommendation, limitations and future work, and the references follow the ten lifecycle chapters.

Part of the report	Where	Page
Abstract	Front matter	3
Results and discussion	Chapter 11	74
Conclusion and recommendation	Chapter 12	76
References and data sources	End	80

Abstract

HVAC service providers such as Aire Serv make three kinds of decisions every week: how many technicians and how much stock a hot or cold spell will need, which customer machines to inspect before they fail, and which customers to phone before they stop buying. We built HVAC Service Planner, a small dashboard that turns three machine learning predictions into short lists a manager can act on: an hourly power demand curve, a ranked inspection list and a ranked call list.

We had no data from a real HVAC company, so each prediction is trained on a public stand-in dataset: a year of hourly HVAC power readings (8,760 rows), the AI4I 2020 machine failure data (10,000 machines, 3.4% failures) and the Online Retail II purchase history (5,939 customers). We compare two models of our choice, Ridge regression and XGBoost, with ordinary linear or logistic regression, a Random Forest and a rule of thumb that a company could use today. Every score comes from data the model did not see: random hours, unseen months, a forecast of the last three months, a heatwave at or above 40 °C, unseen machines and unseen customers in a later period. Each experiment is repeated with five random seeds.

The main results are as follows. For demand, XGBoost misses by 0.065 kW on average, against 0.93 kW for the rule of thumb, but in the heatwave test its error grows to 0.34 kW, because tree models cannot predict beyond the temperatures they were trained on. Ridge regression extends the trend and misses by 0.08 kW there, so the dashboard uses Ridge for hours above 40 °C. For failure, XGBoost and Random Forest find 94% of failures in the 10% of machines they rank highest, against 36% for inspecting the most worn tools first. For churn, every model reaches about 0.78 ROC-AUC and finds 20% of lapsing customers in the top 10% of the list, twice as many as random contact and four points more than the recency rule. That gain is real but modest.

We recommend piloting the Demand Planner with the heatwave safeguard and the Failure Radar on real company data, and treating the Retention List as a low-cost aid until richer customer data are available. The report follows the ten stages of the machine learning lifecycle, from problem definition to monitoring, and each stage was written by one member of the group.

Keywords: HVAC, demand forecasting, predictive maintenance, customer churn, Ridge regression, XGBoost, held-out evaluation, model monitoring.

Table of contents

Abstract 3
Chapter 1. Problem definition 10
1.1 Why this stage comes first 10
1.2 The business we are helping 10
1.3 The product: HVAC Service Planner 11
1.4 Turning each question into a machine learning task 12
1.5 What counts as success 12
1.6 Scope 13
1.7 Constraints that shaped the design 13
1.8 Decisions we made and changed 14
1.9 Risks and assumptions 14
1.10 Deliverables of the whole project 14
Chapter 2. Data collection 15
2.1 What we needed and what we could get 15
2.2 Source links and licences 16
2.3 Dataset 1: hourly HVAC power (Pillar 1) 16
2.4 Dataset 2: machine failures, AI4I 2020 (Pillar 2) 17
2.5 Dataset 3: customer purchases, Online Retail II (Pillar 3) 18
2.6 How the data was collected, step by step 19
2.7 What we left out 19
2.8 Quality, bias and ethics 19
2.9 Data summary 20
Chapter 3. Data cleaning and preprocessing 21
3.1 Purpose of this stage 21
3.2 Checks on the raw files 21
3.3 Cleaning the retail transactions 21
3.4 Outliers: kept on purpose 22
3.5 Building the churn table 22
3.6 Missing values and encodings 23
3.7 Splitting the data 24
3.8 Guards against leakage 25
3.9 Known quirks we kept and report 25
3.10 Output of this stage 26
Chapter 4. Exploratory data analysis 27
4.1 What we wanted to learn 27

4.2 Pillar 1: what drives HVAC power 27
4.3 Pillar 2: what separates failing machines 30
4.4 Pillar 3: how customers behave 31
4.5 Summary of findings 34
4.6 Limits of this analysis 34
Chapter 5. Feature engineering 35
5.1 What a feature is and why we build them 35
5.2 Pillar 1: demand features 36
5.3 Pillar 2: failure features 36
5.4 Pillar 3: customer features 37
5.5 Does feature engineering help? 38
5.6 Which features the models rely on 39
5.7 What we decided not to do 40
5.8 Checks on the features 41
Chapter 6. Model selection 42
6.1 The question of this stage 42
6.2 The candidates 42
6.3 Candidates we left out, and why 44
6.4 The trees-and-extrapolation problem 44
6.5 The selection rule 45
6.6 The contest result at a glance 45
6.7 What the rule selects 46
6.8 Summary of the decision 47
Chapter 7. Model training 48
7.1 What training means here 48
7.2 The training protocol 48
7.3 Settings 49
7.4 What the models cost to train 49
7.5 How much data does each model need? 50
7.6 Sensitivity to the settings 51
7.7 Training the final models 51
7.8 Reproducibility 52
7.9 Problems we hit and what we changed 52
Chapter 8. Model evaluation and tuning 53
8.1 How we evaluate 53
8.2 Pillar 1: demand 53

8.3 Pillar 2: failure 56
8.4 Pillar 3: churn 58
8.5 Stability across seeds 59
8.6 Tuning 59
8.7 Summary of the evaluation 60
8.8 What could make these numbers too good 61
Chapter 9. Model deployment 62
9.1 What deployment means for this project 62
9.2 System architecture 62
9.3 What gets deployed 63
9.4 The three tools 63
9.5 The heatwave safeguard 65
9.6 How the dashboard is tested 66
9.7 Installing and running 66
9.8 Operating limits 66
9.9 What a production deployment would need 66
9.10 Deployment checklist 67
Chapter 10. Monitoring and maintenance 68
10.1 Why a deployed model needs watching 68
10.2 What can go wrong 68
10.3 The monitoring loop 68
10.4 The checks we built 69
10.5 Alert rules 70
10.6 How the checks behave on our data 70
10.7 Maintenance routine 72
10.8 What to do when a check fires 72
10.9 Keeping the project maintainable 73
10.10 Limits of our monitoring 73
10.11 Summary 73
Chapter 11. Results and discussion 74
11.1 The results on one page 74
11.2 What the results tell us 74
11.3 How the product is used 75
11.4 What the numbers would mean in practice 75
11.5 Threats to the validity of the results 75
Chapter 12. Conclusion and recommendation 76

12.1 Conclusion 76
12.2 Recommendation 76
12.3 Closing note 77
Chapter 13. Limitations and future work 78
13.1 Limitations 78
13.2 Future work: where a graph neural network may help 78
13.3 Other future work 79
References and data sources 80
Data sources 80
Methods and software 80
Course material 80

List of figures

1.1 The ten lifecycle stages and who wrote each one 10
1.2 HVAC Service Planner, from data to decision 11
1.3 The bar to beat: rule of thumb against our models on unseen data 13
2.1 The three datasets, one per pillar 15
2.2 One year of hourly readings, shown by day 17
2.3 AI4I machines by type, failure rate by type and failure causes 18
2.4 Invoices and active customers per month, and share of invoices by country 19
3.1 From 1.07 million original lines to 44,870 invoices 21
3.2 Invoice values and the concentration of spend 22
3.3 How one row of the churn table is built 23
3.4 How each pillar is split into training and test data 24
4.1 Average HVAC power by hour and month 27
4.2 Power by HVAC mode, the temperature behind each mode, and the daily rhythm 28
4.3 Correlation between the Pillar 1 columns 29
4.4 Sensor readings of failing and healthy machines 30
4.5 Correlation between the Pillar 2 columns 31
4.6 Customer behaviour in Online Retail II 32
4.7 Lapse rate by number of orders, recent orders and order rhythm 32
4.8 Correlation between the Pillar 3 features and the lapse target 33
5.1 From raw columns to model inputs 35
5.2 Power against temperature, and against the absolute gap 36
5.3 Failure rate by decile of each engineered feature 37
5.4 Lapse rate by trend, return share and tenure 38
5.5 Test score with basic and with engineered inputs 39
5.6 Feature importance for failure and churn (XGBoost on the left, Ridge on the right) 40
5.7 Feature importance for churn 40
6.1 The model families we compare 42
6.2 Why trees stop rising outside the training range 44
6.3 How the tie rule picks a model for failure and churn 45
6.4 Every model on every test 46
7.1 What happens in one run of the experiment 48
7.2 Seconds to fit each model once 50
7.3 Learning curves: test score against the amount of training data 50
7.4 Sensitivity of the main settings 51
8.1 Residuals of Ridge and XGBoost in the forecast months and in the heatwave 55
8.2 Error by hour of day and by month 55
8.3 ROC curves, and the trade-off at different thresholds 56
8.4 Confusion matrices at each model's own threshold 57
8.5 Failures found as the inspection list grows 57
8.6 Calibration of the risk scores 58
8.7 Spread of the scores over the five seeds 59
8.8 Where XGBoost helps and where it does not 60
9.1 System architecture 62
9.2 Demand Planner output for a normal July day and for a heatwave day 63
9.3 Failure Radar: the riskiest machines in a sample of 500 64
9.4 Retention List: the customers most likely to go quiet 65
9.5 How the safeguard chooses the model for each hour 65
10.1 The monitoring loop 69
10.2 Alert rules 70
10.3 Input drift in three comparisons 70

10.4 Demand error and churn calibration as time moves forward 71
10.5 The maintenance calendar 72
11.1 From the result to the decision, for each pillar 75
12.1 Our recommendation, with the evidence and the condition for each tool 76
13.1 Gaps in our results and whether a graph model could close them 78

Chapter 1. Problem definition

Lifecycle stage 1 of 10. Written by Kriti Vajpayee (25931008).

1.1 Why this stage comes first

Every later choice in the project, from which data to collect to which metric to report, follows from how the problem is stated. A vague statement such as "use machine learning for HVAC companies" cannot be tested. We therefore spent the first days writing down who would use the result, what they would decide with it, and what number would tell us the model is good enough. This document records that reasoning. The other nine stage documents refer back to it.

The lifecycle we follow has ten stages. Figure 1.1 shows them in order, with the group member who wrote each one. The arrow that returns from stage 5 to stage 6 only marks the line break in the picture: the work itself ran in order, and we went back to earlier stages whenever a result surprised us (for example, we revisited the churn table in stage 3 after seeing weak results in stage 8).

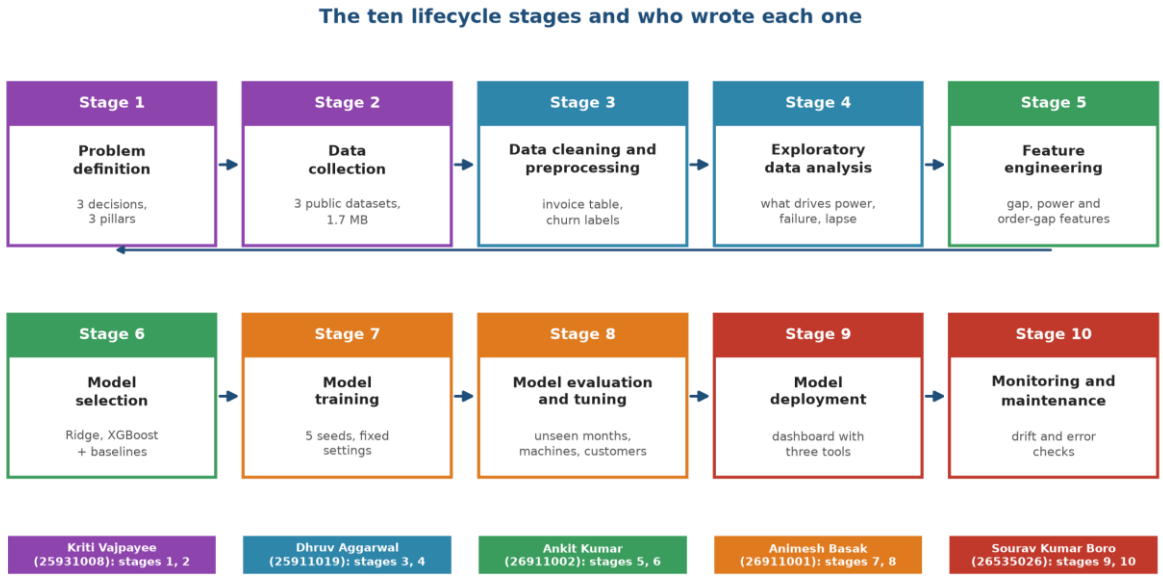


Figure 1.1. *The ten lifecycle stages and who wrote each one*

Figure 1.1 is a map of the report. Stages 1 and 2 (purple) set up the problem and the data. Stages 3 and 4 (blue) clean the data and look at it. Stages 5 and 6 (green) turn the data into model inputs and choose the models. Stages 7 and 8 (orange) train and test them. Stages 9 and 10 (red) put the models into a dashboard and describe how to watch them afterwards. Each stage has one author, whose name and enrollment number sit under its box.

1.2 The business we are helping

Our topic is "using machine learning for HVAC system providers such as Aire Serv". Aire Serv is a franchise that installs, repairs and maintains heating, ventilation and air-conditioning equipment. A franchise manager makes the same kinds of decisions every week, and most of them are made today with experience and spreadsheets:

1. How many technicians and how much stock do we need for the coming days? Demand for repairs follows the weather, because systems work hardest on the hottest and coldest days. A heatwave brings a rush of calls and a high electricity load.

2. Which customer machines should we visit before they break? A planned inspection costs far less than an emergency call-out, but nobody can inspect every machine.
3. Which customers are drifting away? Winning back a customer who is about to go quiet is cheaper than finding a new one, but the sales team can only phone so many people.

These are three separate decisions with three different kinds of data. We decided to build one product that handles all three, so that the group could show breadth, and to keep each part simple enough that every member can explain it.

1.3 The product: HVAC Service Planner

HVAC Service Planner is a small dashboard with three tools. Each tool takes one prediction and turns it into a short list that a manager can act on. Figure 1.2 shows the path from data to decision for each tool.

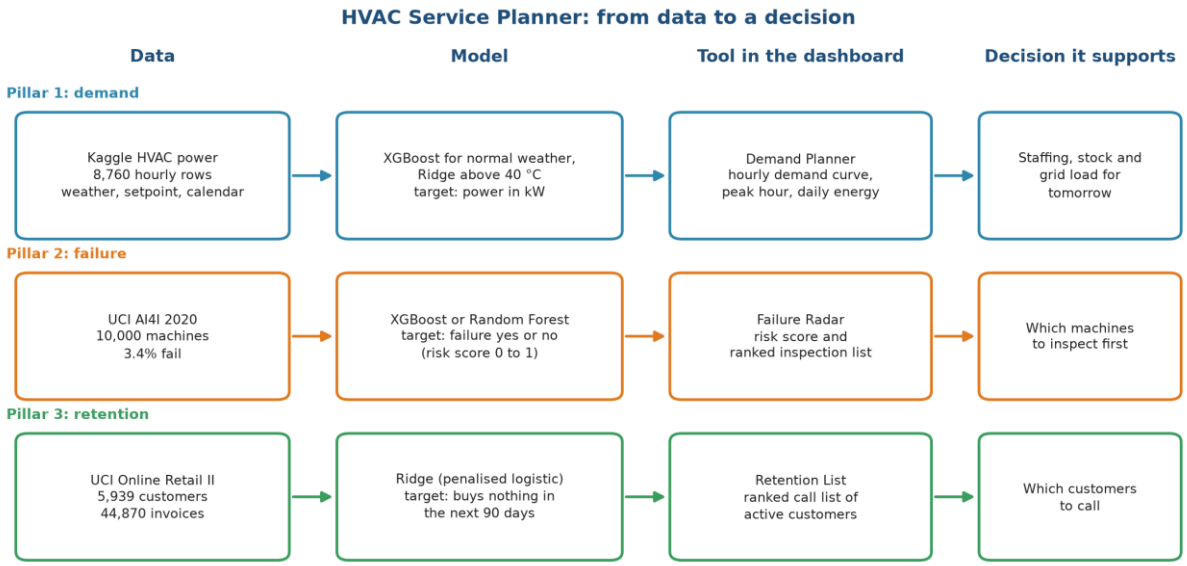


Figure 1.2. *HVAC Service Planner, from data to decision*

Figure 1.2 reads from left to right. The first column names the dataset behind each tool. The second column names the model and the quantity it predicts. The third column is the tool the manager sees in the dashboard, and the last column is the decision that the tool supports. Pillar 1 predicts a number (power in kilowatts), while Pillars 2 and 3 predict a yes or no outcome and return a risk score between 0 and 1. The model names in the middle column are the result of stages 6 to 8; at the time we wrote the problem statement we only knew that we would compare a few simple models.

Tool	Manager's question	What the tool returns	Who uses it
Demand Planner (Pillar 1)	How much power will the HVAC systems draw tomorrow, and when is the peak?	Hourly demand curve, peak hour, energy for the day	Dispatch and stock planning
Failure Radar (Pillar 2)	Which machines should we inspect first?	Risk score per machine and a ranked inspection list	Maintenance lead
Retention List (Pillar 3)	Which active customers are about to go quiet?	Ranked call list with each customer's risk	Customer success and marketing

1.4 Turning each question into a machine learning task

A business question becomes a machine learning problem only when we can say what goes in, what comes out, how far ahead we look and how we score the answer. The table below fixes these four things for each pillar. We wrote it before touching any data, and we have not changed it since.

	Pillar 1: demand	Pillar 2: failure	Pillar 3: retention
Task type	Regression (predict a number)	Binary classification, rare event	Binary classification
Unit of prediction	One building-hour	One machine	One active customer at a cutoff date
Inputs	Outdoor and indoor temperature, humidity, hour, month, HVAC mode	Air and process temperature, rotational speed, torque, tool wear, machine type	Purchase history before the cutoff
Output	Power in kW	Failure yes or no, with a risk score	Lapses yes or no, with a risk score
Looking ahead	Same hour (weather forecast is the input)	The recorded failure label	90 days after the cutoff
Main metric	Mean absolute error in kW	PR-AUC and share of failures in the top 10%	ROC-AUC and share of lapsers in the top 10%
Why this metric	Easy to read: average miss in kW	Failures are 3.4% of machines, so accuracy is meaningless	The team can call only a few customers, so the top of the list matters

Two decisions in this table deserve a note.

For Pillar 2, a model that always predicts "no failure" is correct for 96.6% of machines and finds none of the failures. Accuracy would reward it. We therefore report PR-AUC, which stays low unless the model finds the rare failures with few false alarms, and the share of all failures that fall inside the 10% of machines with the highest risk. A manager understands the second number directly: "if we inspect one machine in ten, we catch this share of the failures".

For Pillar 3, the Online Retail II data has no column that says whether a customer left. An earlier version of our project used the flag "no purchase in 90 days" as the target while also using the days since the last purchase as an input. That is circular, since the input already contains the answer. We now define the target in the future: take a cutoff date, describe each customer using only what happened before it, and ask whether the customer buys nothing in the next 90 days. Stage 3 explains how the table is built.

1.5 What counts as success

We need a bar for "good enough", otherwise any score can be called a success. We set two kinds of bars.

First, the model must beat a rule of thumb that a company could apply today without any machine learning. For each pillar we picked the simplest sensible rule:

- Demand: predict the average load for that hour of the day.
- Failure: inspect the machines with the most tool wear first.
- Retention: call the customers who have not bought for the longest time.

Second, the model must beat two honest baselines from statistics and classic machine learning, so that a more complicated model is justified only if it earns its place. The baselines are ordinary linear or logistic regression (statistics) and a Random Forest (classic machine learning).

Figure 1.3 shows what the rules of thumb achieve on data they have not seen, next to our two models. This is the bar we set for ourselves, and it is also a preview of the result, since we computed it after the work was done.

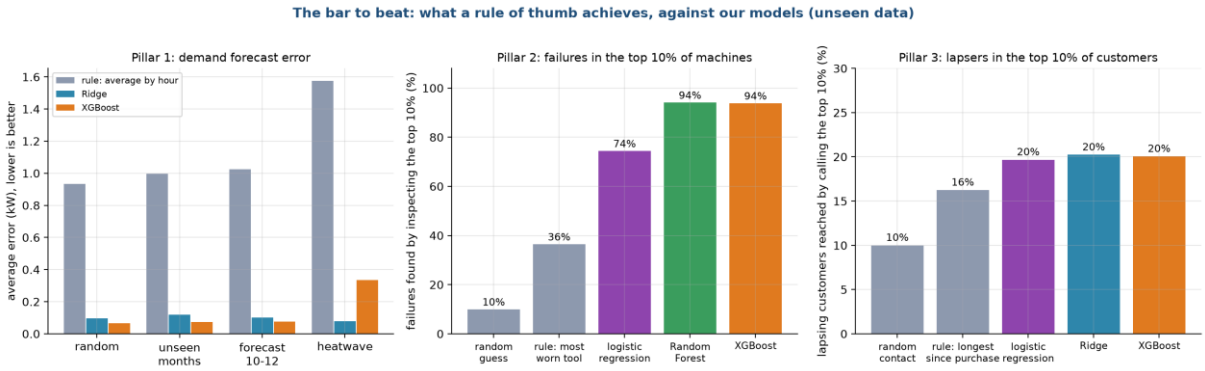


Figure 1.3. *The bar to beat: rule of thumb against our models on unseen data*

In the left panel the grey bars are the error of the "average by hour" rule for demand. It misses by about 0.9 to 1.6 kW depending on the test, while Ridge and XGBoost miss by 0.12 kW or less except in the heatwave test, where XGBoost reaches 0.34 kW. We return to that exception in stage 8. In the middle panel, inspecting the most worn tools first finds 36% of failures in the top 10% of machines, against 10% for a random list, 74% for logistic regression and 94% for Random Forest and XGBoost. In the right panel the recency rule reaches 16% of the customers who lapse in the top 10%, against 10% for random contact and 20% for our models. The retention gain is small, and we say so plainly in the conclusion.

1.6 Scope

In scope:

- Three predictions on public data, each tested on data the model never saw.
- A dashboard that turns predictions into lists and curves.
- A written account of what worked, what did not, and what to watch after deployment.

Out of scope, and stated as limits:

- Real Aire Serv data. We had no access to any HVAC service company, so all three datasets are public stand-ins (stage 2).
- Cross-selling a heating product to an air-conditioning customer. It needs product categories and a customer-product graph, which we leave to future work.
- Several buildings. The demand data covers one building, so we cannot compare sites.
- Real-time operation. The dashboard works on stored data and user inputs, and it does not connect to sensors.

1.7 Constraints that shaped the design

- Time. The submission deadline fell about a week after the topic was fixed.
- Hardware. Everything had to run on a laptop without a graphics card. Our models train in seconds (stage 7), and the whole pipeline runs in under ten minutes.
- Explainability. Every group member must be able to explain every model at the viva. This ruled out stacking many models and pushed us toward Ridge regression and gradient-boosted trees, which we can describe in a few sentences.

- Size. We kept the data to 1.7 MB so the project can be handed in and reproduced without large downloads.

1.8 Decisions we made and changed

The scope above is the result of several changes. Writing them down helps a reader judge how much trust to place in the final choice.

Change	Reason
Dropped the graph neural network that an earlier version used	In our tests it never beat a simple model by a clear margin, and the synthetic graph gave it nothing to learn from. It remains in the future work section as a hypothesis.
Cut the datasets from a larger set of graph tables and zip files to three files	Only three files are needed for the three pillars.
Replaced the circular churn label with a future-based one	The earlier label contained its own input.
Added a statistical baseline (linear and logistic regression), a Random Forest and rules of thumb	A comparison with only our two chosen models proves nothing.
Added a held-out evaluation with five random seeds	A single split can flatter a model by chance.
Decided against ANN, LSTM and similar models	The data is tabular and small, and these models would add complexity without evidence of gain.

1.9 Risks and assumptions

Risk or assumption	Effect if wrong	How we handle it
The datasets resemble what a real HVAC company holds	Scores would not transfer	Stated as a limit in every stage; the method, not the score, is our claim
Pillar 1 data is simulated	Patterns may be cleaner than real life	We call it a clean test bed and add a heatwave test that stresses it
A shop is a fair stand-in for a service company	Churn behaviour differs	We keep the definition simple and state the difference
Five seeds are enough to see noise	Small gaps might be luck	The selection rule treats gaps below one standard deviation as ties

1.10 Deliverables of the whole project

The group submits a report, a presentation, the code, the data sources and a help file. The code is organised so that one command trains and tests every model, and another opens the dashboard. Stage 9 describes the dashboard and stage 10 describes the checks to run after it goes live.

Chapter 2. Data collection

Lifecycle stage 2 of 10. Written by Kriti Vajpayee (25931008).

2.1 What we needed and what we could get

A real HVAC service provider holds three kinds of records: hourly energy readings from its buildings, sensor logs from the machines it maintains, and a history of what each customer bought. We had no access to such a company, so we looked for public datasets that match each record type and are small enough to share with the report. We set four rules for the search:

1. The data must be public and free to use for coursework, with a source link we can cite.
2. Each dataset must support one of the three predictions in stage 1 without anything invented by us.
3. The total size must stay small, so the project runs on a laptop and can be handed in as a zip file.
4. We would rather use a dataset honestly labelled as a stand-in than build artificial data that looks real.

Three datasets passed. Figure 2.1 shows each one with its size, period, columns, target and the main caveat.

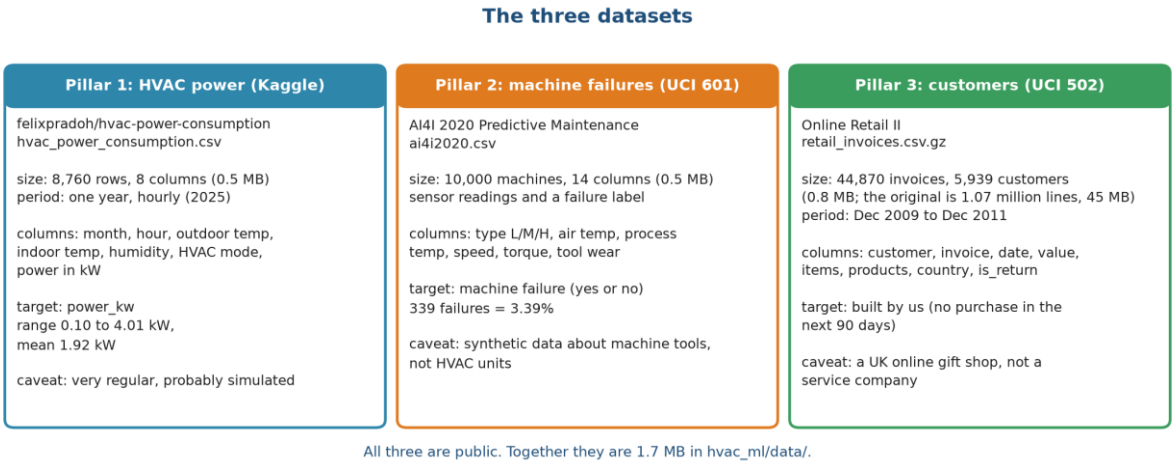


Figure 2.1. *The three datasets, one per pillar*

Figure 2.1 is a data card for each pillar. The blue card is the hourly HVAC power file from Kaggle, 8,760 rows covering one year. The orange card is the AI4I 2020 machine failure file from the UCI repository, 10,000 machines. The green card is Online Retail II from UCI, stored by us as one row per invoice. The line at the bottom says that together they take 1.7 MB. Each card ends with a caveat, because none of the three comes from an HVAC service company.

2.2 Source links and licences

Pillar	Dataset	Link	Licence note
1. Demand	HVAC power consumption (Kaggle, user felixpradoh)	https://www.kaggle.com/datasets/felixpradoh/hvac-power-consumption	We rebuilt this link from the dataset name we recorded. Open it and check the licence before submitting.
2. Failure	AI4I 2020 Predictive Maintenance, UCI #601	https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset	Listed on UCI under Creative Commons Attribution 4.0
3. Churn	Online Retail II, UCI #502	https://archive.ics.uci.edu/dataset/502/online+retail+ii	Listed on UCI under Creative Commons Attribution 4.0

The two UCI files can be downloaded automatically by the code if they are missing. The Kaggle file needs a Kaggle login, so it must be downloaded by hand. The help file explains the steps.

2.3 Dataset 1: hourly HVAC power (Pillar 1)

The file has one row for every hour of 2025, 8,760 rows in all, with no missing values. Table 2.1 lists the columns.

Table 2.1. *Columns of the hourly HVAC power file*

Column	Meaning	Type	Range or values
timestamp	Date and hour	datetime	1 Jan 2025 00:00 to 31 Dec 2025 23:00
Month	Month number	integer	1 to 12
Hour	Hour of day	integer	0 to 23
Ambient_Temp	Outdoor temperature in °C	number	6.06 to 50.38, mean 28.1
Indoor_Temp	Indoor temperature in °C	number	19.07 to 25.81, mean 22.8
Humidity_pct	Relative humidity in %	number	15 to 98, mean 54.6
HVAC_Mode	Cooling, Heating or Idle	category	4,886 cooling, 1,637 heating, 2,237 idle hours
Power_kW	HVAC power draw in kW (our target)	number	0.10 to 4.01, mean 1.92

Figure 2.2 shows the year as daily values. The top panel is the daily mean outdoor temperature with the 19 °C, 27 °C and 40 °C lines that matter later. The bottom panel is the daily mean and peak power.

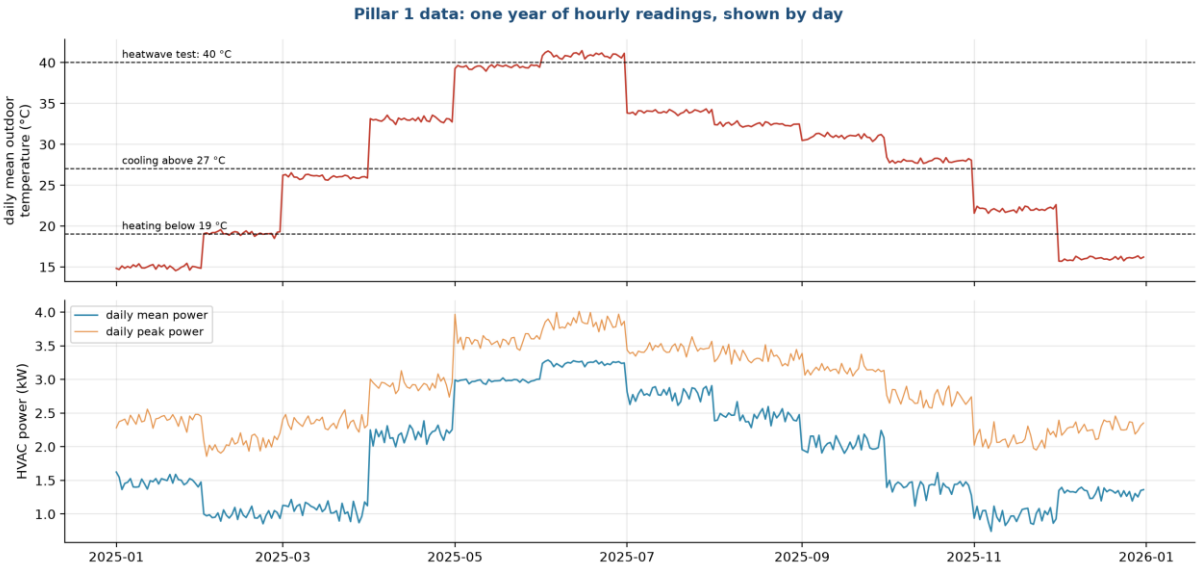


Figure 2.2. *One year of hourly readings, shown by day*

The temperature does not move smoothly through the year. It steps up or down on the first day of almost every month and then stays flat, and the same shape appears in the power curve. Real weather does not behave this way, which is why we believe the file is simulated. We could not confirm this from the Kaggle page, so we describe it as probably simulated. It also tells us something useful: the month is almost a proxy for the temperature level, and a model that sees an unfamiliar month has to rely on the temperature itself. The monthly average is above 39 °C in May and June, and 824 hours between April and September reach 40 °C or more, which is what our heatwave test uses.

2.4 Dataset 2: machine failures, AI4I 2020 (Pillar 2)

The file holds 10,000 simulated machine records. The UCI page describes it as synthetic data built to resemble real industrial maintenance logs. It describes machine tools, not HVAC units, and we use it as a stand-in for "sensor readings that precede a breakdown".

Column	Meaning	Values
UDI, Product ID	Row number and product code	identifiers, not used as inputs
Type	Product quality class	L 6,000, M 2,997, H 1,003
Air temperature [K]	Surrounding air temperature	295.3 to 304.5, mean 300.0
Process temperature [K]	Temperature of the process	305.7 to 313.8, mean 310.0
Rotational speed [rpm]	Spindle speed	1,168 to 2,886, mean 1,539
Torque [Nm]	Twisting force	3.8 to 76.6, mean 40.0
Tool wear [min]	Minutes the tool has been in use	0 to 253
Machine failure	1 if the machine failed (our target)	339 ones, 3.39%
TWF, HDF, PWF, OSF, RNF	Which failure type occurred	46, 115, 95, 98 and 19 cases

The five failure-type columns describe how a failure happened, so they are the answer in disguise. We do not use them as inputs, and a unit test checks that they stay out. Figure 2.3 gives the composition of the data.

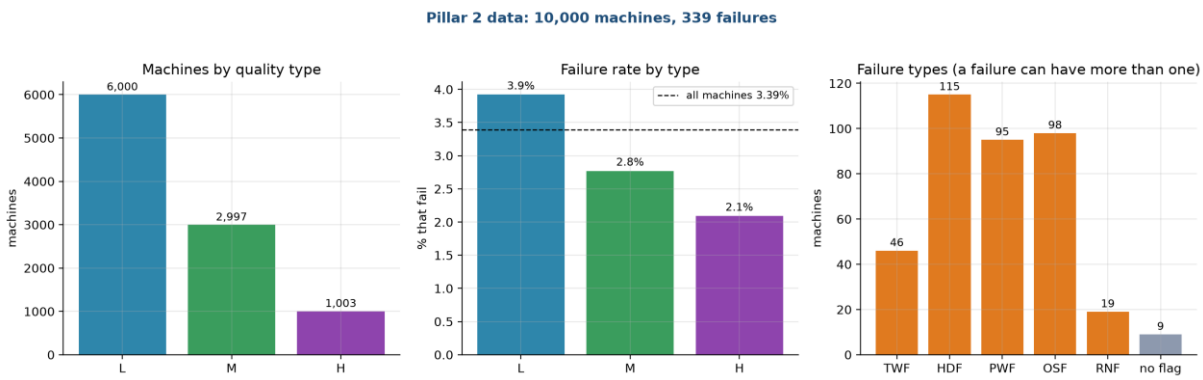


Figure 2.3. *AI4I machines by type, failure rate by type and failure causes*

The left panel shows 6,000 low-quality (L), 2,997 medium (M) and 1,003 high (H) machines. The middle panel shows that failure is more common in the low class, 3.9%, than in the medium (2.8%) and high (2.1%) classes, with the dashed line at the overall 3.39%. The right panel counts failure causes. Heat dissipation (HDF, 115), overstrain (OSF, 98) and power (PWF, 95) account for most cases, tool wear (TWF) for 46 and random failures (RNF) for 19. One failure can have more than one cause, and 9 failures carry no cause flag at all. These nine are a quirk of the dataset, and we leave them in.

2.5 Dataset 3: customer purchases, Online Retail II (Pillar 3)

Online Retail II is a real transaction log from a UK online gift shop, from 1 December 2009 to 9 December 2011. The original workbook has 1,067,371 lines and is about 45 MB. A line is one product on one invoice. We do not need the product level for the churn task, so we collapse it to one row per invoice.

Column in our invoice table	Meaning
customer	Customer id (anonymised number)
invoice	Invoice number; numbers starting with C are returns
date	Date and time of the invoice
value	Money value of the invoice (quantity times price, summed)
items	Total number of units
products	Number of different products
country	Customer's country
is_return	True for a return or cancellation

After this step the table has 44,870 invoices from 5,939 customers: 36,969 purchases and 7,901 returns. The UK accounts for 90% of invoices, and 41 countries appear in total. The file shrinks from 45 MB to 0.76 MB. Figure 2.4 shows how activity changes over time.



Figure 2.4. *Invoices and active customers per month, and share of invoices by country*

The left panel counts purchase invoices per month and the middle panel counts customers who bought in that month. Both peak in October and November of each year, the build-up to Christmas, and drop sharply in December, partly because the last month of data ends on 9 December 2011 and partly because the shop is quiet over the holidays (we did not check the reason in the data). The right panel shows the country split, where the United Kingdom is far ahead of Germany, France, Ireland and the Netherlands. This seasonal pattern matters for churn: a customer who is quiet in December is not necessarily lost. We keep it in mind when we choose the test period in stage 3.

2.6 How the data was collected, step by step

1. We downloaded the AI4I archive from UCI and extracted the CSV file.
2. We downloaded the Kaggle HVAC file by hand, because Kaggle needs a login.
3. We downloaded the Online Retail II archive from UCI, read both sheets of the Excel workbook and joined them.
4. We cleaned the retail lines and grouped them into invoices (stage 3), then saved the result as a compressed CSV.
5. We recorded the source link, file name, size and row count of each file in the project documents.
6. We wrote a loader for each dataset that checks the row count and the absence of missing values every time it runs. A test repeats the check.

2.7 What we left out

The first version of the project carried a larger collection: synthetic tables for locations, devices, spare parts and residents, a derived customer file whose "sleeper" column was just the days since last purchase compared with 90, and the original zip archives. They were needed only for the graph neural network, which we removed. A backup of that version sits outside the project folder. Removing them cut the data to three files and made the origin of every number easy to trace.

2.8 Quality, bias and ethics

- All three datasets are stand-ins. The numbers show that the method works on these data. They do not promise the same scores for a real company.
- The HVAC data covers one building for one year. We cannot say how the model would behave in another climate or another building.
- The failure data is synthetic and describes machine tools. The failure patterns come from the rules used to generate it, so a model can learn those rules almost exactly, which partly explains the high scores in stage 8.

- The retail data is real but anonymised. Customer ids are numbers, and no names or addresses are in the file. We use it only for coursework and we do not try to identify anyone.
- The retail shop sells gifts and mostly to businesses, and 90% of invoices are British. Results may not carry over to other countries.

2.9 Data summary

Item	Pillar 1	Pillar 2	Pillar 3
Rows in our file	8,760	10,000	44,870 invoices
Columns	8	14	8
Size on disk	0.49 MB	0.52 MB	0.76 MB
Time span	1 year, hourly	none (one snapshot)	2 years
Missing values	none	none	none after cleaning
Target	Power in kW	Failure yes or no	Built in stage 3
Positive share	not applicable	3.39%	31% to 58% by cutoff

Chapter 3. Data cleaning and preprocessing

Lifecycle stage 3 of 10. Written by Dhruv Aggarwal (25911019).

3.1 Purpose of this stage

Raw data is rarely ready for a model. This stage records every change we made between the downloaded files and the tables that the models read, and why. It has four parts: cleaning (removing or repairing bad records), building the churn table (the one place where we construct the target ourselves), encoding and scaling (turning columns into numbers a model can use), and splitting (deciding which rows may be used for training and which are kept for testing). The splits matter most, because a mistake there makes every later score too good to be true.

3.2 Checks on the raw files

Before any cleaning we checked each file for missing values, impossible values and duplicates.

Check	HVAC power	AI4I failures	Online Retail II
Rows	8,760	10,000	1,067,371 lines
Missing values	none	none	about 22.8% of lines have no customer id
Impossible values	none; power is never negative	none	a few lines with a price of zero or less; negative quantities on returns
Duplicate rows	none (one row per hour)	none (unique row number)	not checked line by line; invoices are grouped
Value ranges	plausible	plausible	very wide, see outliers below

The HVAC and AI4I files are already tidy, so their cleaning is limited to renaming columns to short names and adding a few checks. The retail file needed real work.

3.3 Cleaning the retail transactions

Figure 3.1 follows the lines of the original workbook through our cleaning steps.

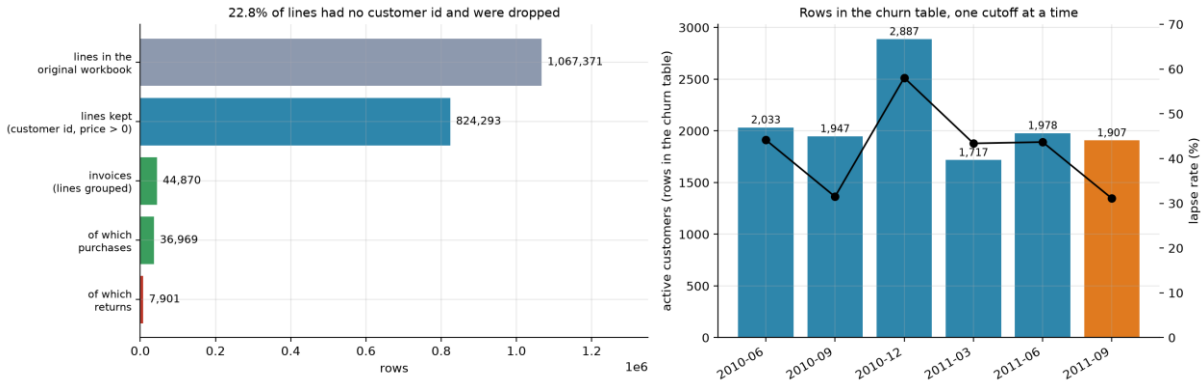


Figure 3.1. *From 1.07 million original lines to 44,870 invoices*

The top bar is the original workbook, 1,067,371 lines. The second bar is what remains after we drop lines without a customer id and lines with a price of zero or less, 824,293 lines, which means 22.8% of the original lines were removed. Grouping the remaining lines by invoice gives 44,870 invoices (third bar). Of these, 36,969 are purchases and 7,901 are returns. The right panel of the same figure shows how many rows each churn snapshot gets, with the lapse rate as a line, which section 3.5 explains. The invoice table is only 0.76 MB, against 45 MB for the original workbook.

The steps, in the order we applied them:

1. Joined the two sheets of the workbook (the data comes in two yearly sheets).
2. Dropped lines with no customer id. Without an id we cannot follow a buyer over time, so such lines are useless for churn. This removed about 243,000 lines.
3. Dropped lines with a price of zero or less. These are stock adjustments and write-offs, not sales. Almost all of them were already in the group above.
4. Marked invoices whose number starts with "C" as returns. We keep them in a separate flag and use them only to compute each customer's return share. A return never counts as a purchase, so it cannot make a lasping customer look active.
5. Computed the value of each line as quantity times price, and grouped lines by customer and invoice, so that each invoice becomes one row with its date, total value, units, number of distinct products and country.
6. Kept only positive quantities for purchases.

3.4 Outliers: kept on purpose

Invoice values range from 0.38 to 168,470 currency units. Figure 3.2 shows how skewed they are.



Figure 3.2. *Invoice values and the concentration of spend*

The left panel is a histogram of invoice values on a logarithmic axis. Half of all invoices are below 305, 95% are below 1,254 and 99% are below 3,698, yet a few reach six figures. The right panel is a concentration curve: the top 1% of customers account for 32% of all spending. The curve bends sharply upward, which is typical of wholesale customers.

We did not delete these large invoices. They come from real customers, and a customer who spends heavily is exactly the customer a service company wants to keep. Deleting them would remove the cases that matter most for retention. Instead, we give the models logarithmic versions of spend and order value, so that a handful of huge numbers cannot dominate a linear model. Tree models do not care about scale, so XGBoost and Random Forest are unaffected.

3.5 Building the churn table

The retail file has no column that says a customer has left, so we build the target. This is the most delicate step in the project, because a careless version leaks the answer into the inputs. Figure 3.3 shows how each row of the table is built.

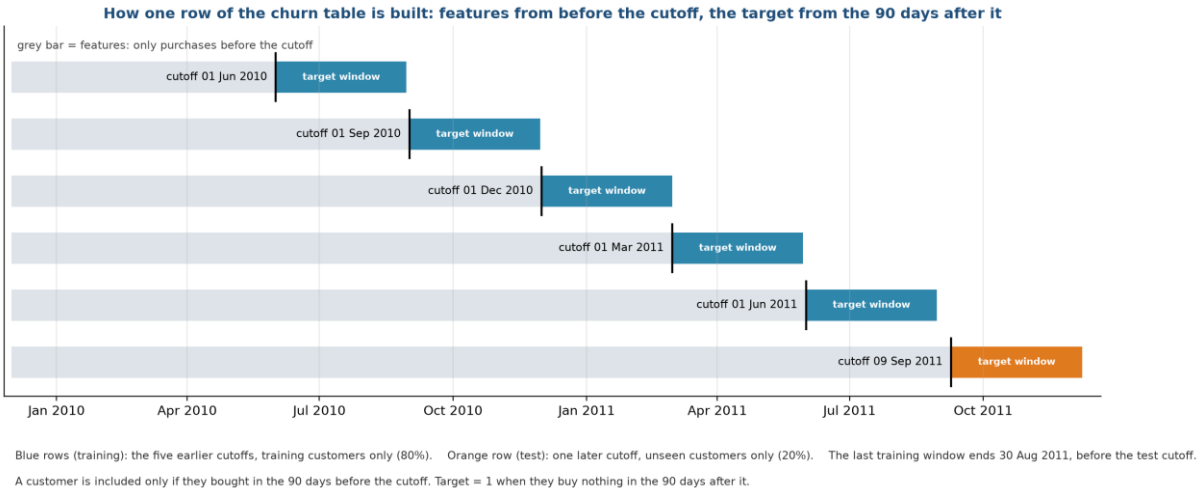


Figure 3.3. *How one row of the churn table is built*

Each horizontal line is one snapshot of the customer base. The grey bar is the history we are allowed to use for the features: only purchases before the cutoff date (the black tick). The coloured bar is the 90-day target window after the cutoff, in which we only look to see whether the customer bought anything. A customer is included in a snapshot only if they bought in the 90 days before the cutoff, so we are asking "of the customers who are active now, who will go quiet?". The target is 1 if they buy nothing in the next 90 days.

We take five training snapshots (1 June 2010, 1 September 2010, 1 December 2010, 1 March 2011 and 1 June 2011, blue bars) and one later test snapshot (9 September 2011, orange bar). The last training window ends on 30 August 2011, before the test cutoff, so no training label looks into the test period. The test window ends on 8 December 2011, the last full day in the data.

Table 3.1 gives the size and the lapse rate of every snapshot, which we also plotted in stage 2.

Table 3.1. *Size and lapse rate of each churn snapshot*

Cutoff	Active customers	Share that lapse
1 June 2010	2,033	44%
1 September 2010	1,947	31%
1 December 2010	2,887	58%
1 March 2011	1,717	43%
1 June 2011	1,978	44%
9 September 2011 (test)	1,907	31%

The December 2010 snapshot has the highest lapse rate, because the 90 days after 1 December include the quiet holiday weeks, and the September snapshots have the lowest, because their window covers the pre-Christmas buying season. The same customer can appear in several snapshots, once for each cutoff at which they were active. A customer is a different row at each cutoff, with different features and a different outcome.

3.6 Missing values and encodings

After the steps above, only one column has gaps: the average days between orders. It is undefined for a customer with a single order. We fill it with the median of the training data, inside the model pipeline, so that the median never comes from test rows.

Other columns need a numeric form:

Column	Treatment	Reason
Hour and month (Pillar 1)	Sine and cosine of the angle on a circle	Hour 23 is next to hour 0 and December is next to January. A plain number would put them far apart.
HVAC mode, machine type	One column per category with values 0 or 1	Models cannot read text, and there is no order between categories.
Money and counts (Pillar 3)	Original value plus a log version	Spend is very skewed (section 3.4).
Every numeric input	Standardised to mean 0 and standard deviation 1 inside the linear models	Ridge penalises large coefficients, so inputs must have comparable scale. Tree models skip this step.

All imputing and scaling sit inside the model pipeline, so they are learned from the training rows alone and applied unchanged to the test rows. Doing it outside the pipeline, on the full table, would let test statistics leak into training.

3.7 Splitting the data

We decided how to split before looking at any model score. Figure 3.4 shows the designs.

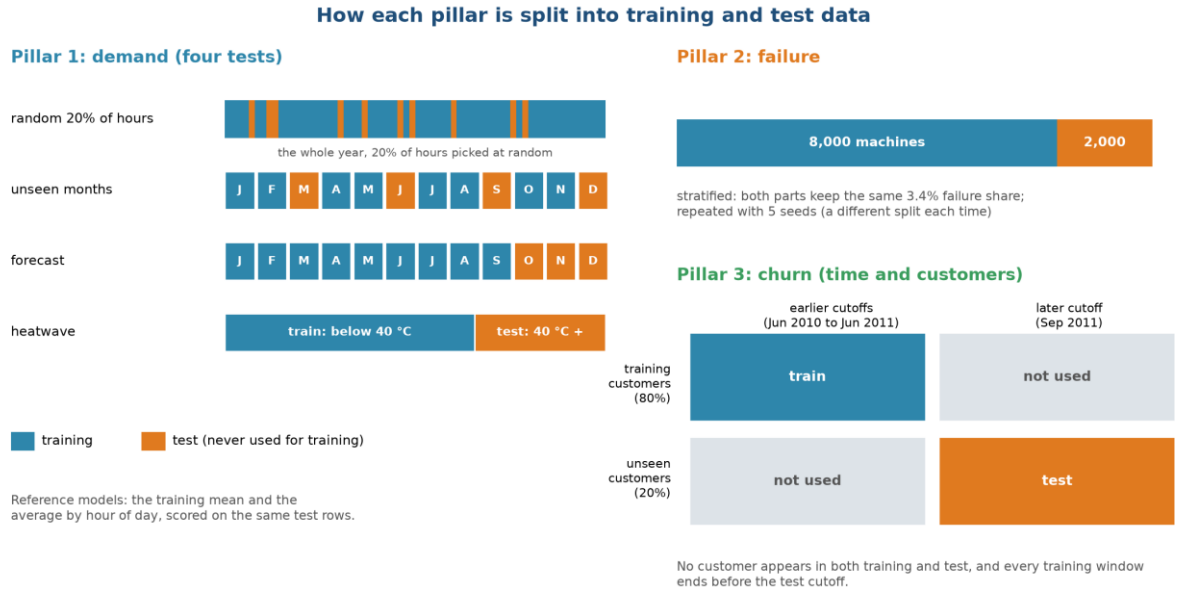


Figure 3.4. *How each pillar is split into training and test data*

Pillar 1 has four tests, each harder than the last. The first holds out a random 20% of hours. It is the easy case, since the neighbours of every test hour are in training, so we treat it as an optimistic reference. The second holds out whole months (March, June, September and December), so the model must predict months it never saw. The third trains on January to September and predicts October to December, which imitates a real forecast. The fourth trains on hours below 40 °C and tests on hours at or above 40 °C, which imitates a heatwave. The number of hours in training and test is 7,008 and 1,752 for the random split, 5,832 and 2,928 for unseen months, 6,552 and 2,208 for the forecast, and 7,936 and 824 for the heatwave.

Pillar 2 uses a stratified 80/20 split, with 8,000 machines for training and 2,000 for testing. Stratified means both parts hold the same 3.4% share of failures, so the test set has 68 failures. We repeat the split with five different seeds.

Pillar 3 splits customers and time at once. First, 20% of customers are picked at random and set aside. Training rows come from the earlier cutoffs and the 80% of customers. Test rows come from the later cutoff and the 20% of customers set aside. No customer appears on both sides, and no training window overlaps the test period. This leaves about 8,450 training rows and about 360 test rows, 118 of which are customers who lapse. The test set is small, so the scores vary from seed to seed, and we report that spread in stage 8.

3.8 Guards against leakage

Leakage means that information from the answer or the test period reaches the model during training. It produces scores that look excellent and fail in real use. Seven checks protect us, and six of them are automated tests that run before every release.

Risk	What we did	Test
Features that use the future (churn)	Features only look at invoices before the cutoff; the test deletes all later invoices and checks that no feature changes	features use only the past
Target built from the past (churn)	The target only looks at invoices after the cutoff	label comes from the future
Same customer on both sides	Customers are split before rows are built	test customers never appear in training
Overlapping windows	The last training window ends before the test cutoff	training cutoffs end before the test cutoff
Answer hidden in the inputs (failure)	The five failure-type columns are never inputs	failure features
Test statistics in the pipeline	Imputing and scaling are inside the model pipeline	by design
Choosing a threshold on test data	The alert threshold comes from out-of-fold training scores	by design

3.9 Known quirks we kept and report

Writing the cleaning rules made us notice several oddities. We kept them and list them here, because a reader should know about them.

- The temperature series in the HVAC file moves in steps at the start of each month (see the year overview in stage 2). We think it is simulated.
- 368 hours have a power value of exactly 0.10 kW. This looks like a floor in the simulation, and all of them are in idle mode.
- Nine failures in AI4I have no failure-cause flag set, and 24 machines have more than one cause flag.
- 100 of the 12,469 rows in the churn table (0.8%) count a customer as active, with a last purchase 90 days before the cutoff (rounded down to whole days), although the order count for the same 90-day window is zero. The two definitions disagree by part of a day at the boundary. We kept these rows because they are under 1% of the table.
- The retail data stops on 9 December 2011, so the last month is partial. Our test window ends on 8 December to avoid this.

3.10 Output of this stage

Output	Rows	Used for
HVAC table with short column names	8,760	Pillar 1
AI4I table with short column names	10,000	Pillar 2
Invoice table	44,870	Pillar 3, input to the churn builder
Churn snapshots, six cutoffs	12,469 rows in all (10,562 training cutoff rows, 1,907 test cutoff rows)	Pillar 3

Chapter 4. Exploratory data analysis

Lifecycle stage 4 of 10. Written by Dhruv Aggarwal (25911019).

4.1 What we wanted to learn

Exploratory data analysis (EDA) is the stage where we look at the cleaned data before fitting any model. We had three goals. First, to see what drives each target, so the features in stage 5 come from evidence and not from guesses. Second, to find problems that would mislead a model, such as strong overlaps between inputs or patterns that only hold in part of the data. Third, to decide which models are plausible: a relationship that is a straight line favours a linear model, while a threshold or a hump favours trees.

Every finding below is tied to a decision in a later stage. We use only the data that a model may legitimately see, and for Pillar 3 we use the churn table from stage 3.

4.2 Pillar 1: what drives HVAC power

Power by hour and month

Figure 4.1 is a heat map of average power for each month (columns) and hour of the day (rows).

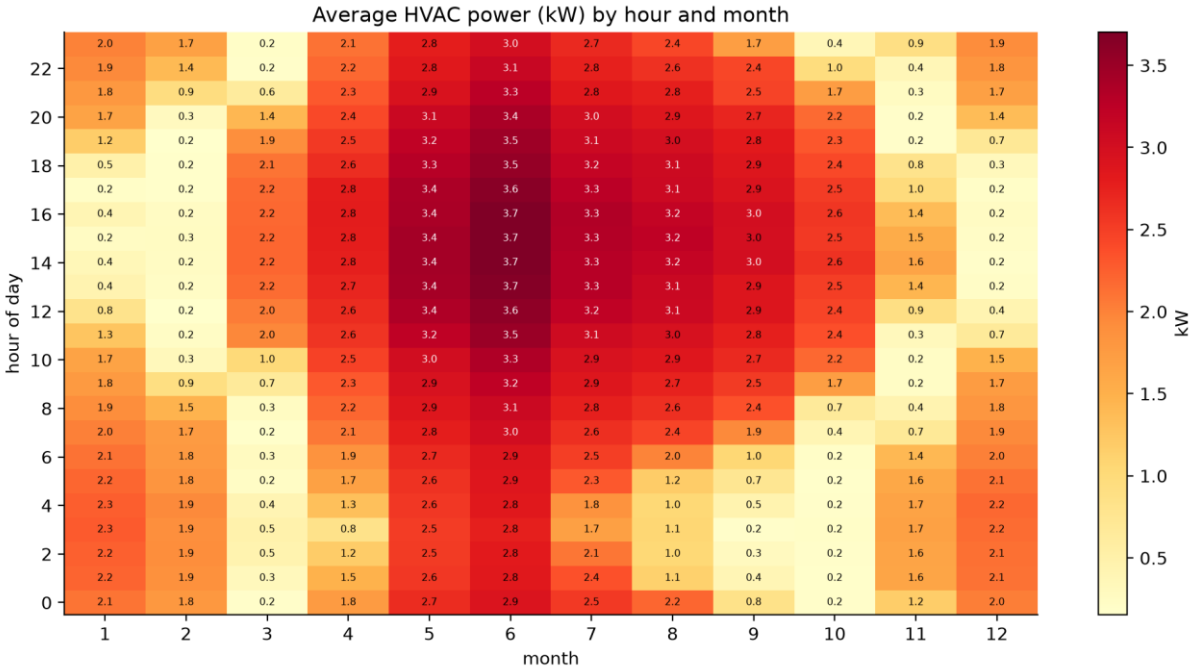


Figure 4.1. *Average HVAC power by hour and month*

Dark red cells are high power and pale yellow cells are close to zero. The map is dominated by the months: May to August are dark red all day, with a peak of 3.7 kW between 12:00 and 17:00 in June. January and December are medium, around 2 kW. February, March, October and November are mostly pale, which means the system sits in idle mode for most hours, with bursts of heating in the cold nights and mornings. Inside a month, the hour matters less than the month, because the daily range of temperature in this data is modest compared with the jump from one month to the next. This is a direct result of the monthly steps we saw in stage 2. It tells us that month alone would explain a good part of the power, and that a model trained without some months has to take its knowledge from the temperature itself.

Mode and temperature

Figure 4.2 looks at the HVAC mode.

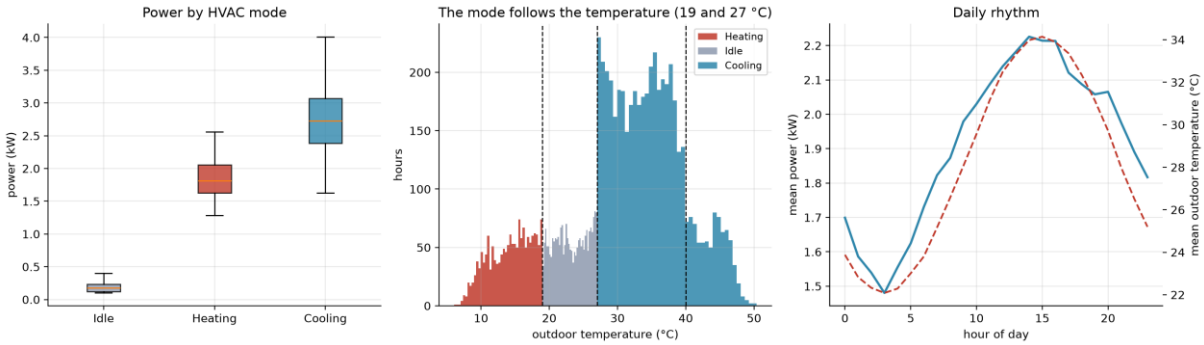


Figure 4.2. *Power by HVAC mode, the temperature behind each mode, and the daily rhythm*

The left panel shows power for each mode as a box plot. Idle hours draw 0.19 kW on average, heating hours 1.84 kW and cooling hours 2.73 kW, with no overlap between idle and the other two. The middle panel is a histogram of the outdoor temperature coloured by mode. The mode is a pure function of the temperature: heating below 19 °C, idle between 19 °C and 27 °C, cooling above 27 °C. The dotted lines at 19, 27 and 40 °C mark the mode switches and the heatwave threshold. The right panel shows the average power (solid) and outdoor temperature (dashed) over the 24 hours of a day. Power follows the temperature curve closely, from a low of 1.48 kW at 03:00 to a high of 2.23 kW at 14:00.

Two things follow from this. The mode column is almost redundant given the temperature, so including it does little harm and little good. More importantly, the 824 hours at or above 40 °C all lie in cooling mode, and the model must predict power for them from a temperature range that only exists in the test period. This is why the heatwave test is hard.

Which inputs matter

Figure 4.3 shows the correlation between every pair of columns.

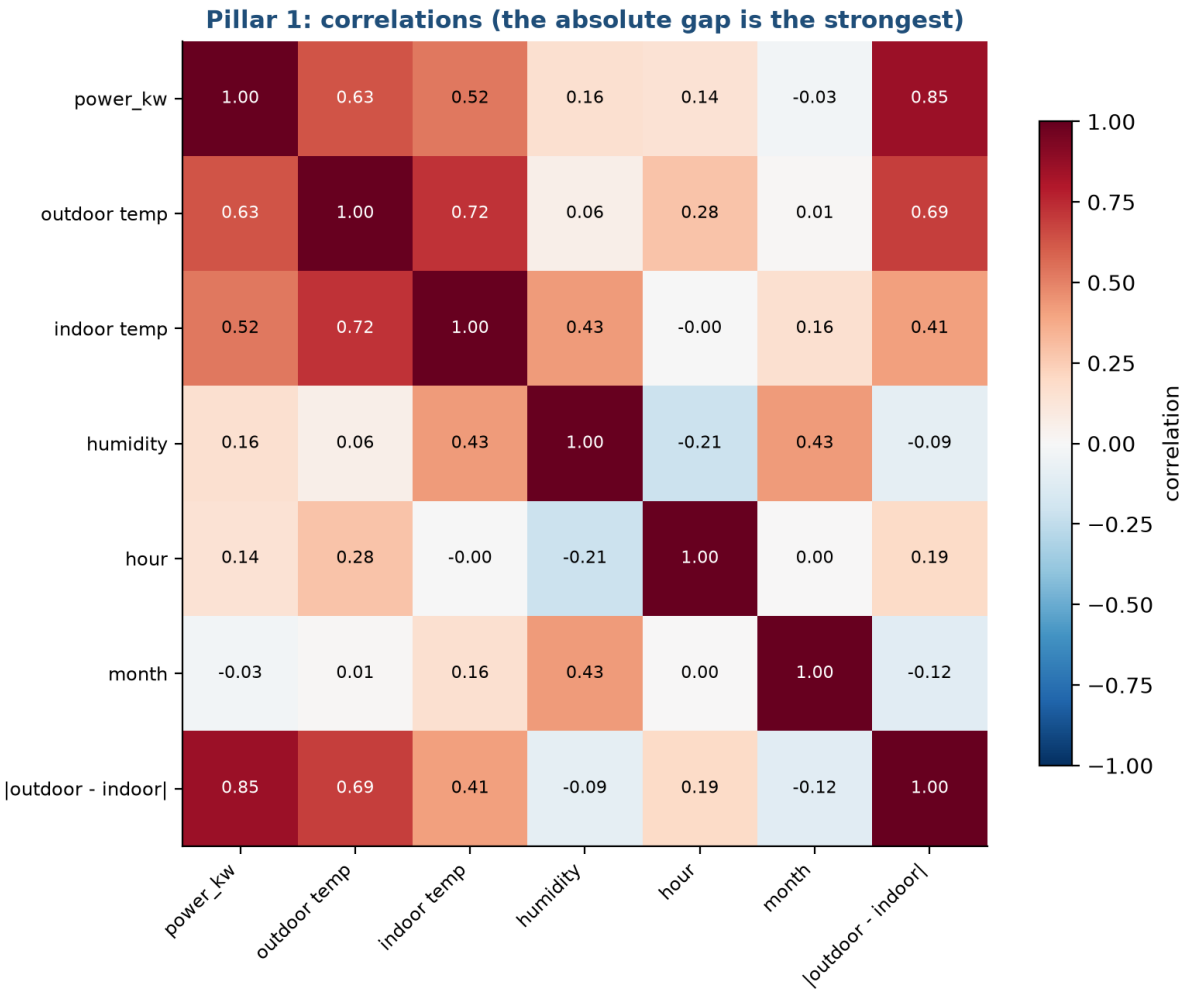


Figure 4.3. *Correlation between the Pillar 1 columns*

Each cell is the correlation coefficient between the column on the left and the column at the bottom, from -1 (opposite) to +1 (move together). Look at the first row, which is the correlation with power. The outdoor temperature gives 0.63 and the indoor temperature 0.52. Humidity (0.16) and hour of day (0.14) are weak, and month is close to zero (-0.03) because the relationship between month and power is a hump (high in summer, low in winter, with heating in between). The standout is the last column: the absolute gap between outdoor and indoor temperature, |outdoor - indoor|, has a correlation of 0.85 with power. Heating and cooling both cost power in proportion to how far the outdoor temperature is from the indoor setpoint, in either direction. A raw temperature misses this, because a cold day and a hot day both give a big gap but are at opposite ends of the temperature scale.

Decisions that come from this

- Add the absolute gap as a feature, and also the gap separated by mode (stage 5).
- A linear model with the gap should do well, and it should extrapolate, because the relationship is a straight line in the gap.
- A tree model has no way to go beyond the highest gap it saw. We expect it to fail in the heatwave test and we check it in stage 8.

4.3 Pillar 2: what separates failing machines

Sensor distributions

Figure 4.4 compares every sensor reading for machines that failed (red) and machines that did not (blue).

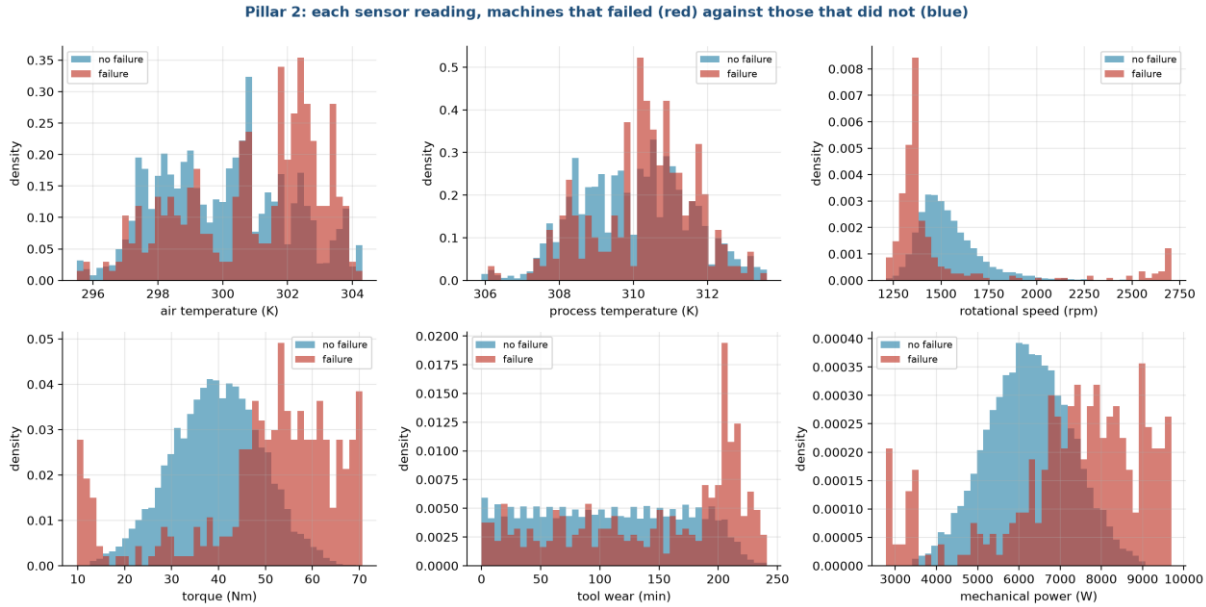


Figure 4.4. *Sensor readings of failing and healthy machines*

The panels are density histograms, so the two colours can be compared even though there are about 28 healthy machines for each failure. Where the red bars sit apart from the blue ones, that reading carries information.

- Torque: healthy machines centre around 40 Nm, while failures spread to both ends, with a cluster below 20 Nm and a large share above 50 Nm. Failure rates are 18.2% in the top decile of torque.
- Rotational speed: failures concentrate at low speed, near 1,300 rpm, and there is a small second group above 2,600 rpm. The lowest speed decile has a 16.7% failure rate.
- Tool wear: the two distributions look alike up to about 200 minutes, then failures pile up in the last stretch. The top decile has a 12.6% failure rate.
- Mechanical power, torque times speed, which we added as a derived column: failures sit at both extremes, below 4,000 W and above 8,000 W. The top decile has a 16.9% failure rate.
- Air and process temperature: failures lean toward higher temperatures, and the overlap is large.

The torque and power panels show a "U" shape: failure is more likely at both low and high values. A linear model draws one straight line, so it cannot capture a U. Tree models can. This foretells the large gap between logistic regression and the tree models that we see in stage 8.

Correlations, including the derived inputs

Figure 4.5 adds three engineered columns to the correlation map.

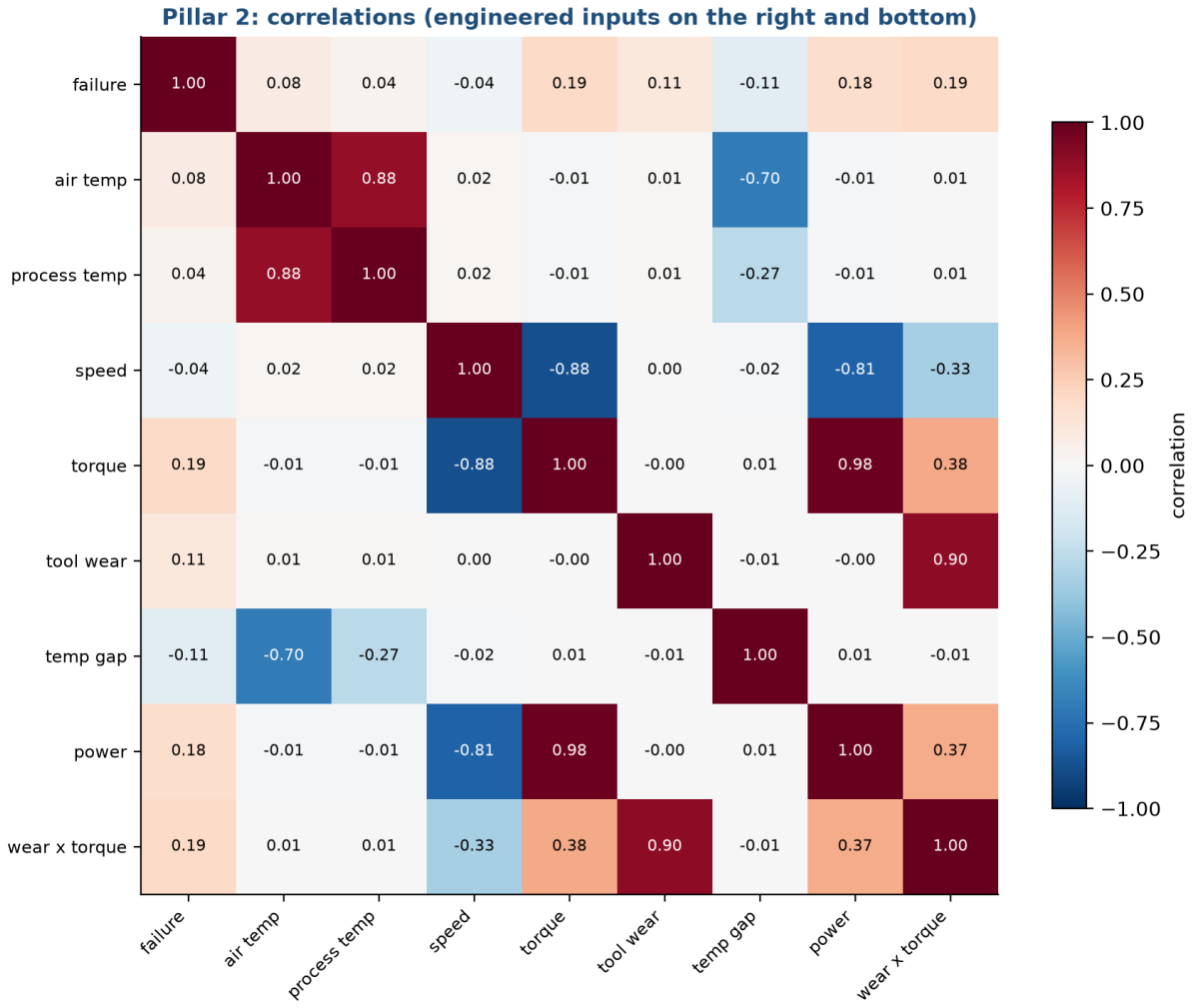


Figure 4.5. *Correlation between the Pillar 2 columns*

The first row gives the correlation with failure. All values are small: torque 0.19, wear times torque 0.19, mechanical power 0.18, tool wear 0.11, the temperature gap -0.11, air temperature 0.08. A correlation measures straight-line association, so these numbers understate the U-shaped effects in Figure 4.4. The rest of the map shows redundancy: air and process temperature move together (0.88), torque and speed are strongly opposite (-0.88), and mechanical power is almost the same as torque (0.98). We keep all of them. Ridge handles correlated inputs by shrinking their coefficients, and trees pick whichever column helps.

Decisions that come from this

- Add the temperature gap, mechanical power and wear times torque (stage 5). They sum up the physics behind heat, power and overstrain failures.
- Expect tree models to beat linear ones. Do not use accuracy as a metric.
- Do not use the failure-type columns as inputs, because they describe the outcome.

4.4 Pillar 3: how customers behave

Distributions

Figure 4.6 describes the customers in the retail data.

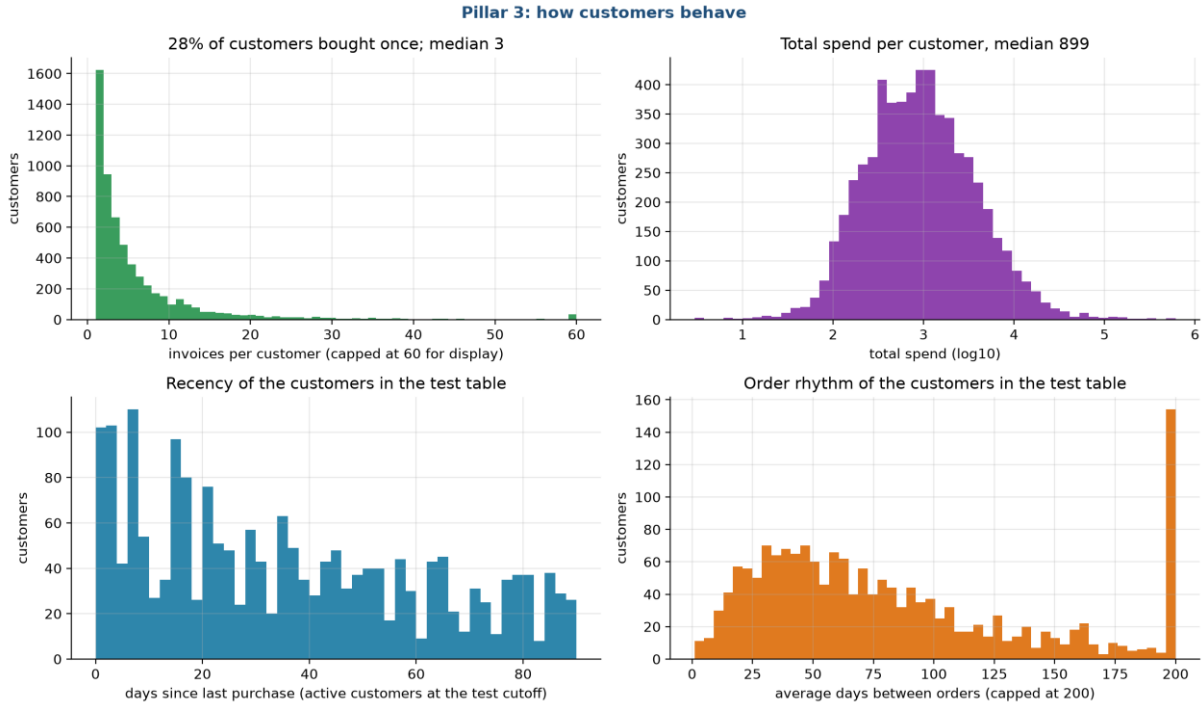


Figure 4.6. *Customer behaviour in Online Retail II*

The top left panel counts customers by the number of invoices. More than a quarter of the customers (28%) bought only once, and the median is three invoices, while a few buy hundreds of times. The top right panel shows total spend per customer on a log scale. The median is 899 currency units and the shape is a bell on the log axis, which is why we added log versions of money columns. The two lower panels describe the active customers in the test snapshot: days since the last purchase is spread evenly from 0 to 90 days with a lump near zero, and the average gap between orders ranges widely, with a pile at the right edge for customers who buy rarely (the axis is capped at 200 days).

Who lapses

Figure 4.7 splits the lapse rate by three customer features, using all six snapshots together.

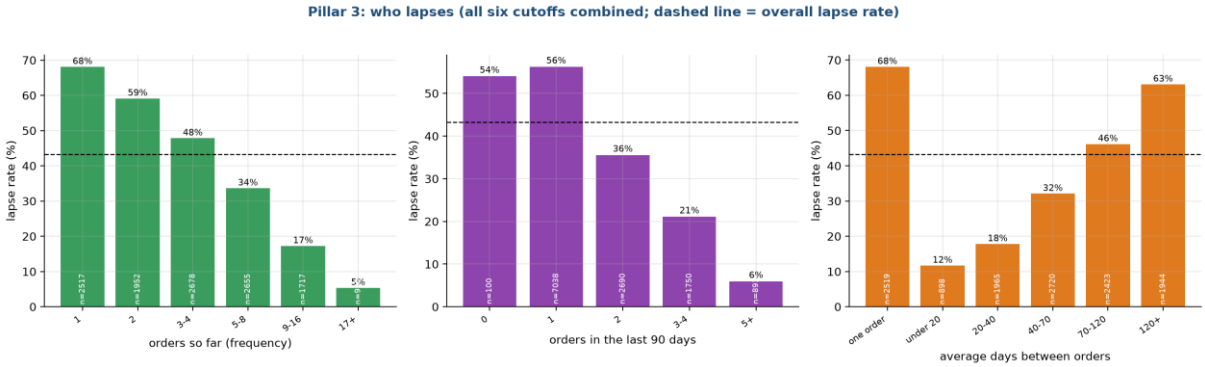


Figure 4.7. *Lapse rate by number of orders, recent orders and order rhythm*

The dashed line is the overall lapse rate of about 43%. The numbers inside the bars show how many customers fall in each group, and small groups are less reliable.

- By number of orders so far (left): a customer with one order lapses 68% of the time, with two orders 59%, with three or four 48%, and those with 17 or more lapse 5% of the time. Loyalty builds with each order.

- By orders in the last 90 days (middle): customers with no recent order (a boundary case of only 100 rows) or one order lapse 54% to 56%, those with two lapse 36%, three or four 21% and five or more 6%.
- By average days between orders (right): customers with one order only lapse 68%, those who reorder in under 20 days lapse 12%, and those who reorder every 120 days or more lapse 63%. A shorter rhythm predicts staying.

The effect is large in each panel, so a model has useful signal to find. What the panels do not show is how these features interact, or whether the signal in one overlaps the others.

Correlations with the target

Figure 4.8 gives the correlation among the churn features and with the target.

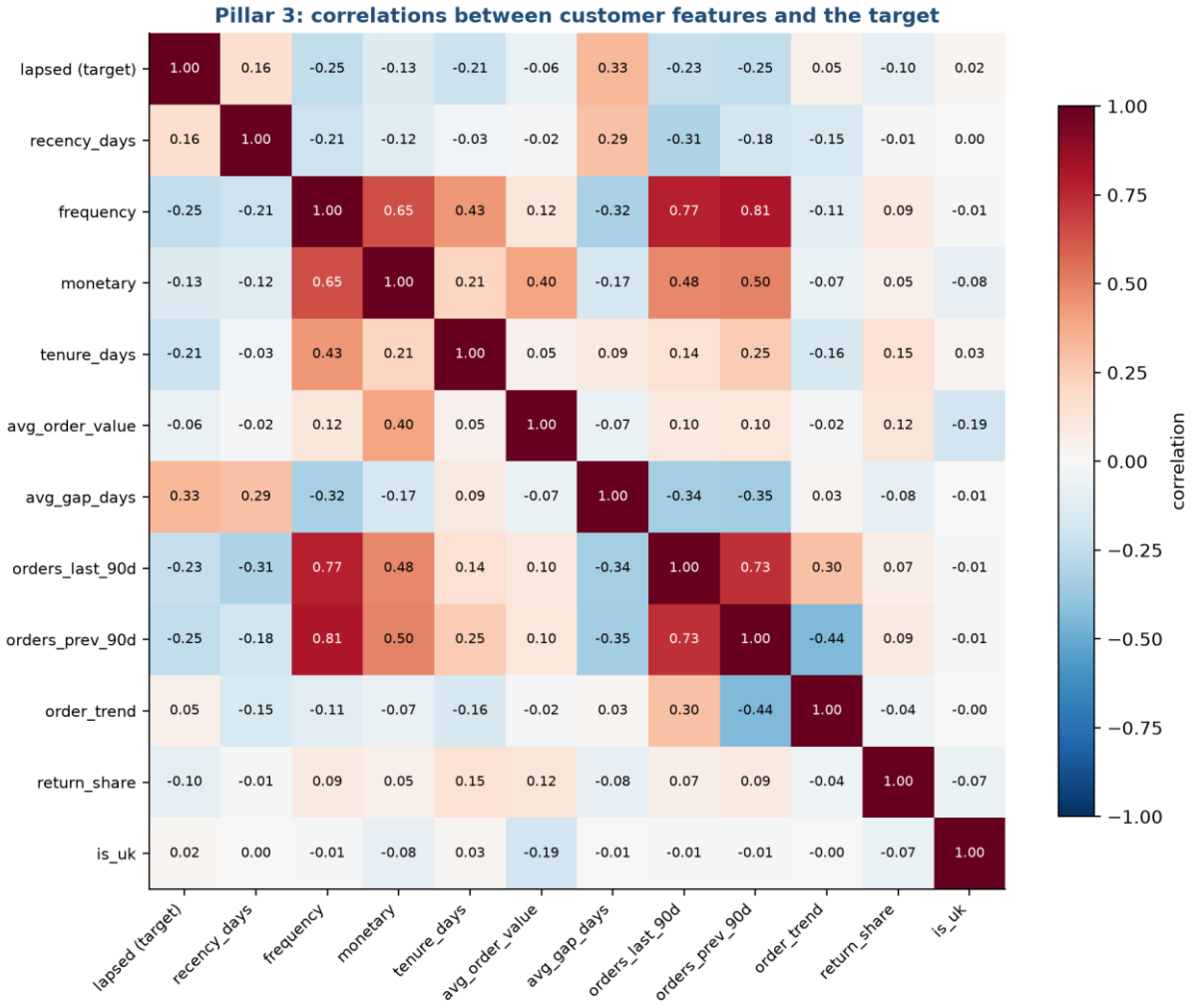


Figure 4.8. *Correlation between the Pillar 3 features and the lapse target*

In the first row, the strongest correlations with lapsing are the average gap between orders (0.33), the number of orders (frequency, -0.25), orders in the previous 90 days (-0.25), orders in the last 90 days (-0.23) and tenure (-0.21). The days since the last purchase, recency, has a correlation of only 0.16. The rule of thumb "call the customers who have been quiet longest" relies on exactly this weakest signal. Some features are tightly bound: frequency with orders in each 90-day window (0.77 and 0.81). Because the customers in the table are active in the last 90 days, recency is squeezed into a narrow range from 0 to 90 days, which limits how much it can tell.

Decisions that come from this

- Add the order rhythm, tenure, orders in two consecutive windows and their difference (stage 5).
- Keep log versions of the money and count columns, because of the skew.
- Do not expect a large gain from complex models. All the signal is in the customer's own order history, and the correlations are modest. This matches the result in stage 8, where every model reaches about the same score.

4.5 Summary of findings

Pillar	Main finding	Number	Consequence
1	Power is driven by the gap between outdoor and indoor temperature	correlation 0.85	Add the absolute gap; a linear model on it can extrapolate
1	Idle hours draw 0.19 kW, cooling 2.73 kW	by mode	Keep mode as an input
1	824 hours reach 40 °C or more, all in cooling mode	9.4% of hours	Use them as the heatwave test
2	Failure is U-shaped in torque, speed and power	top-decile failure rate 17% to 18% against 3.4% overall	Trees should beat linear models
2	No single input has a high correlation with failure	at most 0.19	Combine inputs; use PR-AUC
3	Lapse falls from 68% to 5% as order count rises	frequency	The signal is in order history
3	Recency is the weakest of the main signals	correlation 0.16	The recency rule should be a weak baseline
3	Money columns are very skewed	top 1% hold 32% of spend	Use log versions

4.6 Limits of this analysis

- We looked at relationships in the whole data set or in the training snapshots. Such plots can suggest a feature, but they cannot prove that it works on unseen data. Stage 8 tests that.
- Correlation measures only straight-line links. We added the decile failure rates and the group bars to show curved and threshold effects.
- The temperature steps in the HVAC file (stage 2) are probably an artefact of how the data was made, so the clean relationships we see may be cleaner than in real buildings.

Chapter 5. Feature engineering

Lifecycle stage 5 of 10. Written by Ankit Kumar (26911002).

5.1 What a feature is and why we build them

A model sees a table of numbers. Each column is a feature, and each row is one example (an hour, a machine or a customer). A model can only use what is in the table. If the useful quantity is a ratio, a difference or a count over a window, and the table holds only the raw parts, a simple model may never find it. Feature engineering means computing those quantities ourselves from knowledge of the problem.

Our rule for this stage is that every new feature must have a reason we can state in one sentence, and it must use only information available at prediction time. We compare two versions of the table for every pillar: the basic one, which holds the raw columns with minimal conversion, and the engineered one, which adds the new quantities. Comparing them tells us how much of any improvement comes from better inputs and how much from a better model.

Figure 5.1 summarises the steps for the three pillars.

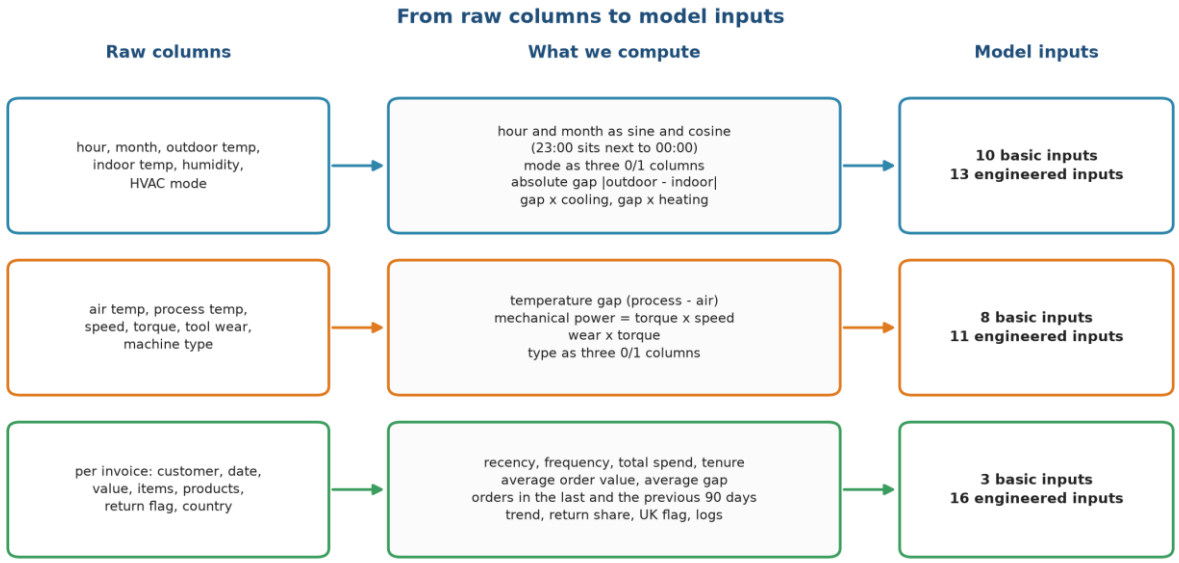


Figure 5.1. *From raw columns to model inputs*

Each row is one pillar. The left box lists the raw columns. The middle box lists what we compute. The right box gives the final count of inputs: Pillar 1 goes from 10 basic to 13 engineered inputs, Pillar 2 from 8 to 11, and Pillar 3 from 3 to 16.

5.2 Pillar 1: demand features

Basic inputs (10)

Input	How it is built	Why
Outdoor temperature	as is	the main driver of load
Indoor temperature	as is	the setpoint the system works toward
Humidity	as is	affects cooling effort
Hour, as sine and cosine	sin and cos of 2 x pi x hour / 24	so that 23:00 sits next to 00:00
Month, as sine and cosine	sin and cos of 2 x pi x (month - 1) / 12	so that December sits next to January
Mode (cooling, heating, idle)	three 0 or 1 columns	the system is in one of three states

Engineered inputs (3 more, 13 in all)

Input	How it is built	Why
Absolute gap	the absolute value of outdoor minus indoor temperature	the exploration in stage 4 showed a correlation of 0.85 with power
Gap while cooling	absolute gap times the cooling indicator	the cost per degree of gap differs between cooling and heating
Gap while heating	absolute gap times the heating indicator	same reason

Figure 5.2 shows why the absolute gap is useful.

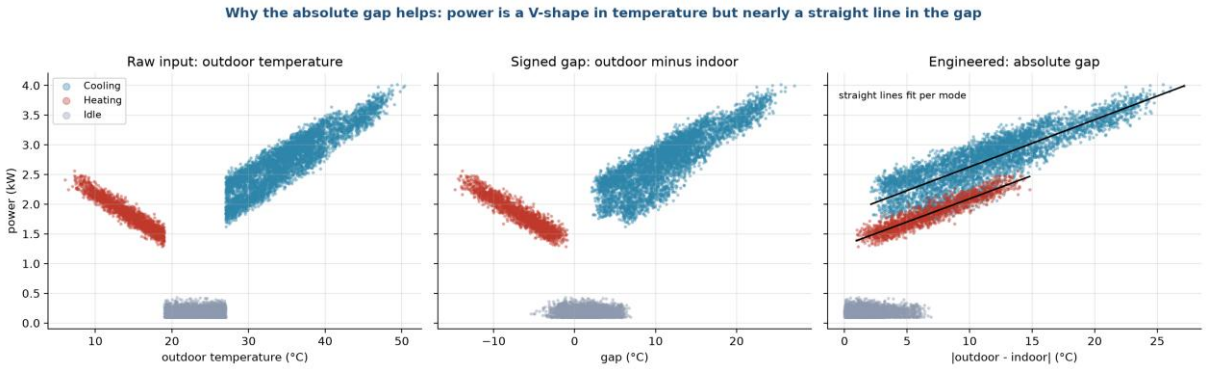


Figure 5.2. *Power against temperature, and against the absolute gap*

The left panel plots power against outdoor temperature. The points form a V: power is high when it is very cold, drops to near zero in the comfortable middle, and climbs again when it is hot. A straight line fitted to such a V is a poor description, because it would have to slope one way for the cold half and the other way for the hot half. The right panel plots power against the absolute gap. The V has folded into a single rising band, which a straight line follows well. The cooling and heating features then let the line have a different slope on each side. We did not add the mode thresholds (19 and 27 °C) as separate features, because the mode column already carries them.

5.3 Pillar 2: failure features

Basic inputs (8)

The five sensor readings (air temperature, process temperature, rotational speed, torque and tool wear) and the machine type as three 0 or 1 columns.

Engineered inputs (3 more, 11 in all)

Input	How it is built	Physical meaning
Temperature gap	process temperature minus air temperature	how much heat the process produces; a small gap means heat is not leaving
Mechanical power	torque times rotational speed times 2 pi / 60, in watts	the work done; very low and very high power both strain the machine
Wear times torque	tool wear in minutes times torque	overstrain: a worn tool under heavy load

We chose these because they are the physical quantities named in the dataset's own description of the failure modes (heat dissipation, power failure, overstrain). We did not add the failure thresholds themselves, because that would put the generator's rules directly into the table and make the problem trivial. Figure 5.3 shows how the failure rate changes across the range of each new feature.

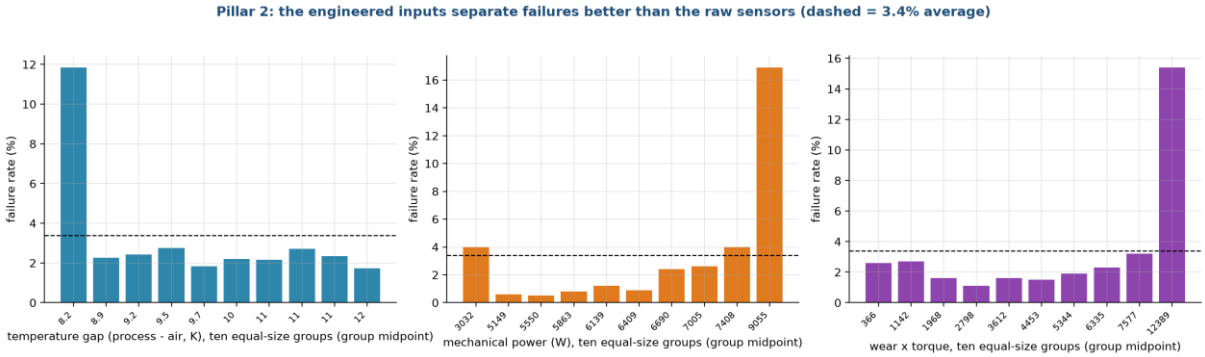


Figure 5.3. *Failure rate by decile of each engineered feature*

Each bar is one tenth of the machines, ordered from the lowest value to the highest, and its height is the share of those machines that failed. The overall rate is 3.4%. For the temperature gap, the lowest tenth (gaps near 8.2 K) has a failure rate of about 12%, more than three times the average, while the other nine tenths sit near 2%. For mechanical power, the failure rate is under 1% in the middle of the range, about 4% in the lowest tenth and about 17% in the highest, which is the U shape we saw in stage 4. For wear times torque, the rate stays between 1% and 3% for nine tenths of the machines and then jumps to about 15% in the highest tenth. These are the effects we hoped to expose. They are threshold-shaped or U-shaped, so a tree can use them more easily than a straight line, and a straight line gets only part of the benefit.

5.4 Pillar 3: customer features

Basic inputs (3)

The classic customer summary: recency (days since the last purchase), frequency (number of purchase invoices before the cutoff) and monetary value (total spend).

Engineered inputs (13 more, 16 in all)

Input	How it is built	Why
Tenure	days from first purchase to the cutoff	long-standing customers are steadier
Average order value	total spend divided by number of invoices	size of the typical order
Average gap between orders	tenure divided by (invoices - 1); empty for one-time buyers	the customer's own rhythm; the strongest correlation with lapsing (0.33)

Input	How it is built	Why
Average items and average products per invoice	means over invoices	how big and varied the baskets are
Orders in the last 90 days	count of invoices in the 90 days before the cutoff	recent activity
Orders in the 90 days before that	count in the window 90 to 180 days before	earlier activity to compare with
Order trend	last 90 days minus previous 90 days	whether activity is rising or falling
Return share	returns divided by (returns + purchases)	how often the customer sends goods back
UK flag	1 if the customer is in the United Kingdom	the shop sells mostly to the UK
Log spend, log frequency, log order value	natural log of one plus the value	spend is very skewed (stage 3)

All of these use only invoices dated before the cutoff, and a unit test deletes every invoice after the cutoff and checks that no feature changes. Figure 5.4 looks at three of the new features.

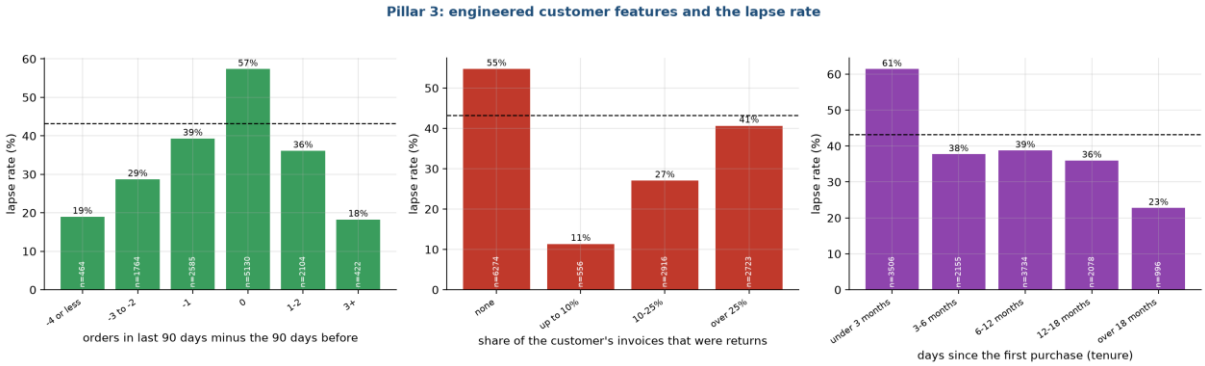


Figure 5.4. *Lapse rate by trend, return share and tenure*

The bars show the share of customers who go on to lapse within each group, and the dashed line is the overall rate of about 43%. The left panel is the order trend. Customers whose orders did not change between the two windows lapse most often (57%), and most of them are customers with a single recent order. Customers whose activity moved a lot in either direction lapse far less (19% for a large fall, 18% for a large rise), because a big change in either direction only happens to customers who order often. The pattern is not a straight line, so a plain linear model would miss it and a tree model may not. The middle panel is the return share. Customers with no returns at all lapse 55% of the time, while those with up to 10% returns lapse only 11%. We expected returns to signal unhappy customers. In this shop, sending goods back means the customer is engaged and orders often. The right panel is the tenure: customers in their first three months lapse 61% of the time and those with more than 18 months only 23%. Each effect is moderate, and the first two run against a first guess, which is a reason to measure them instead of assuming. The gain comes from using many small signals together.

5.5 Does feature engineering help?

For every pillar and model we ran the same test with basic and with engineered inputs. Figure 5.5 compares the two.

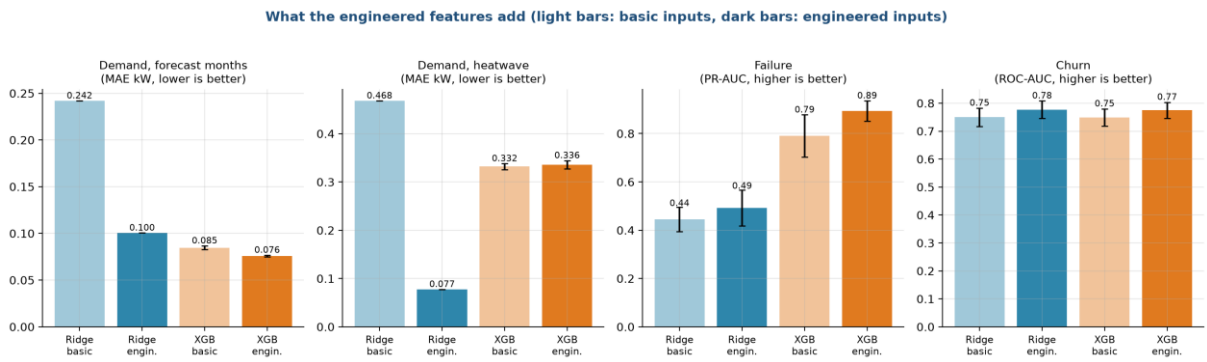


Figure 5.5. *Test score with basic and with engineered inputs*

The clearest result is the first group of bars. For demand, Ridge regression with basic inputs misses by 0.18 to 0.24 kW on the first three tests, and engineered inputs cut that error to 0.10 to 0.12 kW, about half. In the heatwave test the error falls from 0.47 to 0.08 kW. A linear model cannot build the absolute gap from its parts, so it needs it handed over. XGBoost gains almost nothing for demand, with 0.065 kW on the random test whether or not the gap is included, because trees can build such a shape themselves by splitting. For failure, the PR-AUC rises from 0.45 to 0.49 for Ridge and from 0.79 to 0.89 for XGBoost. For churn the ROC-AUC rises from 0.75 to 0.78 for both models.

Test	Model	Basic inputs	Engineered inputs
Demand, random split (MAE, kW)	Ridge	0.182	0.096
	XGBoost	0.065	0.065
Demand, unseen months (MAE, kW)	Ridge	0.224	0.118
	XGBoost	0.070	0.072
Demand, forecast (MAE, kW)	Ridge	0.242	0.100
	XGBoost	0.085	0.076
Demand, heatwave (MAE, kW)	Ridge	0.468	0.077
	XGBoost	0.332	0.336
Failure (PR-AUC)	Ridge	0.445	0.492
	XGBoost	0.790	0.893
Churn (ROC-AUC)	Ridge	0.750	0.777
	XGBoost	0.749	0.775

5.6 Which features the models rely on

After training, we looked at which inputs each model uses most. Figure 5.6 shows this for Pillars 2 and 3.

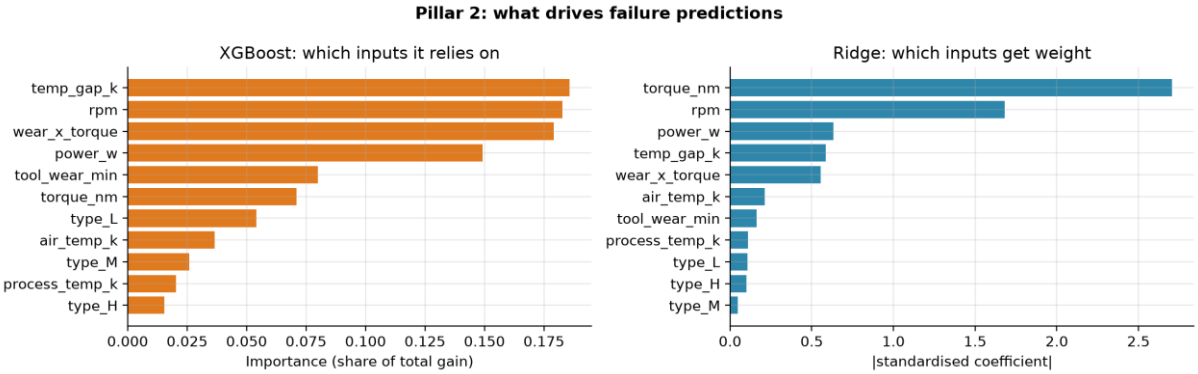


Figure 5.6. *Feature importance for failure and churn (XGBoost on the left, Ridge on the right)*

For failure, XGBoost spreads its reliance over the temperature gap, rotational speed, wear times torque and mechanical power, three of which are our engineered features. Ridge puts the largest weights on torque and speed and little on the others. The orange bars measure the share of the total improvement in the fit that each input provides, and the blue bars show the size of the standardised coefficient.

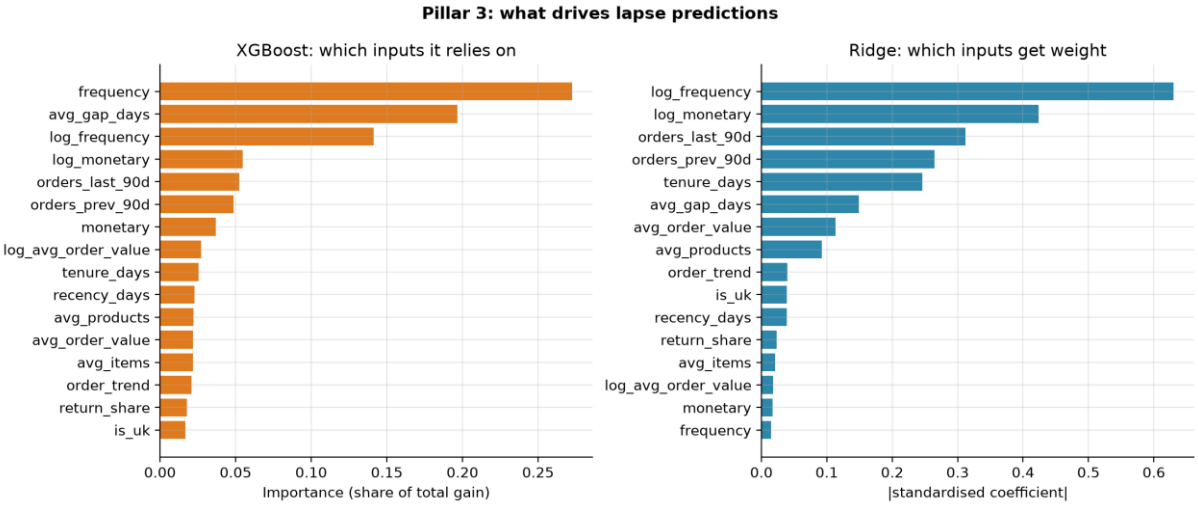


Figure 5.7. *Feature importance for churn*

For churn, both models rely most on how often the customer has ordered (frequency or its log) and, for XGBoost, the average gap between orders. Recency, the basis of the usual rule of thumb, ranks only tenth for XGBoost and eleventh for Ridge.

5.7 What we decided not to do

- No automatic feature search. Tools that generate hundreds of combinations can find a pattern that appears only by chance, and nobody can explain the result at a viva.
- No target encoding or other features computed from the answer. They need great care to avoid leaks, and we have no need for them.
- No deleting of correlated features. Ridge shrinks them, and trees pick one of them.
- No features from the test period. The future-based churn target is built separately from the features.

5.8 Checks on the features

Check	Result
No missing values in demand features	tested
Failure features exclude the failure-type columns	tested
Churn features use only invoices before the cutoff	tested
Imputation and scaling are learned from training rows only	in the model pipeline

Chapter 6. Model selection

Lifecycle stage 6 of 10. Written by Ankit Kumar (26911002).

6.1 The question of this stage

Which models should we try, and how will we decide which one to ship? A common mistake is to pick a fashionable model first and show that it works. We did the reverse. We set up a fair contest between simple and more complex models, with the rules fixed in advance, and let the numbers on unseen data choose. This document describes the candidates, the reasons for including each, the rule that picks the winner, and the outcome. Stage 7 then explains how the models were trained, and stage 8 gives the full test results.

6.2 The candidates

We use five kinds of predictor for each pillar. Figure 6.1 sums up their roles.

The model families we compare

Model	Role in this project	How it fits	Beyond the training range?	Main point
Rule of thumb	no learning	average by hour; most worn tool first; longest since last purchase	n/a	what a company does today
Linear / logistic regression	statistical baseline	one weight per input, no penalty	yes (straight line)	explainable; cannot bend
Ridge	our model 1	same, with an L2 penalty chosen by cross-validation	yes (straight line)	stable with many inputs
Random Forest	classic ML baseline	300 trees on random subsets, averaged	no (flat beyond training range)	few settings; handles thresholds
XGBoost	our model 2	400 or 300 small trees, each fixing the errors of the previous ones	no (flat beyond training range)	most accurate on tables

Figure 6.1. *The model families we compare*

The table in Figure 6.1 lists the rule of thumb, linear and logistic regression, Ridge, Random Forest and XGBoost. For each it gives its role in this project, how it fits, whether it can predict beyond the range of values it saw in training, and the main point to remember. The column about the training range turns out to matter a great deal for the heatwave test (section 6.4).

Candidate	Type	In plain words	Why it is in the contest
Rule of thumb	No learning	Demand: the average load for that hour. Failure: inspect the most worn tool first. Churn: call the customer who has been quiet longest.	The cheapest alternative. If a model cannot beat it, the product is not worth building.
Linear regression (demand) and logistic regression (failure, churn)	Statistical baseline	Each input gets one weight; the prediction is the weighted sum (passed through an S-shaped curve for yes or no tasks). Almost no penalty.	The standard tool of statistics. Its coefficients can be read directly.
Ridge	Our model 1	The same as above, but large weights are penalised, which keeps the model stable when inputs are correlated.	Simple, fast, hard to overfit, and able to extrapolate.
Random Forest	Classic machine learning baseline	300 decision trees, each trained on a random subset of rows and columns, with their answers averaged.	A strong, low-tuning model that handles thresholds and interactions.
XGBoost	Our model 2	Many small trees built one after another, each correcting the errors of the ones before it.	Usually the most accurate model on tables of this size.

How Ridge works

A linear model predicts power as a sum, such as: intercept + w1 x gap + w2 x humidity + and so on. Ordinary least squares picks the weights that make the training error as small as possible. With many inputs that move together, those weights can become huge and opposite in sign, and the model becomes unstable. Ridge adds a penalty: it minimises the training error plus a constant (alpha) times the sum of the squared weights. A larger alpha shrinks the weights toward zero. We choose alpha from seven values between 0.001 and 1,000 by five-fold cross-validation on the training data only. For the yes or no tasks, the same idea is applied to logistic regression, and the strength is set by the constant C, chosen from five values from 0.01 to 100.

How XGBoost works

A single decision tree splits the data by questions such as "is torque above 52 Nm?". XGBoost builds a sequence of small trees. The first tree makes a rough prediction. The second tree is fitted to the errors that remain, the third to the errors after the second, and so on. Each tree is scaled by a learning rate, and we add up all the trees. For demand we use 400 trees of depth 4 with a learning rate of 0.05, and for failure and churn 300 trees of depth 3. Each tree sees 80% of the rows and 80% of the columns. We set these once, and we did not tune them on test data. Stage 8 shows that the results change little over a wide range of settings.

6.3 Candidates we left out, and why

Model	Reason for leaving it out
Graph neural network	An earlier version used a GraphSAGE model. It never beat a simple model by a clear margin, and the synthetic graph held no extra information (see future work).
Neural network (ANN), LSTM, RNN	Our data is tabular and small: 8,760 rows, 10,000 rows and about 8,500 rows. Such models need much more data to beat gradient-boosted trees, take longer to train, and are hard to explain at a viva. The demand series is hourly weather-driven load, which the temperature explains better than the sequence order.
Support vector machines, k-nearest neighbours, Naive Bayes	They add little over the five we already compare and make the contest harder to follow.
Stacking and blending	They make the result hard to explain and hide which model did the work.
Automatic model search	It would tune on the data and would be hard to defend as honest.

6.4 The trees-and-extrapolation problem

One property of tree models decides the outcome of the heatwave test. Figure 6.2 shows a toy example that has nothing to do with HVAC.

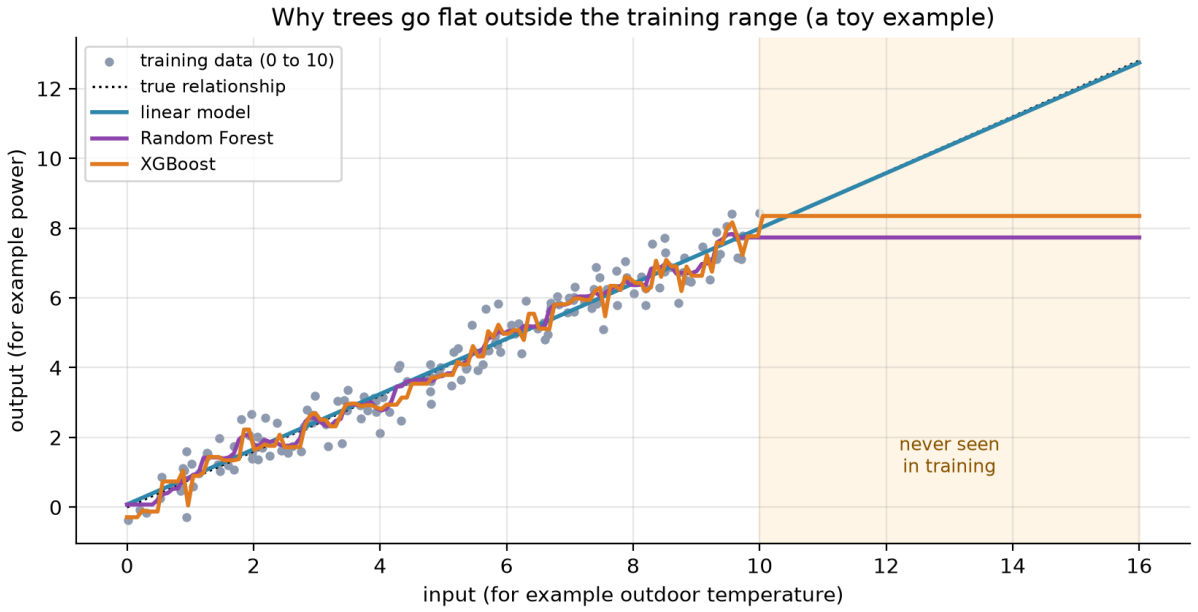


Figure 6.2. *Why trees stop rising outside the training range*

The grey dots are training points between 0 and 10, from a relationship that rises in a straight line. The dotted black line is the true relationship, which continues past 10. All three models fit the training range well. Beyond 10, in the shaded area, the linear model (blue) continues along the straight line and stays close to the truth. The Random Forest (purple) and XGBoost (orange) stay flat at the last value they saw. A tree can only output averages of training values, so it cannot predict anything higher than the highest values it was trained on. In our data, power grows in proportion to the gap between outdoor and indoor temperature, and the heatwave hours have larger gaps than any hour in training. We

therefore expected linear models to win the heatwave test and trees to lose it, and chose to make the heatwave test a part of the contest.

6.5 The selection rule

We wrote the rule before running the contest.

1. Run every candidate on every test, five times, with seeds 42 to 46.
2. For each test, take the mean score of each model over the five seeds. The model with the best mean is the leader.
3. Every model whose mean lies within one standard deviation of the leader is declared tied with it. The standard deviation is the leader's spread over the five seeds.
4. Among tied models, the simplest one wins. Simplest first: linear or logistic regression, Ridge, Random Forest, XGBoost.

The idea is to avoid choosing a complicated model for a difference that is just noise. Figure 6.3 shows the rule in action for the two classification pillars.

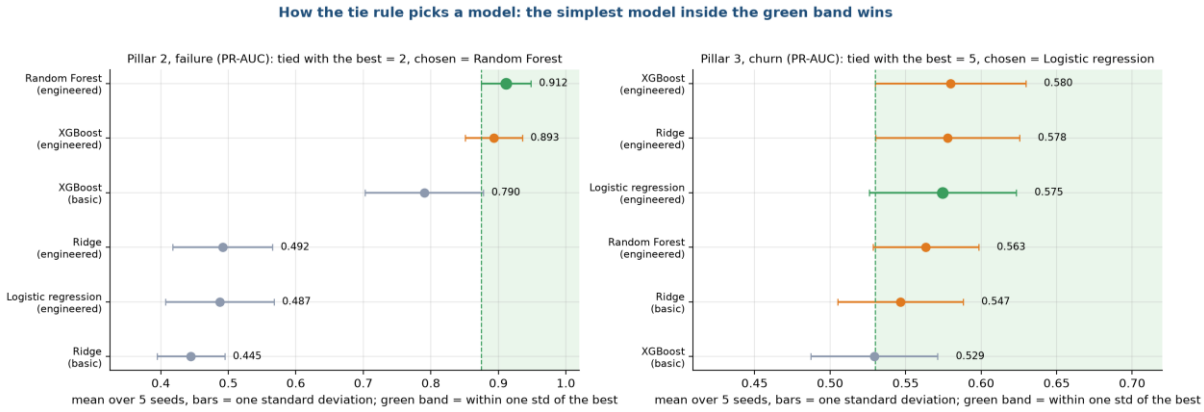


Figure 6.3. *How the tie rule picks a model for failure and churn*

Each dot is the mean over five seeds with a bar of one standard deviation. The green band is the leader's mean minus one standard deviation, up to the top. In the failure panel (left), the leader is Random Forest at 0.912 PR-AUC, and only XGBoost (0.893) falls inside the band, so the two are tied and Random Forest is the simpler. Logistic regression and Ridge sit at about 0.49 and are far outside the band. In the churn panel (right), the leader is XGBoost with a PR-AUC of 0.580, but five models fall in the band, from logistic regression at 0.575 to Ridge with basic inputs at 0.547. The simplest is logistic regression, so it wins. The spread is wide because the churn test set has only about 360 customers.

6.6 The contest result at a glance

Figure 6.4 shows every model on every test at once.

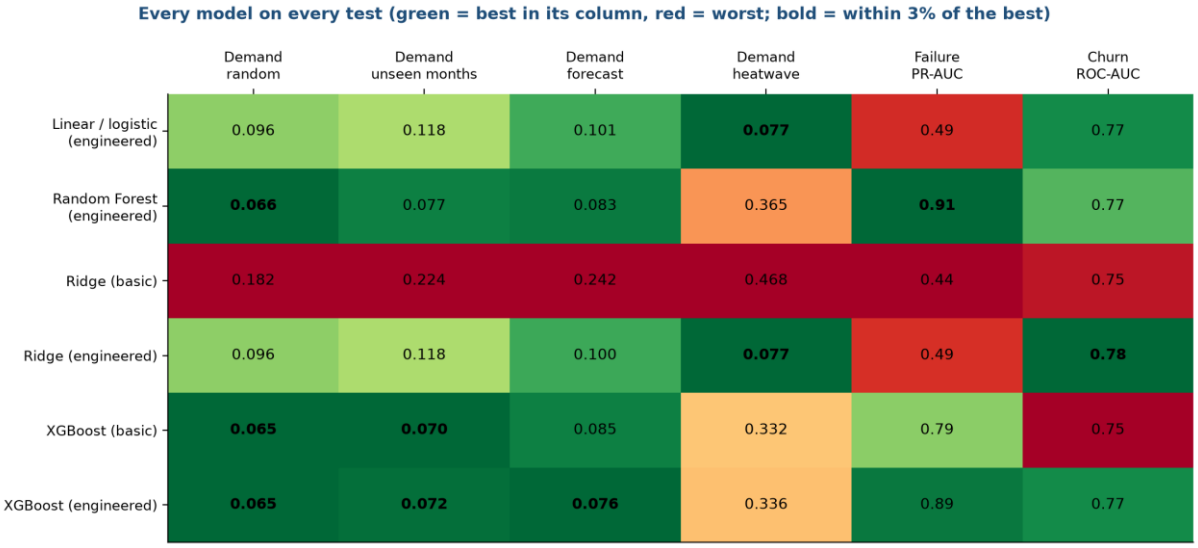


Figure 6.4. *Every model on every test*

Each row is a model and each column is a test. The number in the cell is the score: mean absolute error in kilowatts for the four demand tests (lower is better) and PR-AUC or ROC-AUC for failure and churn (higher is better). The colours show how each model compares with the others in its column, green for the best and red for the worst. Read the columns in turn.

- Demand, random, unseen months and forecast: the tree models (Random Forest and XGBoost) are green, with errors of 0.065 to 0.085 kW, and the linear models (Ridge and linear regression with engineered inputs) are in the middle with 0.10 to 0.12 kW. Ridge with basic inputs is red at 0.18 to 0.24 kW.
- Demand, heatwave: the picture flips. Linear regression and Ridge with engineered inputs are green at 0.077 kW. Random Forest (0.365) and XGBoost (0.33 to 0.34) are worse by a factor of four to five, and Ridge with basic inputs is the worst at 0.468.
- Failure: Random Forest (0.91) and XGBoost (0.89) lead. The linear models are red at 0.44 to 0.49.
- Churn: the five engineered or tied models sit between 0.77 and 0.78 ROC-AUC. Only the models with basic inputs are slightly worse.

6.7 What the rule selects

Test	Leader (mean)	Models tied with the leader	Pick by the rule	What the product uses
Demand, random split	XGBoost, basic inputs (0.065 kW)	XGBoost, engineered inputs	XGBoost	XGBoost
Demand, unseen months	XGBoost, basic inputs (0.070 kW)	none	XGBoost	XGBoost
Demand, forecast	XGBoost, engineered inputs (0.076 kW)	none	XGBoost	XGBoost
Demand, heatwave	Linear regression and Ridge (0.077 kW)	none	Linear regression (Ridge gives the same score)	Ridge
Failure	Random Forest (0.912 PR-AUC)	XGBoost	Random Forest	XGBoost
Churn	XGBoost (0.580 PR-AUC)	four more, see Figure 3	Logistic regression	Ridge

For demand the answer is a pair: XGBoost for ordinary weather and Ridge for heat beyond 40 °C or outside the temperature range of the training data. We call this the heatwave safeguard, and stage 9 describes how it works in the dashboard.

For failure, the rule alone picks Random Forest. The product uses XGBoost, which finished inside the same band (0.893 against 0.912), because it is one of the two models the group set out to compare and the gap is smaller than one standard deviation. Either would be a sound choice. A reader who prefers the strict rule would add Random Forest to the saved models, which is a small change because every model has the same interface.

For churn, the rule picks logistic regression. The dashboard uses the Ridge version of it, which scores 0.777 ROC-AUC against 0.775 for plain logistic regression. The two are the same method with a different penalty, and we kept Ridge because it is slightly more stable when the inputs are strongly correlated.

6.8 Summary of the decision

Pillar	Recommended model	Reason in one line
Demand	XGBoost, with Ridge as the heatwave safeguard	Trees are best on ordinary weather, and only a linear model extrapolates
Failure	XGBoost or Random Forest	Both reach about 0.9 PR-AUC, far above linear models at 0.49
Retention	Ridge (penalised logistic regression)	Every model ties, so we use a simple one

Chapter 7. Model training

Lifecycle stage 7 of 10. Written by Animesh Basak (26911001).

7.1 What training means here

Training is the step where a model looks at labelled examples and sets its internal numbers so that its predictions come close to the known answers. For Ridge those numbers are the weights on each input. For XGBoost they are the splits and leaf values of the trees. In this project, training is cheap: every model fits in under two seconds on a laptop processor. The work is in doing it fairly and repeatably. This document describes the protocol, the settings, the cost, how much data each model needs, and how sensitive the result is to the settings.

7.2 The training protocol

Every experiment follows the same six steps, shown in Figure 7.1.

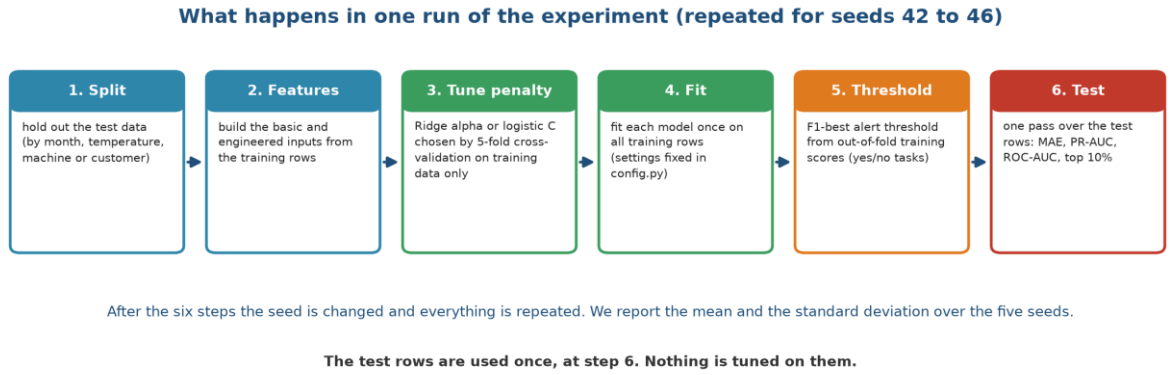


Figure 7.1. *What happens in one run of the experiment*

The six boxes read from left to right.

1. Split. The test data is set aside first (by month, by temperature, or by machine and customer), as described in stage 3.
2. Features. The basic and engineered inputs are built from the training rows.
3. Tune the penalty. Ridge picks its penalty (alpha for numbers, C for yes or no) by five-fold cross-validation on the training rows. That means the training rows are split into five parts, the model is fitted on four and scored on the fifth, and this is repeated five times. No test row is involved.
4. Fit. Each model is fitted once on all training rows. The tree settings are fixed in one configuration file and are not tuned.
5. Threshold. For the yes or no tasks, a model returns a risk score from 0 to 1. To raise an alert we need a cut-off. We choose the cut-off that gives the best F1 score on out-of-fold training scores, meaning scores that the model produced for rows it had not been trained on within the cross-validation. This keeps the test rows out of the choice.
6. Test. The fitted model scores the test rows once. We record the error for demand, and PR-AUC, ROC-AUC, F1, precision, recall and the top-10% capture for failure and churn.

After the six steps, the seed is changed and everything is repeated. We use five seeds, 42 to 46, and report the mean and the standard deviation. A seed changes the random 20% of hours, the stratified split of machines, the 20% of customers set aside, and the random choices inside the tree models. The

tests by month and by temperature do not depend on the seed, so only the model's own randomness varies there, and Ridge gives the same answer every time (a standard deviation of zero).

7.3 Settings

Setting	Value	Where it applies
Seeds	42, 43, 44, 45, 46	all experiments
Share of data held out	20% (rows for AI4I, customers for retail, hours for the random demand split)	all pillars
Held-out months	3, 6, 9, 12	demand, unseen months
Forecast months	10, 11, 12 (train on 1 to 9)	demand, forecast
Heatwave threshold	40 °C	demand, heatwave
Training cutoffs	1 Jun, 1 Sep, 1 Dec 2010; 1 Mar, 1 Jun 2011	churn
Test cutoff	9 Sep 2011	churn
Prediction window	90 days	churn
Ridge penalty candidates	0.001, 0.01, 0.1, 1, 10, 100, 1000	demand, 5-fold cross-validation
Logistic penalty candidates (C)	0.01, 0.1, 1, 10, 100	failure and churn, 5-fold cross-validation, scored by average precision
XGBoost for demand	400 trees, depth 4, learning rate 0.05, 80% of rows and of columns per tree	
XGBoost for failure and churn	300 trees, depth 3, learning rate 0.05, 80% of rows and of columns per tree	
Random Forest	300 trees, at least 5 rows per leaf	
Linear and logistic regression	no penalty (C = 1,000,000 for logistic)	statistical baseline

For the final models in the dashboard, cross-validation chose an alpha of 0.1 for the demand Ridge and a C of 0.1 for the failure and churn Ridge. The alert thresholds learned on training data are 0.44 for XGBoost on failure (0.26 for Ridge) and 0.34 for XGBoost on churn (0.35 for Ridge).

7.4 What the models cost to train

Figure 7.2 gives the time for one fit on a laptop processor.

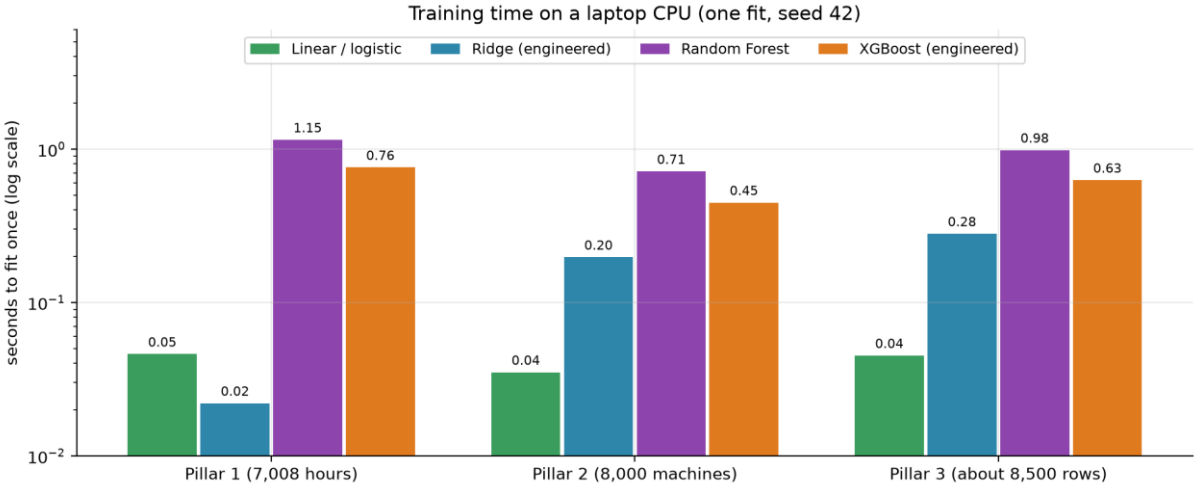


Figure 7.2. *Seconds to fit each model once*

The vertical axis is logarithmic, so each gridline is ten times the one below. The linear and logistic regressions take 0.02 to 0.05 seconds. Ridge takes 0.02 seconds for demand and 0.2 to 0.3 seconds for failure and churn, where it tries five penalties five times each. XGBoost takes 0.45 to 0.76 seconds and Random Forest 0.7 to 1.1 seconds. Nothing takes longer than 1.2 seconds. The complete experiment, which fits 18 models five times on each of the splits, writes all tables and figures, and trains the final models, runs in about 11 minutes. A single prediction takes 1.4 to 2.4 milliseconds, so the dashboard answers immediately. The saved file with the Ridge and XGBoost models for the three pillars is 1.4 MB.

These numbers matter for one reason: nothing in this project needs a graphics card, a cluster or a long wait. Every member can rerun it before the viva.

7.5 How much data does each model need?

A learning curve shows the test score as the training set grows. If the curve is still rising at the right-hand end, more data would help. If it is flat, the model has learned what it can. Figure 7.3 shows the curves for the three pillars.

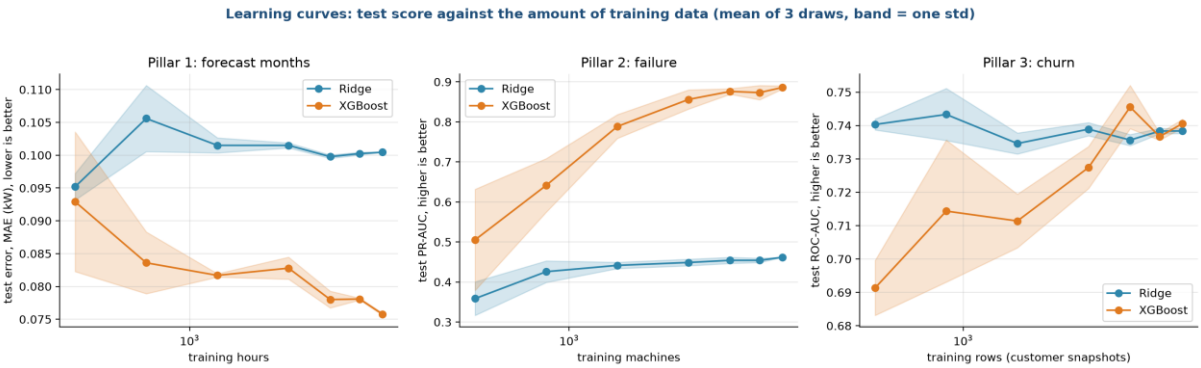


Figure 7.3. *Learning curves: test score against the amount of training data*

Each point is the mean of three random draws of the training rows, and the shaded band is one standard deviation. The axis for the training size is logarithmic.

- Demand, forecast months (left): the error of Ridge stays near 0.10 kW from about 600 hours up, so it has learned its straight line early. XGBoost starts at 0.09 kW with few hours, gets down to about 0.076 kW with all 6,552 hours, and keeps improving slowly. It needs more data than Ridge, but even 600 hours is enough to match it.

- Failure (middle): Ridge gains little, from 0.36 to 0.46 PR-AUC, because its limit is the straight-line form and not the data. XGBoost climbs from 0.5 with 400 machines to 0.88 with 8,000. The curve flattens at the right but has not fully levelled, so more machines would probably help a little.
- Churn (right): Ridge is flat near 0.74 ROC-AUC from the smallest size. XGBoost starts at 0.69 and reaches the same 0.74 only with the full set of rows. It never passes Ridge.

The learning curves explain much of what we see in stage 8. The tree models need data to build their splits, and they have enough for failure, where the signal is strong and shaped like thresholds. For churn there is no extra signal beyond what a straight line captures.

7.6 Sensitivity to the settings

We did not tune the settings on test data, but a reader may fairly ask whether a lucky setting drives the results. Figure 7.4 varies the main settings one at a time, with the other settings fixed and seed 42.

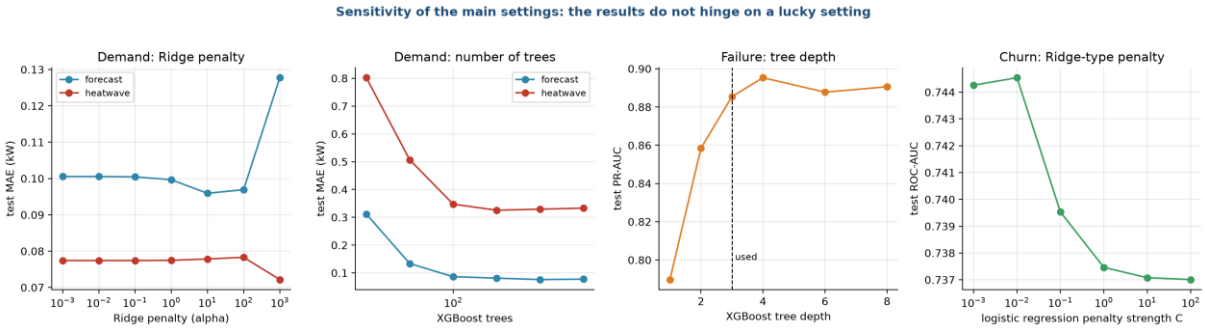


Figure 7.4. *Sensitivity of the main settings*

- Ridge penalty for demand (first panel): across penalties from 0.001 to 100, the error on the forecast test moves from 0.100 to 0.097 kW and on the heatwave test stays at 0.077 kW. Only at 1,000 does the forecast error rise to 0.128 kW. The cross-validated choice of 0.1 sits in the flat region.
- Number of trees for demand (second panel): with 25 trees the error is large, 0.31 kW on the forecast and 0.80 kW in the heatwave. By 100 trees the forecast error is 0.085 kW and it stays near 0.076 to 0.08 kW up to 800 trees. In the heatwave, the error stays between 0.32 and 0.35 kW no matter how many trees we add, which confirms that the failure is not a matter of settings.
- Tree depth for failure (third panel): depth 1 gives a PR-AUC of 0.79, depth 2 gives 0.86 and depth 3 (our choice) gives 0.885. Deeper trees reach 0.89, which is within noise. The dashed line marks our setting.
- Penalty for churn (fourth panel): the ROC-AUC changes only from 0.745 to 0.737 across penalties from very strong to almost none. The vertical axis spans less than 0.01, so the panel looks steep, and the effect is small.

The results do not depend on a particular setting. This supports our decision to fix the settings in advance.

7.7 Training the final models

The experiments above fit models on training rows and score them on test rows. The dashboard needs models that have seen all the data, so after the experiments we refit the Ridge and XGBoost models for each pillar on every row and save them in one file. The thresholds for the alerts are taken from the training scores. These models are only used for demonstration in the dashboard. Their scores come

from the experiments, not from the final refit, which would be unfair because the model has seen the rows (the retention table in stage 9 says so).

7.8 Reproducibility

- All settings are in one configuration file, and no setting is changed inside other files.
- Every random choice uses a seed. Rerunning the experiment gives the same tables.
- The data files are small and part of the project folder.
- The pipeline writes a log file, a summary table of means and standard deviations, a table with every run, a table with the model chosen for each test, and the figures.
- A quick mode runs one seed in a couple of minutes and writes to a separate folder, so it can never overwrite the real results.
- Seventeen automated tests check the data, the splits, the leakage rules, the metrics, a short end-to-end run and the monitoring functions.

7.9 Problems we hit and what we changed

Problem	What we did
A quick check overwrote the real results	Quick mode now writes to its own folder
An early churn label used its own input	We rebuilt the target from the future (stage 3)
Accuracy-like scores looked impressive for failure	We switched to PR-AUC and top-10% capture (stage 1)
Running the tests during the full experiment slowed both	We do not run them at the same time

Chapter 8. Model evaluation and tuning

Lifecycle stage 8 of 10. Written by Animesh Basak (26911001).

8.1 How we evaluate

Every score in this document comes from data the model did not see during training, and every score is the mean over five seeds with its standard deviation. We do not tune anything on the test data. The tests are described in stage 3 and the training protocol in stage 7. Here we explain the metrics, report the results pillar by pillar, look at the errors, and say what the results mean for the product.

The metrics

Metric	Used for	What it measures	Good value
MAE, mean absolute error	Demand	The average size of the miss in kW, ignoring direction	Low
RMSE	Demand	Like MAE but large misses count more	Low
R squared	Demand	The share of the variation in power that the model explains; 0 is the same as always predicting the mean, and a negative value is worse than that	Close to 1
Peak-hour MAE	Demand	The MAE on the busiest 10% of hours	Low
ROC-AUC	Failure, churn	The chance that a randomly chosen positive case gets a higher risk score than a randomly chosen negative one	Close to 1 (0.5 is a coin flip)
PR-AUC (average precision)	Failure, churn	How well the model keeps precision high while finding more positives; it is the honest metric for rare events	Above the base rate (0.034 for failure, about 0.32 for churn)
Precision	Failure, churn	Of the cases we flag, the share that really are positive	High
Recall	Failure, churn	Of the real positives, the share we flag	High
F1	Failure, churn	The balance of precision and recall at one threshold	High
Top-10% capture	Failure, churn	The share of all positives found in the 10% of cases with the highest risk	High (0.10 for a random list)
Lift	Churn	Top-10% capture divided by the share of customers contacted	Above 1

8.2 Pillar 1: demand

Table 8.1 gives the mean absolute error in kW for every model on every test. The reference rules never see the weather, and the other models use the engineered inputs unless noted.

Table 8.1. Mean absolute error (kW) of every demand model on every test

Test	Training mean (rule)	Average by hour (rule)	Linear regression	Ridge, basic	Ridge	Random Forest	XGBoost, basic	XGBoost
Random 20% of hours	0.942	0.934	0.096	0.182	0.096	0.066	0.065	0.065
Unseen months 3, 6, 9, 12	1.017	0.995	0.118	0.224	0.118	0.077	0.070	0.072
Forecast, months 10 to 12	1.012	1.024	0.101	0.242	0.100	0.083	0.085	0.076
Heatwave, 40 °C and above	1.648	1.574	0.077	0.468	0.077	0.365	0.332	0.336

The standard deviation over the five seeds is 0.001 to 0.002 kW on the random test, 0.009 or less in the heatwave for XGBoost, and exactly zero for the linear models on the three tests that do not depend on the seed. Table 8.2 adds the explained variation.

Table 8.2. Explained variation (R squared) for demand

Test	Ridge R squared	XGBoost R squared	Rule R squared (average by hour)
Random	0.987	0.995	0.037
Unseen months	0.982	0.994	0.056
Forecast	0.981	0.990	-1.040
Heatwave	0.800	-2.281	-52.280

Three findings stand out.

First, any model that sees the weather beats the rules of thumb by a factor of ten or more. The average mean power is 1.92 kW, so an error of 0.065 kW is 3.4% of the load, while the rule of thumb is wrong by about half of the load.

Second, trees are better than a straight line in ordinary conditions. XGBoost beats Ridge by 32%, 39% and 24% in the first three tests (Figure 8.8 below). The straight line cannot capture how the relationship bends at the mode switches. The model with basic inputs and Ridge is the weakest, which again shows that the engineered gap is worth having for linear models.

Third, in the heatwave the order reverses. XGBoost is wrong by 0.336 kW and has a negative R squared of -2.28, which is worse than predicting the average of the heatwave hours. Ridge is wrong by 0.077 kW with an R squared of 0.80. Random Forest behaves like XGBoost (0.365 kW).

Where the errors are

Figure 8.1 looks at the heatwave and forecast residuals, meaning actual power minus predicted power.

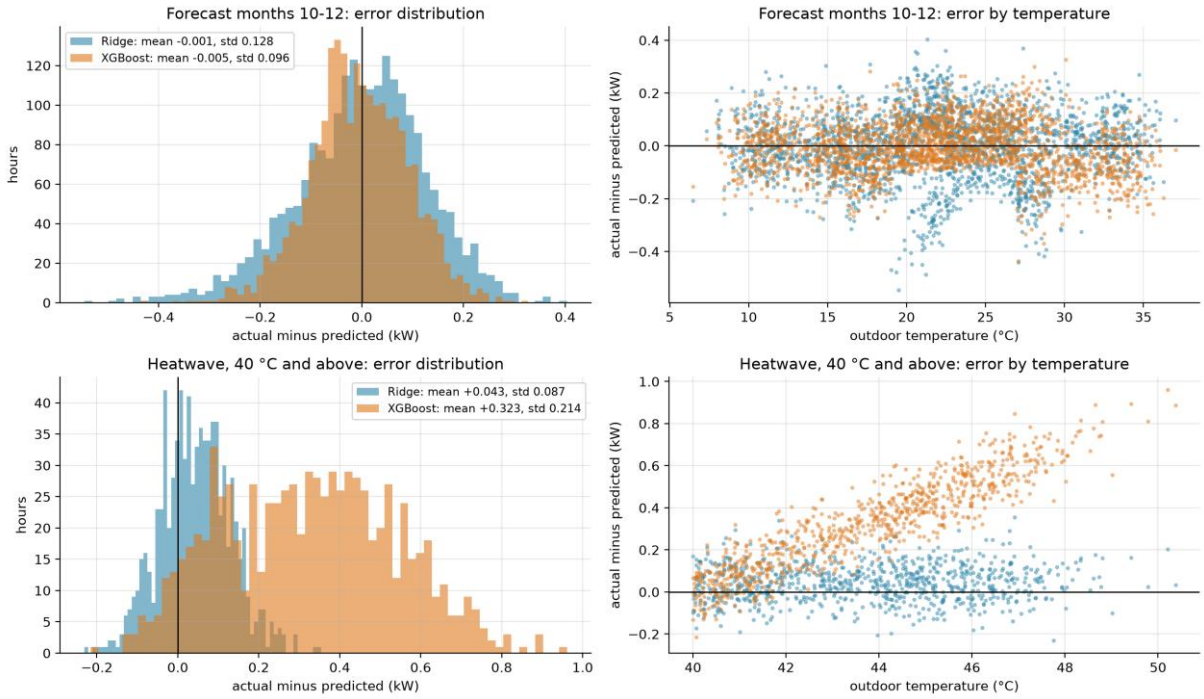


Figure 8.1. *Residuals of Ridge and XGBoost in the forecast months and in the heatwave*

The top row is the forecast test, months 10 to 12. The left panel is a histogram of the residuals. Both models are centred on zero (mean -0.001 kW for Ridge and -0.005 kW for XGBoost), and XGBoost is narrower, with a standard deviation of 0.096 kW against 0.128 kW. The right panel plots the residual against the outdoor temperature. Ridge shows a slight pattern, with larger negative misses near 20 °C and near 28 °C, which are close to the places where the mode changes. XGBoost has no pattern.

The bottom row is the heatwave test. The residual of XGBoost (orange) has an average of +0.32 kW, so it underestimates the power. The right panel shows why: the residual rises with temperature, from close to zero at 40 °C to 0.8 kW and more near 50 °C. The model stays flat at the highest power it saw in training while the real power keeps rising. Ridge (blue) is centred near zero with a small average of +0.04 kW at every temperature, because it extends the straight line.

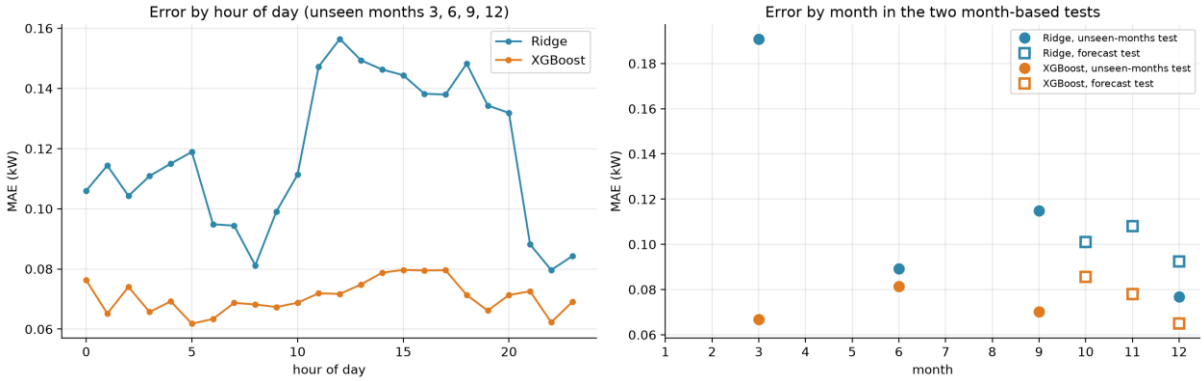


Figure 8.2. *Error by hour of day and by month*

Figure 8.2 shows error against time. On the left, the average error by hour of day in the unseen-months test is 0.06 to 0.08 kW for XGBoost at every hour, but for Ridge it is 0.08 to 0.12 kW at night and 0.14 to 0.16 kW between 11:00 and 20:00. The linear model finds the afternoon harder, when the temperature is high and the bends in the relationship matter most. On the right, each marker is the error in one held-out month. Ridge has its largest error in March (0.19 kW), a month that is mostly idle

in this data, which the straight line captures poorly. XGBoost stays between 0.065 and 0.085 kW in every held-out month.

What this means for the Demand Planner

A manager reading the Demand Planner sees an hourly curve that is right to within about 0.07 kW in ordinary weather, and to within 0.08 kW in a heatwave, because the dashboard switches to Ridge above 40 °C. Without the switch, the heatwave curve would be low by about 0.3 kW per hour, and it would be low by the largest amount on the hottest days, which are the days the manager cares about most.

8.3 Pillar 2: failure

Table 8.3 gives the mean and standard deviation over five seeds on the 2,000 held-out machines.

Table 8.3. *Failure prediction scores on 2,000 held-out machines (mean ± standard deviation over five seeds)*

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	Failures in top 10%
Rule: most worn tool first	0.655 ± 0.027	0.123 ± 0.013	0.197 ± 0.011	0.158 ± 0.013	0.262 ± 0.012	0.365 ± 0.024
Logistic regression	0.930 ± 0.021	0.487 ± 0.080	0.480 ± 0.059	0.457 ± 0.065	0.512 ± 0.075	0.744 ± 0.069
Ridge, basic inputs	0.898 ± 0.022	0.445 ± 0.050	0.470 ± 0.048	0.456 ± 0.060	0.497 ± 0.085	0.662 ± 0.067
Ridge	0.925 ± 0.021	0.492 ± 0.074	0.489 ± 0.059	0.514 ± 0.088	0.471 ± 0.045	0.732 ± 0.069
Random Forest	0.988 ± 0.007	0.912 ± 0.037	0.887 ± 0.041	0.927 ± 0.046	0.850 ± 0.041	0.941 ± 0.040
XGBoost, basic inputs	0.980 ± 0.009	0.790 ± 0.088	0.727 ± 0.094	0.726 ± 0.138	0.738 ± 0.070	0.912 ± 0.039
XGBoost	0.987 ± 0.007	0.893 ± 0.042	0.851 ± 0.045	0.901 ± 0.039	0.806 ± 0.052	0.938 ± 0.035

The base rate is 3.4%, so a random guess has a PR-AUC of 0.034. XGBoost reaches 0.89 and Random Forest 0.91. The linear models reach 0.49, which is still ten times better than random but half of what the trees achieve. In the top 10% of machines, XGBoost holds 94% of all failures, against 73% for Ridge, 74% for logistic regression and 36% for the rule of thumb.

Ranking quality

Figure 8.3 shows the ROC curves and the effect of the alert threshold.

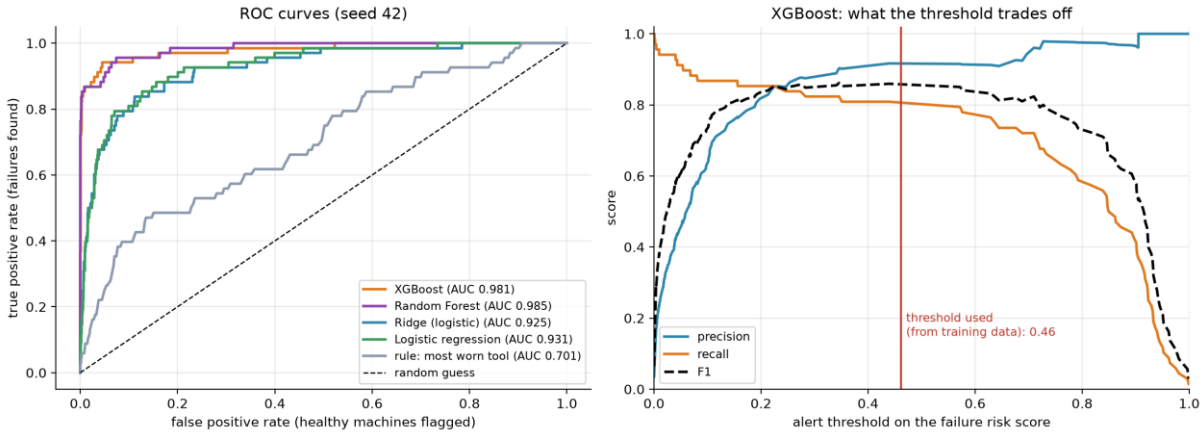


Figure 8.3. *ROC curves, and the trade-off at different thresholds*

The left panel plots the share of failures found against the share of healthy machines flagged, for every possible threshold. A curve that hugs the top-left corner is better. Random Forest (AUC 0.985) and

XGBoost (0.981) rise almost vertically: they find about 85% of the failures before they flag any noticeable number of healthy machines. Ridge (0.925) and logistic regression (0.931) rise more slowly. The rule that sorts by tool wear (0.701) is far from the corner and not much above the diagonal, which stands for random guessing. The right panel follows one model, XGBoost, as the threshold on its risk score moves from 0 to 1. At low thresholds the recall (orange) is high and the precision (blue) is low, since almost every machine is flagged. At high thresholds the opposite happens. The F1 score (dashed) is flat and high between about 0.2 and 0.6, so the result does not depend on a precise threshold. The red line marks 0.46, the value chosen from training data for this seed.

What the alerts look like

Figure 8.4 shows the confusion matrices for the two failure models and the two churn models at their own thresholds, for seed 42.

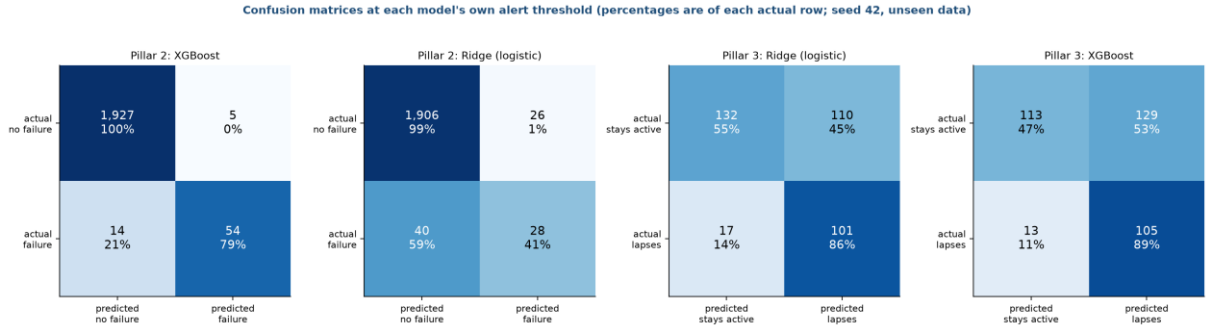


Figure 8.4. *Confusion matrices at each model's own threshold*

Each square counts the cases in one outcome. The two left panels are for failure. XGBoost flags 59 machines out of 2,000. Of those, 54 really fail (precision 91%) and 5 do not. It misses 14 of the 68 failures, so its recall is 79%. Ridge flags 54 machines and gets 28 right, with 26 false alarms and 40 failures missed, so its recall is 41%. In practice the XGBoost list wastes about one inspection in eleven, and Ridge's list wastes about one in two.

Inspection effort

Figure 8.5 turns the ranking into a statement a manager can use.

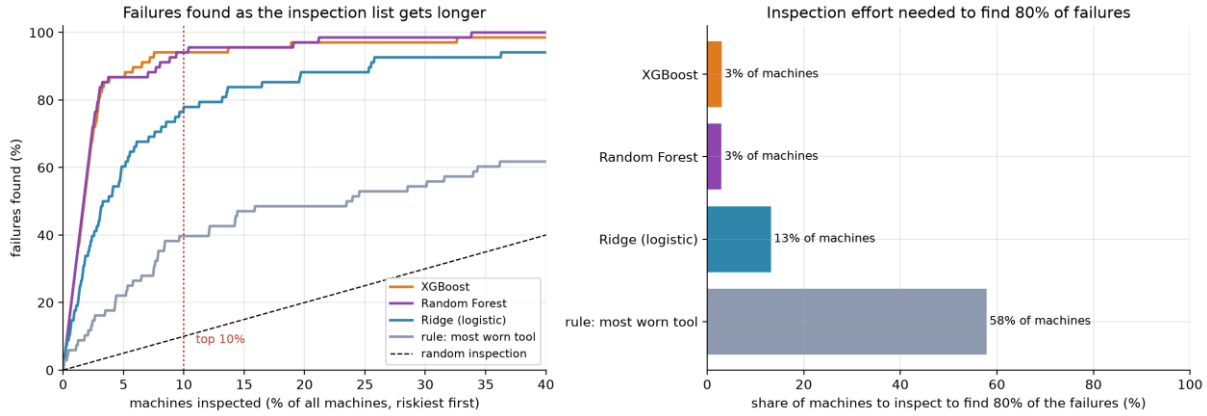


Figure 8.5. *Failures found as the inspection list grows*

The left panel plots the share of failures found (vertical) against the share of machines inspected (horizontal), riskiest first. The dotted line at 10% shows the top-10% capture that we quote: XGBoost and Random Forest are at about 94% there. The dashed diagonal is a random inspection order. The right panel counts how many machines must be inspected to find 80% of the failures: about 3% of

machines with XGBoost or Random Forest, 13% with Ridge and 58% with the most-worn-tool-first rule. The rule needs about 19 times as many inspections as XGBoost to catch the same share of failures.

Are the probabilities honest?

A risk of 20% should mean that one in five such machines fails. Figure 8.6 checks this.

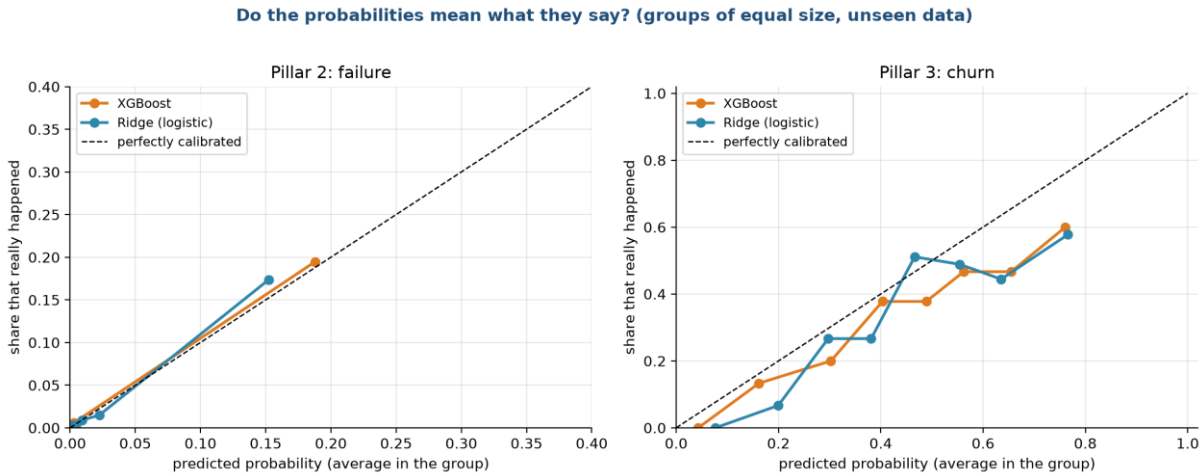


Figure 8.6. *Calibration of the risk scores*

Each marker groups cases of similar predicted probability and plots the actual rate of positives in the group against the average prediction. The dashed diagonal is a perfectly calibrated model. For failure (left panel), nearly all machines have a predicted probability below 3%, and the few with higher scores lie close to the diagonal, with 0.15 predicted against 0.17 observed for Ridge and 0.19 against 0.20 for XGBoost. Failures are so rare that the groups contain few cases, and we do not read much into the exact values. For churn (right panel), both models are slightly over-confident at the high end: of the customers given a probability of about 0.76, about 0.58 to 0.60 lapse. The scores rank customers well, and they are not exact probabilities, so the dashboard shows them as a risk percentage and a rank.

8.4 Pillar 3: churn

Table 8.4 gives the results on unseen customers in the later period. The test set holds about 360 customers, of whom about 32% lapse.

Table 8.4. *Churn prediction scores on unseen customers in the later period*

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	Lapsers in top 10%	Lift
Rule: longest since purchase	0.607 ± 0.041	0.417 ± 0.019	0.478 ± 0.022	0.318 ± 0.020	0.964 ± 0.019	0.162 ± 0.017	1.63 ± 0.17
Logistic regression	0.775 ± 0.033	0.575 ± 0.049	0.608 ± 0.024	0.459 ± 0.024	0.903 ± 0.019	0.197 ± 0.021	1.97 ± 0.21
Random Forest	0.772 ± 0.022	0.563 ± 0.035	0.595 ± 0.017	0.445 ± 0.018	0.901 ± 0.030	0.201 ± 0.025	2.01 ± 0.25
Ridge, basic inputs	0.750	0.547	0.593	0.476	0.788	0.187	
Ridge	0.777	0.578	0.609	0.465	0.885	0.202	2.0
XGBoost, basic inputs	0.749	0.529	0.590	0.491	0.741	0.178	
XGBoost	0.775	0.580	0.609	0.462	0.895	0.200	2.0

The result is plainer than for failure. Every model with the engineered inputs lands between 0.77 and 0.78 ROC-AUC, and the differences are far smaller than the standard deviation of 0.02 to 0.03. The rule of thumb reaches 0.61. So the model beats the rule clearly (0.78 against 0.61), and complexity buys nothing: logistic regression ties with XGBoost.

In terms of the call list, contacting the 10% of customers with the highest risk reaches 20% of those who lapse, twice as many as a random list (10%) and about 4 points more than the recency rule (16%). This is the weakest of the three results, and we say so in the conclusion.

The confusion matrices in Figure 8.4 (two right panels) show the trade-off. At its threshold of about 0.35, Ridge flags 211 of 360 customers. It catches 101 of the 118 who lapse (recall 86%) and wrongly flags 110 of 242 who stay (precision 48%). XGBoost catches 105 of 118 (89%) at the cost of 129 false alarms. The thresholds are set by F1 on the training data, which favours recall when lapsing is common, so the lists are long. For a small call budget, the manager should follow the ranking from the top instead of using the threshold, as the Retention List does.

Why does a more complex model not help?

The learning curves in stage 7 show that XGBoost needs the full training set just to reach Ridge, and the importance plots in stage 5 show that the signal comes from simple counts, such as how often and how recently the customer ordered. A customer's own history holds a limited amount of information about the next 90 days, and every model hits the same limit. Better inputs, such as product categories or the behaviour of similar customers, are more likely to help than a bigger model. We list them in the future work.

8.5 Stability across seeds

Figure 8.7 shows each seed as a dot.

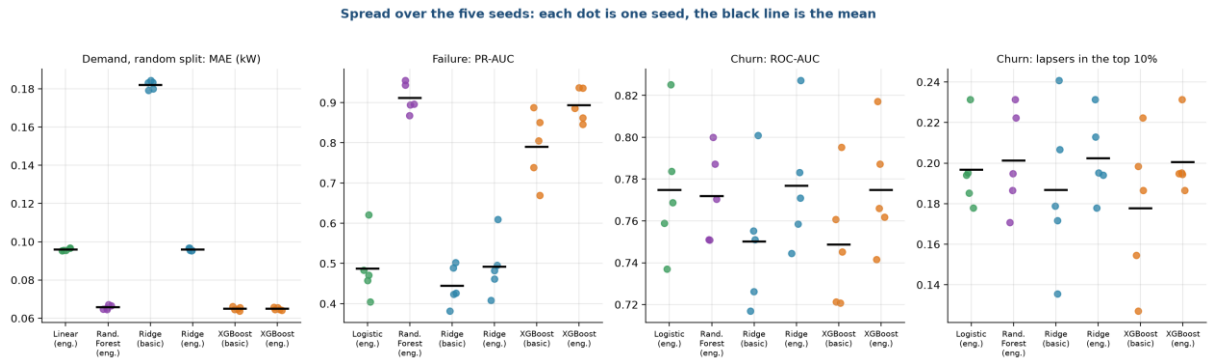


Figure 8.7. *Spread of the scores over the five seeds*

The black line is the mean of five seeds. The demand panel (first) shows almost no spread: all five dots sit on top of each other, because the large training set makes the model nearly deterministic. The failure panel (second) shows a wide spread for logistic regression and Ridge (0.38 to 0.62) and for XGBoost with basic inputs (0.67 to 0.89), and a tighter one for Random Forest and XGBoost with engineered inputs (0.85 to 0.95). The test set holds 68 failures, so one more or fewer detected changes the score visibly. The two churn panels have the widest spread, from 0.72 to 0.83 in ROC-AUC and from 0.13 to 0.24 for top-10% capture, which is a consequence of the small test set. This is why the selection rule treats gaps below one standard deviation as ties.

8.6 Tuning

We tuned very little, and on purpose.

What	How it was chosen	On which data
Ridge penalty (alpha, C)	Five-fold cross-validation	Training rows only
Alert threshold	Best F1 on out-of-fold scores	Training rows only
XGBoost settings	Fixed in advance (stage 7)	None
Random Forest settings	Fixed in advance	None
Feature set	Compared basic and engineered	Test scores reported for both
Model per pillar	Tie rule on mean and standard deviation (stage 6)	Test scores of the five seeds

Two points deserve honesty. First, the choice of model by the tie rule uses the test scores of the five seeds, so a small amount of selection pressure remains. With six candidates and one clear winner in each demand test, we think the effect is small. Second, we did not run a separate validation set, since the data is too small to hold out a third part. A truly independent check would need data from a real service company.

Stage 7 showed that a wide range of settings gives nearly the same score, so heavy tuning would not change the picture. The one place where more tuning is pointless is the heatwave: more trees, deeper trees or a lower learning rate cannot make a tree predict above its training range.

8.7 Summary of the evaluation

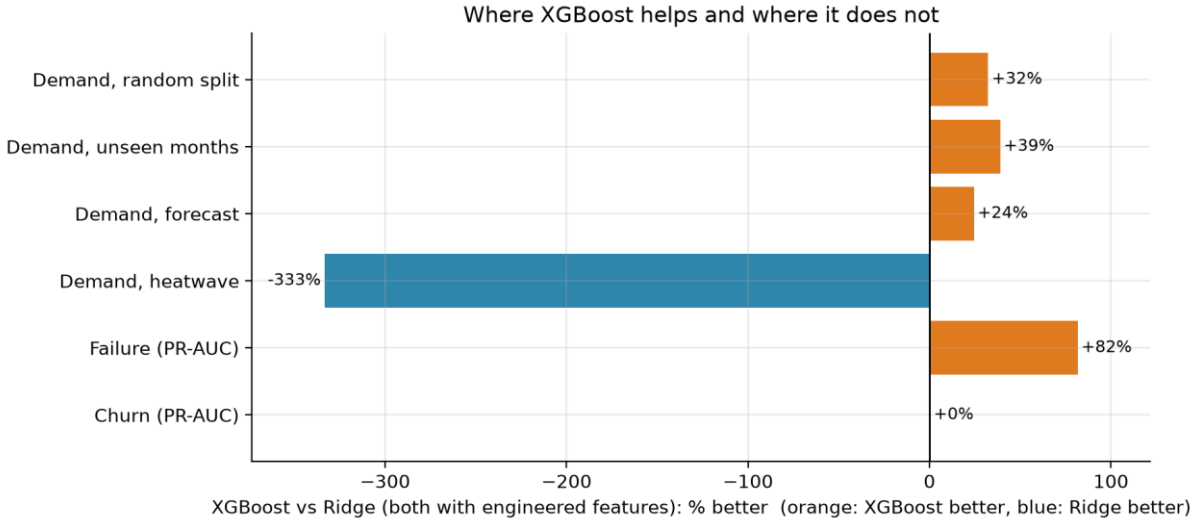


Figure 8.8. *Where XGBoost helps and where it does not*

Figure 8.8 compares XGBoost with Ridge, both with engineered inputs, as a percentage improvement. XGBoost is better by 32%, 39% and 24% on the first three demand tests and by 82% on failure (PR-AUC). It is 333% worse in the heatwave, where its error is more than four times that of Ridge, and it is the same for churn.

Pillar	Best result on unseen data	Compared with the rule of thumb	Verdict
Demand	MAE 0.065 kW (XGBoost), 0.077 kW in a heatwave (Ridge)	Rule: 0.93 kW and 1.57 kW	Strong, with the heatwave safeguard
Failure	PR-AUC 0.89 to 0.91; 94% of failures in the top 10%	Rule: PR-AUC 0.12 and 36%	Strong
Churn	ROC-AUC 0.78; 20% of lapsers in the top 10%	Rule: ROC-AUC 0.61 and 16%	Real but modest

8.8 What could make these numbers too good

- The HVAC data looks simulated, and the AI4I data is synthetic. Both follow clean rules that a model can learn. The scores for demand and failure are therefore likely higher than on real service data.
- The churn test set is small, so its scores carry large error bars.
- The tie rule selects from the same test scores that we report.
- The failure test has 68 failures per seed.
- The heatwave test covers one building, one year and one kind of extrapolation.

Chapter 9. Model deployment

Lifecycle stage 9 of 10. Written by Sourav Kumar Boro (26535026).

9.1 What deployment means for this project

Deployment is the step that puts a trained model in front of the people who use its answers. A model that sits in a notebook helps nobody. For HVAC Service Planner, deployment means a small web dashboard that a dispatch manager, a maintenance lead or a customer-success team can open in a browser, and that turns the saved models into three tools. We did not deploy to a cloud service, because we have no real users or real data. We built the dashboard so it can run on any laptop with one command, and we describe what would change for a production deployment in section 9.9.

9.2 System architecture

Figure 9.1 shows the whole system, from data files to the people who use the results.

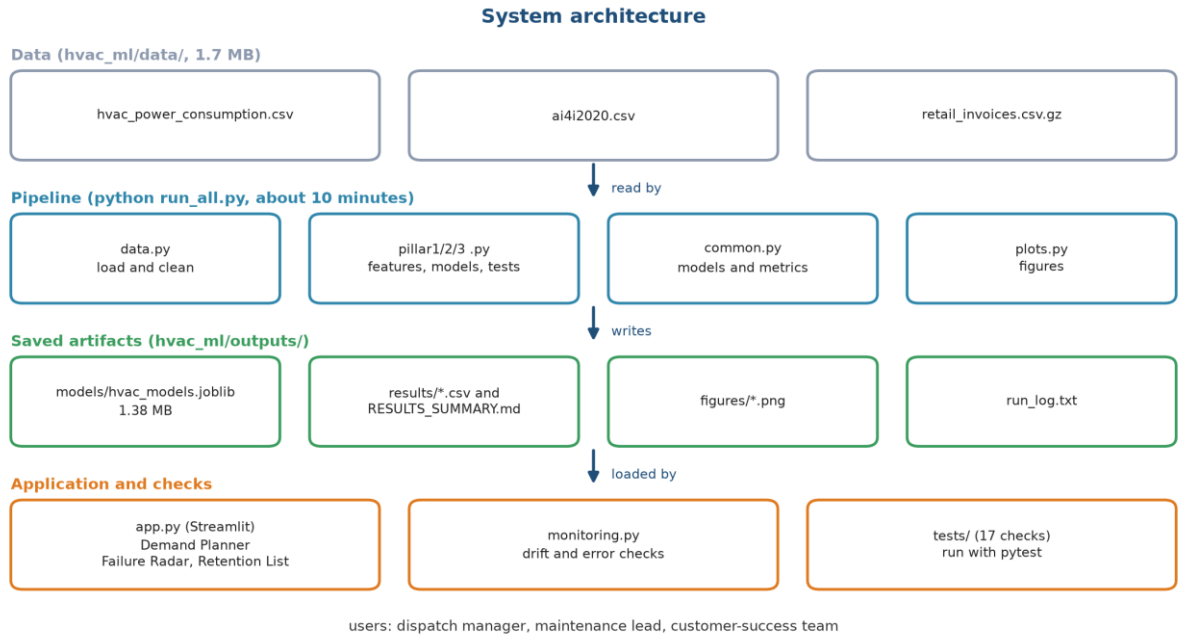


Figure 9.1. *System architecture*

The picture has four layers, read from top to bottom. The grey boxes are the three data files, 1.7 MB in all. The blue boxes are the pipeline, the programs that clean the data, build features, train and test the models, and draw the figures. A full run takes about ten minutes. The green boxes are what the pipeline writes: the saved models in a single file of 1.38 MB, the result tables, the figures and a log. The orange boxes are the programs that use these results: the dashboard, the monitoring checks (stage 10) and the automated tests. The users at the bottom are a dispatch manager, a maintenance lead and a customer-success team.

The design keeps training and serving apart. The pipeline can be run again at any time and produces a fresh models file. The dashboard only reads that file and the data. It never trains anything while a user waits, so every click is answered in a few milliseconds (1.4 to 2.4 ms per single prediction).

9.3 What gets deployed

Artefact	Content	Size
Models file	For each pillar, a Ridge model and an XGBoost model, trained on all rows, with the alert thresholds for the yes or no tasks and the list of features	1.38 MB
Data files	The three datasets, needed for the sample lists and for the range of the demand data	1.7 MB
Dashboard program	The three tools in one page with three tabs	one file
Supporting programs	Feature builders, the monitoring checks, settings	a handful of files
Tests	Seventeen automated checks	one file

The models file includes the time at which it was trained and the choice made by the selection rule, so anyone can see which version the dashboard is using.

9.4 The three tools

Tool 1: Demand Planner

The manager sets the month, the indoor setpoint, the humidity and the lowest and highest outdoor temperature of the day. The program builds a smooth temperature curve for the 24 hours, with the coolest hour at 05:00 and the hottest at 15:00, and works out the HVAC mode for each hour. It builds the model inputs for each hour and asks both models for a prediction. The tool shows the planned demand curve, the peak demand and the hour at which it occurs, the total energy for the day, and the number of hours that used the heatwave safeguard. The safeguard is explained in section 9.5.

Figure 9.2 shows the output for two days.

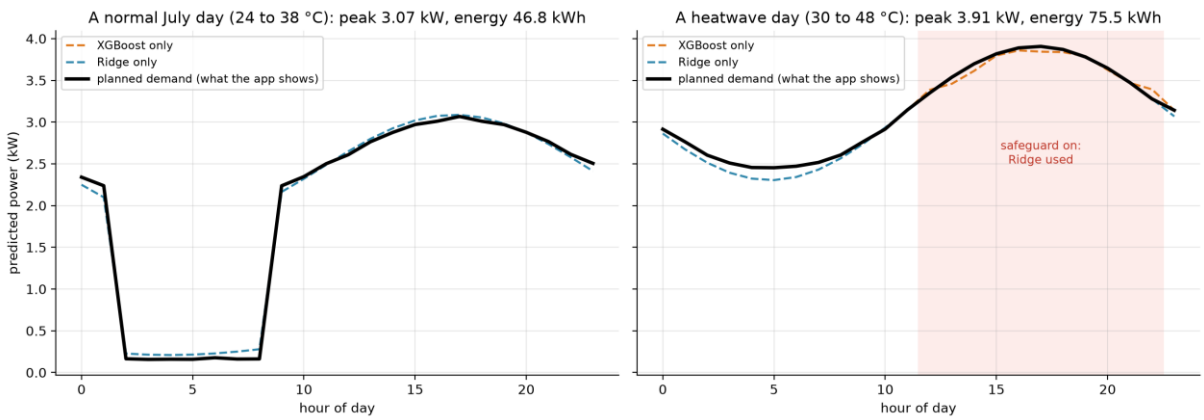


Figure 9.2. *Demand Planner output for a normal July day and for a heatwave day*

The left panel is a normal July day with temperatures from 24 to 38 °C. The black line is the planned demand: it falls to about 0.2 kW in the early hours, when the outdoor temperature is in the idle band, and rises to a peak of 3.07 kW in the late afternoon. The dashed orange and blue lines are the XGBoost and Ridge predictions alone, and they agree closely. The day's energy is 46.8 kWh. The right panel is a heatwave day from 30 to 48 °C. From about 11:00 to 22:00 the temperature is above 40 °C, which the pink band marks, and the planner uses the Ridge model there. The peak is 3.91 kW and the daily energy is 75.5 kWh. On this day the two models still agree closely, because the temperatures stay inside the range seen in training, but they would drift apart for temperatures above 50 °C, which is why the safeguard uses the range as well as the 40 °C line.

Tool 2: Failure Radar

The maintenance lead enters one machine's readings (type, air and process temperature, speed, torque and tool wear) and sees its risk of failure as a percentage, whether it is above the alert threshold (44% for XGBoost), and how it compares with the 10,000 machines in the dataset. A table lists the riskiest machines in a random sample of 500, and a caption says how many of the actual failures in the sample the top 10% would catch. Figure 9.3 shows that table.

Failure Radar: the 12 riskiest machines in a sample of 500 (alert threshold 44%)

Product ID	Type	Torque (Nm)	Speed (rpm)	Tool wear (min)	Risk (%)	Alert	Actually failed
L51822	L	54.6	1238	226	99.6	inspect	yes
L49778	L	67.5	1320	20	96.4	inspect	yes
M20475	M	71.3	1257	122	96.0	inspect	yes
L53705	L	73.6	1312	49	95.2	inspect	yes
L51800	L	52.6	1336	172	94.6	inspect	yes
M19287	M	48.9	1379	107	94.1	inspect	yes
L51801	L	51.0	1341	174	94.1	inspect	yes
M23874	M	61.3	1340	207	93.6	inspect	yes
M21659	M	12.2	2636	100	93.0	inspect	yes
L51724	L	55.1	1349	187	92.7	inspect	yes
L48763	L	61.0	1310	189	89.7	inspect	yes
L54939	L	59.2	1272	198	89.4	inspect	yes

Figure 9.3. *Failure Radar: the riskiest machines in a sample of 500*

Each row is a machine, with its identifier, type, torque, speed and tool wear, the risk the model gives it, whether it passes the alert threshold, and, for illustration, whether it did fail. All twelve machines shown have risks from 89% to 100% and all did fail. The model was trained on all 10,000 machines, so this list is a demonstration of the tool and not a test. The honest scores are in stage 8.

Tool 3: Retention List

The customer-success team chooses how many customers they can contact, from 10 to 200. The tool lists the active customers with the highest risk of buying nothing in the next 90 days, with each customer's risk, days since the last order, number of orders, spend and recent orders. A caption says what share of the customers who really lapsed this list would reach, and compares it with random contact. Figure 9.4 shows an example.

Retention List: the 12 active customers most likely to go quiet (illustration; the saved model saw these customers)

Customer	Risk (%)	Days idle	Orders	Spend	Orders (90 d)	Actually lapsed
14495	93.0	29	2	92	1	no
17660	91.5	49	2	359	1	yes
14997	90.6	86	2	386	1	no
14171	90.5	6	2	499	1	yes
16641	90.0	13	2	554	1	yes
17958	90.0	24	2	974	1	yes
14079	90.0	32	2	619	1	yes
15374	89.7	36	2	307	1	yes
18281	89.7	88	2	201	1	yes
15256	89.3	56	2	170	1	yes
15219	89.0	84	2	685	1	yes
13302	88.5	63	2	159	1	yes

Figure 9.4. *Retention List: the customers most likely to go quiet*

The twelve customers shown all have two orders in total and one in the last 90 days, and risks from 88% to 93%. They are the kind of customer the profile in stage 4 predicts: few orders, a short history. Ten of the twelve lapsed. As with the Failure Radar, the model saw these rows in its final fit, so this is an illustration. In the honest test in stage 8, the top 10% of the list reaches 20% of the customers who lapse, about twice as many as random contact.

9.5 The heatwave safeguard

The Demand Planner does not trust XGBoost everywhere. Figure 9.5 shows the rule.

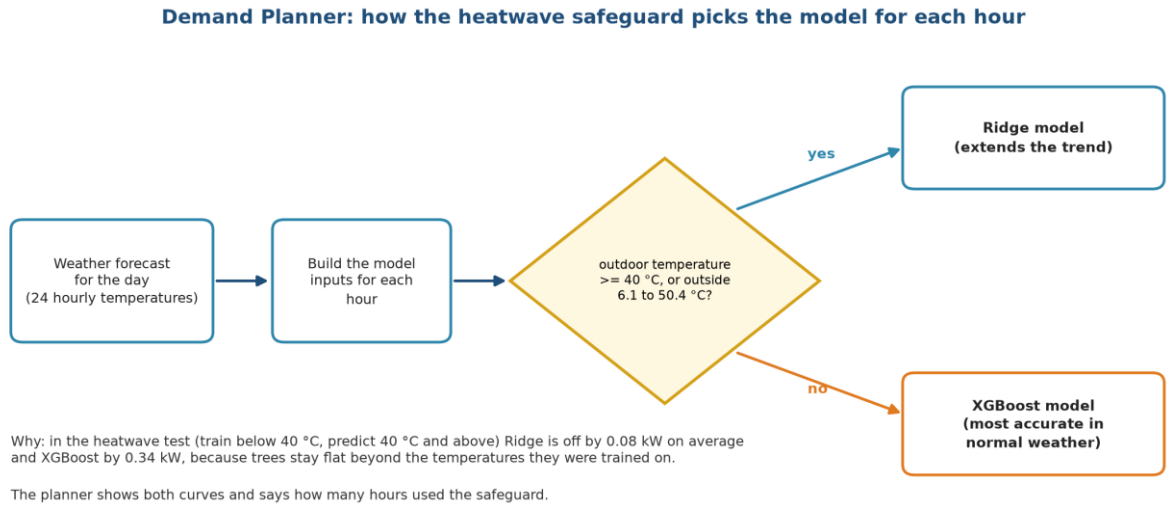


Figure 9.5. *How the safeguard chooses the model for each hour*

For each hour the program checks the outdoor temperature. If it is at or above 40 °C, or outside the range of temperatures seen in the training data (6.1 to 50.4 °C), the hour is predicted by Ridge. Otherwise it is predicted by XGBoost. The reason is the result in stage 8: on the heatwave test Ridge is off by 0.08 kW on average and XGBoost by 0.34 kW, because trees stay flat beyond the temperatures they were trained on. The planner shows both curves and says how many hours used the safeguard, so the manager can see that the switch happened. A warning text appears whenever it is used.

The switch has a visible cost: at the boundary, the two models may predict a little differently, and the curve can have a small step. We accepted this because a visible step is better than a hidden error of 0.3 kW or more.

9.6 How the dashboard is tested

- We ran the app without a browser, using Streamlit's own test runner, and checked that it starts, that it shows the three tabs and that it raises no exception. This check is not part of the automated suite yet, and it should be added.
- The seventeen automated tests of the pipeline run before each release (stage 7). They cover the data, the splits, the leakage rules, the metrics and the monitoring functions, and they do not cover the dashboard code.
- We did not run a user test with real staff. A production deployment would need one.

9.7 Installing and running

A user needs Python 3.10 or newer, about 300 MB of disk space for the packages and no internet connection once the packages are installed. The steps are:

1. Create a virtual environment and install the listed packages.
2. Run the pipeline once to train the models and write the models file. This takes about ten minutes.
3. Start the dashboard. It opens in the browser at the local address.

The help file in the project folder gives the exact commands, the places where the data comes from, and a table of common problems and their fixes.

9.8 Operating limits

Limit	Meaning
The data are stand-ins	The dashboard shows the method, and the numbers are not a forecast for a real company.
One building	Demand Planner is trained on one building and one year of data.
Static data	The dashboard reads stored files. It is not connected to live sensors or to a customer system.
Single user	It runs on one machine for one user at a time.
No login	There is no access control, and the data are public.
Failure model describes machine tools	The Failure Radar shows how to rank machines. It does not know HVAC units.

9.9 What a production deployment would need

If an HVAC company adopted this, five things would change.

1. Real data feeds. Weather forecasts for the demand tool, sensor logs for the failure tool and the invoice or service history for the retention tool would flow in on a schedule.
2. A scheduled job instead of a manual run. Retraining would be triggered by the monitoring rules in stage 10 and would write a new versioned models file.
3. A hosted service with access control, so that staff log in and see only the data that concerns them.

4. A way to log every prediction and, later, the outcome, so the monitoring checks have something to compare. At present the checks run on stored datasets.
5. A rollback plan. The previous models file stays on disk, and the dashboard can be pointed back at it if a new model misbehaves.

9.10 Deployment checklist

Check	Status
Models file loads, and the version and time are recorded	done
Dashboard starts without errors on a clean install	done
Heatwave safeguard switches at 40 °C and outside the training range	done
Failure and churn lists labelled as illustrations where the model has seen the rows	done
Limits stated in the help file and the report, and captions mark the lists that are illustrations	done
Live data connection	not built
Access control	not built
Automatic retraining	not built

Chapter 10. Monitoring and maintenance

Lifecycle stage 10 of 10. Written by Sourav Kumar Boro (26535026).

10.1 Why a deployed model needs watching

A model that scored well in testing can stop working afterwards without any error message. The world changes: a summer arrives that is hotter than any in the training data, a sensor is replaced and reads two degrees higher, a new group of customers joins. The model keeps returning numbers, and the numbers get worse. Nobody notices until a heatwave arrives with too few technicians or a call list is full of customers who were never at risk.

Monitoring is the routine that catches this early. Maintenance is what we do when it fires: investigate, retrain, or switch to a safer model. This document explains what we watch, the thresholds at which we act, how the checks behave on our own data, and the calendar of routine work. Everything here is written with a service company in mind, but the checks are tested on the datasets of this project, because we have no live system.

10.2 What can go wrong

Problem	Plain meaning	HVAC example	How we notice
Input drift (data drift)	The inputs look different from the training data	Outdoor temperatures reach 52 °C, above anything in the training year	Compare the distribution of each input with the training distribution
Concept drift	The link between inputs and the answer changes	A new building has better insulation, so the same temperature gap draws less power	The error rises while the inputs look normal
Rate drift	The share of positives changes	More customers lapse after a price rise	Compare the predicted share with the share that really happened
Data quality	Missing values, broken sensors, changed units	A sensor reports Fahrenheit instead of Celsius	Range and missing-value checks (the loaders already assert sizes and missing values)
Silent software change	A package update changes results	A new scikit-learn version shifts the last digit	The tests and the fixed seeds

10.3 The monitoring loop

Figure 10.1 shows the routine as a cycle.

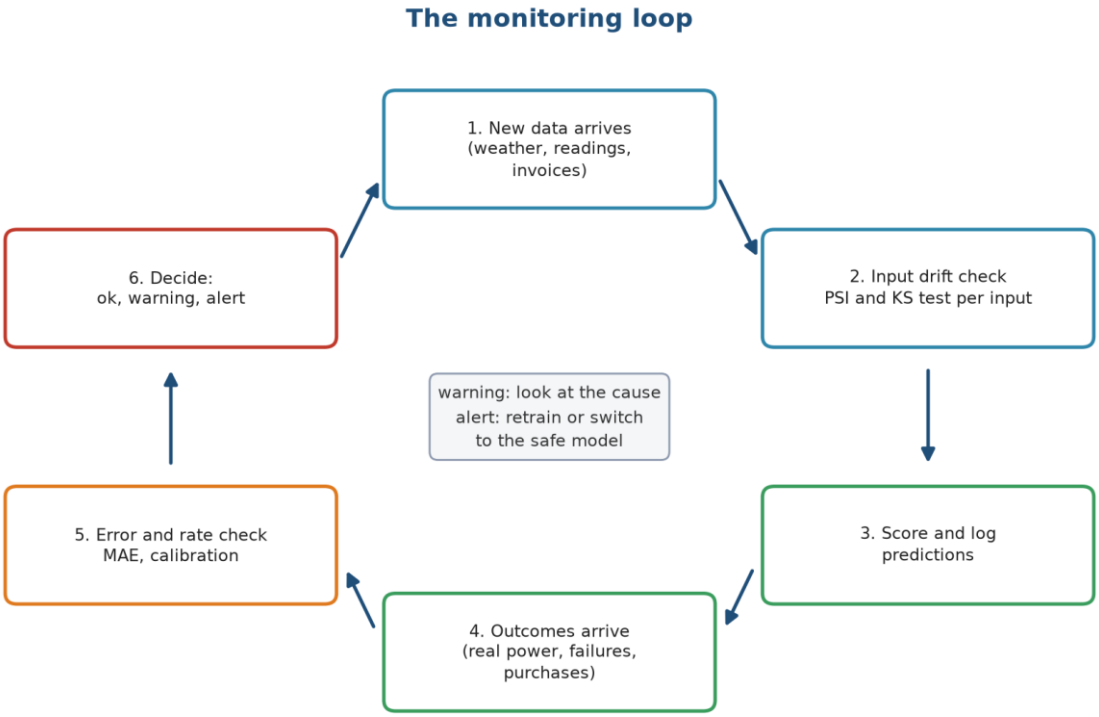


Figure 10.1. *The monitoring loop*

The loop has six steps. New data arrives (1), whether weather, sensor readings or invoices. An input drift check compares it with the training data (2). The model scores it and the prediction is logged (3). Later the real outcome arrives, such as the measured power, the failure record or the purchases of the next 90 days (4). An error and rate check compares the predictions with the outcomes (5). The last step is a decision (6): everything is fine, or a warning says to look at the cause, or an alert says to retrain or switch to the safe model. The cycle then starts again with the next batch of data.

Checks 2 and 5 are separate on purpose. The input check needs no outcome, so it can be run the moment new data arrives, and it warns early. The error check needs the real outcome, so it comes later, and it is the final proof of whether the model works.

10.4 The checks we built

Check 1: input drift with PSI and the KS test

The Population Stability Index (PSI) compares the distribution of one input in new data with its distribution in the training data. We cut the training values into ten bins of equal size and count what share of the new data falls in each bin. If the shares match, the PSI is near zero. If new data piles up in a few bins, it grows. The usual reading is below 0.10 for a stable input, 0.10 to 0.25 for a moderate shift and above 0.25 for a large shift. We use those thresholds. The Kolmogorov-Smirnov (KS) test is a second view: it gives the largest gap between the two cumulative distributions and a p-value. We also report how far the mean has moved, in units of the training standard deviation.

Check 2: error against the tested error

For demand, we compute the mean absolute error of each new month and divide it by the error measured in testing, 0.065 kW. A ratio of 1.5 or more is a warning, and 2.0 or more is an alert.

Check 3: rate against the observed rate

For failure and churn, we compare the share of cases the model flags, and its average predicted probability, with the share that really turned out positive. A gap of 0.10 is a warning and a gap of 0.20 is an alert.

Checks 1 and 2 are implemented as functions, and tested by three automated tests, which check that the PSI is small for the same distribution and large for a shifted one, that the drift table flags the column that moved, and that the error ratio maps to the right status. Check 3 is implemented as a function that returns the numbers. The 0.10 and 0.20 thresholds for it are applied by hand for now.

10.5 Alert rules

Figure 10.2 lists the rules and the action that follows.

Alert rules (the first two rows are implemented in monitoring.py; the rest are applied by hand)

Check	What is measured	Warning at	Alert at	Action
Input drift	PSI of each model input	>= 0.10	>= 0.25	check the data source; refit if it persists
Demand error	monthly MAE vs 0.07 kW tested	>= 1.5x	>= 2.0x	refit on the newest months; check sensors
Heatwave safeguard	hours above 40 °C using Ridge	any	error >= 2x 0.08 kW	extend the Ridge range; review features
Failure ranking	failures found in top 10% (tested 94%)	< 85%	< 70%	refit with the new failures; review sensors
Churn calibration	predicted vs actual lapse rate	gap > 0.10	gap > 0.20	refit on the latest cutoff; re-set the threshold

Figure 10.2. *Alert rules*

Each row is one check. The columns give what is measured, the level that triggers a warning, the level that triggers an alert and what we do. The five checks are input drift, demand error, the heatwave safeguard, the failure ranking and the churn calibration. The failure ranking check uses the 94% top-10% capture found in testing: a fall below 85% warns and a fall below 70% is an alert. The heatwave safeguard check watches the hours above 40 °C that used Ridge: any use is noted, and an error of 0.16 kW or more on those hours (twice the tested 0.08 kW) is an alert that the Ridge range needs extending.

10.6 How the checks behave on our data

We ran the checks on the project data, as a rehearsal of what they would do on live data.

Input drift

Figure 10.3 shows the PSI of the main inputs in three comparisons.

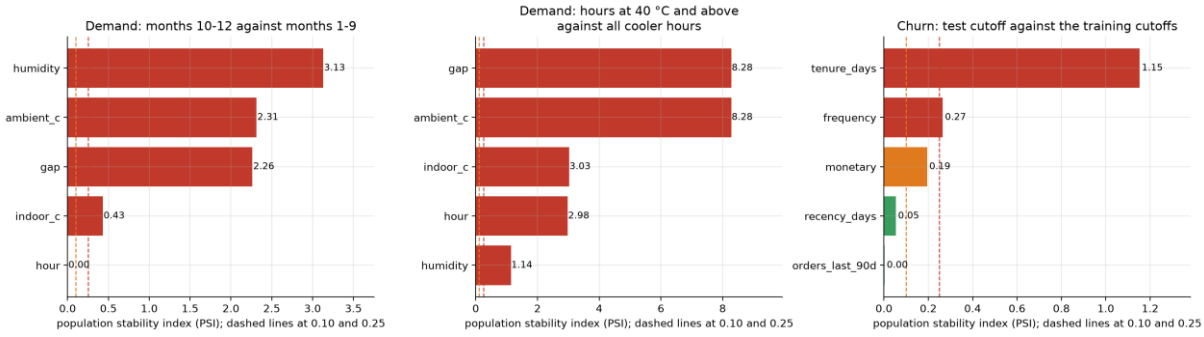


Figure 10.3. *Input drift in three comparisons*

Each bar is one input and its colour shows the status: green is stable, orange is a warning and red is an alert. The two dashed lines are the 0.10 and 0.25 thresholds.

- Demand, months 10 to 12 against months 1 to 9 (left): humidity (PSI 3.13), outdoor temperature (2.31), the temperature gap (2.26) and indoor temperature (0.43) all alert. Only the hour of day is stable. The reason is the season: autumn weather is not like the average of the first nine months. This is a useful lesson. A drift check that compares a new month with the whole of the training period will fire every season. In production the comparison should be with the same season in the previous year, or with the training data from the same months.
- Demand, heatwave hours against cooler hours (middle): the PSI of the outdoor temperature is 8.3 and that of the gap is 8.3. This is the drift we expected, since the heatwave hours were defined by the temperature. The check would have flagged the heatwave before the model failed, which is the purpose of an input check. The alert on these inputs is the reason for the safeguard.
- Churn, test cutoff against the training cutoffs (right): tenure drifts strongly (PSI 1.15), because customers are a year older at the later cutoff. Frequency alerts (0.27), spend gives a warning (0.19), and recency (0.05) and recent orders (0.003) are stable. A growing customer base is expected to look older, and the alert says that tenure needs a new training window, not that the model is wrong. The churn scores in stage 8 were nevertheless steady.

The lesson is that PSI tells us when the inputs have moved. It does not say whether the model is hurt. That is the job of the error check.

Performance over time

Figure 10.4 shows what the error check would have shown if we had run the models month by month, training only on earlier data.

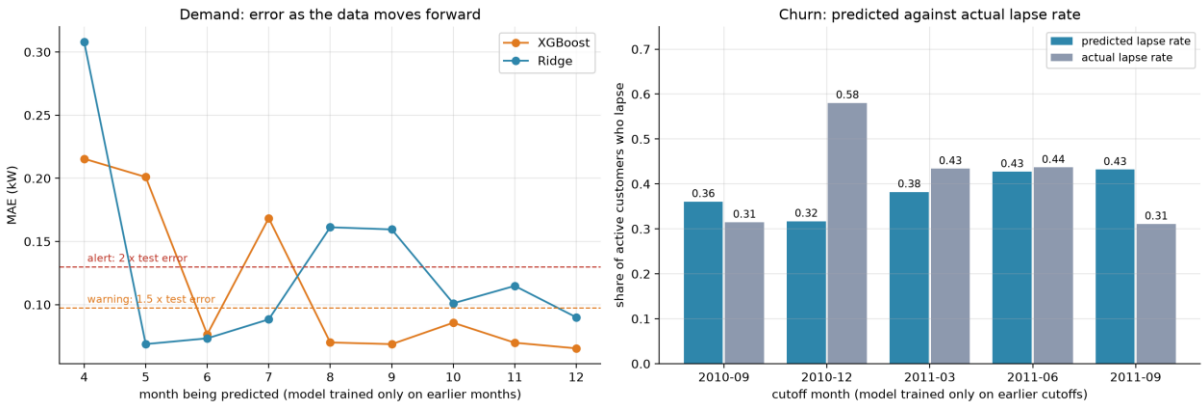


Figure 10.4. *Demand error and churn calibration as time moves forward*

The left panel is the monthly demand error for XGBoost (orange) and Ridge (blue), where each month is predicted by a model trained only on the months before it. The dashed lines are the warning (1.5 times the tested 0.065 kW, which is 0.098 kW) and the alert (0.13 kW). In April and May the errors of XGBoost are 0.22 and 0.20 kW, above the alert line, because the model has only seen the cold months and the spring weather is new to it, and July (0.17 kW) is above it too. Ridge is above the alert line in April (0.31 kW), and again in August and September (0.16 kW). From October on, both models are below the alert line, and XGBoost (0.066 to 0.086 kW) is also below the warning line. The panel shows two things: a model needs data from every season before it can be trusted, and our check would have raised the alarm in each of those months. The right panel is the churn check. For each cutoff, a model trained on the earlier cutoffs predicts a lapse rate (blue) that we compare with the real one (grey). The gaps are 0.05 in September 2010, 0.26 in December 2010 (an alert), 0.05 in March 2011, 0.01 in June 2011 and 0.12 in September 2011 (a warning). The December 2010 miss comes from the holiday weeks: more customers than usual go quiet, and a model trained on spring and summer does

not know. The model ranks customers well even when it misjudges the total rate, so the retention list stays useful, but the number of customers who will lapse should be corrected for the season.

10.7 Maintenance routine

Figure 10.5 sets out the work over a year.

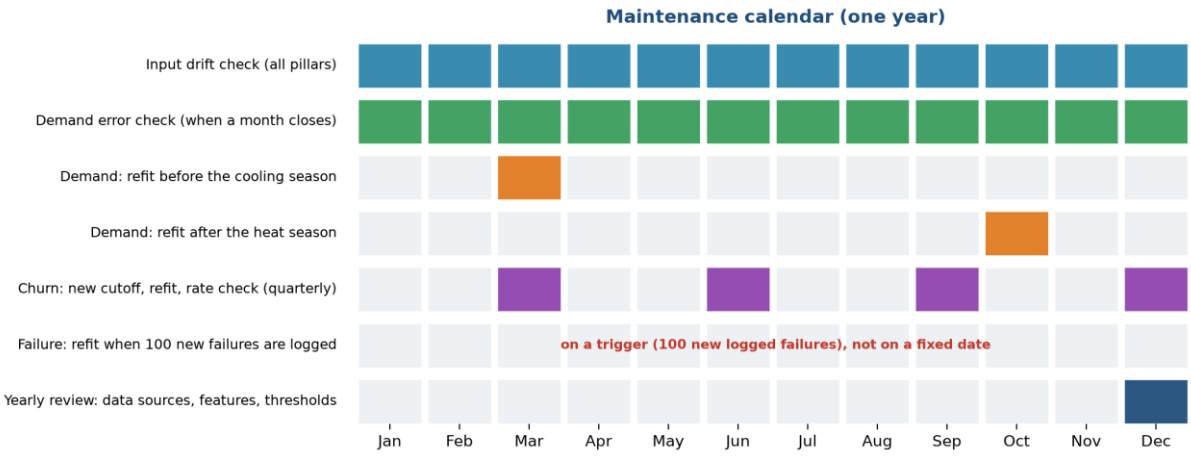


Figure 10.5. *The maintenance calendar*

Each row is a task and each coloured square a month in which it happens. The input drift check runs every month for all three pillars, and the demand error check runs when each month closes (blue and green rows). The demand model is refitted before the cooling season, in March, so that it includes the newest winter data, and again after the heat season, in October, so that it learns the new highest temperatures. The churn model gets a new cutoff, a refit and a rate check each quarter (purple). The failure model is not refitted on a calendar. It is refitted when 100 new failures have been logged, since failures are rare and retraining without new failures would change nothing. In December there is a yearly review of the data sources, the features, the thresholds and this document (dark blue).

Task	Who	How often	Output
Drift check, all inputs	Data analyst	Monthly	Table of PSI and KS per input
Error check, demand	Data analyst	Each month's end	MAE against 0.065 kW
Rate check, churn and failure	Data analyst	Quarterly	Predicted against real share
Refit with new data	Data analyst	As in Figure 5	New models file with a new date
Compare new and old models	Maintenance lead	At each refit	Same test as stage 8, on a fixed set
Approve and deploy	Manager	At each refit	Replace the file, keep the old one
Yearly review	Whole team	Each December	Updated thresholds and documents

10.8 What to do when a check fires

1. Warning. Look at the cause before changing anything. Find which input moved and since when. Check that the data source still reports in the same units. A warning on a seasonal input is often normal.
2. Alert on one input, error unchanged. Note it and watch the error check. Do not retrain on drift alone.
3. Alert on the error. Refit on the newest data, test the new model on the same fixed test set as in stage 8, and deploy it only if it is not worse on the old test and better on the recent data.
4. Alert in a heatwave. Use the safeguard (Ridge) for the affected hours, check that the range of the safeguard covers the new temperatures, and extend the training data.

5. Anything else. Roll back to the previous models file, which we keep for this purpose.

10.9 Keeping the project maintainable

- One configuration file holds every setting, including the monitoring thresholds.
- Each models file records its training time and the results of the selection rule.
- The tests run before every change. They would catch a broken split or a leaked feature.
- The documents in this folder, one per lifecycle stage, are meant to be updated each time the process changes.
- Package versions are recorded in the requirements file. A package update that changes a result is caught by comparing with the saved results.

10.10 Limits of our monitoring

- We have no live system, so the checks ran on stored data. We do not log predictions as they are made.
- PSI and KS are affected by sample size. With thousands of hourly rows, even a small shift gives a tiny p-value, so we rely on the PSI size and the size of the mean shift and not on the p-value.
- The thresholds (0.10, 0.25, 1.5 times and 2 times) are common rules of thumb, not values fitted to our data.
- Seasonal inputs need a comparison with the same season of the previous year, which we only have for the retail data.
- The failure model has too few failures to be monitored month by month.
- A drift check cannot tell whether a change is a real change in the world or a fault in the data feed. A person has to look.

10.11 Summary

Question	Answer
What do we watch?	The distribution of each input, the monthly demand error, and the predicted against actual positive rate
When do we act?	PSI above 0.25, error above twice the tested value, a rate gap above 0.20
What do we do?	Look at the cause first, then refit or fall back to the safe model
How often do we refit?	Demand twice a year, churn each quarter, failure after 100 new failures
What did the rehearsal show?	The checks fire when the weather moves into new ground, and a model needs a full year of data before it can be trusted

Chapter 11. Results and discussion

11.1 The results on one page

The table below collects the headline result of each pillar, next to the rule of thumb that a company could apply today. All numbers come from data the models did not see and are means over five seeds.

Pillar	Measure	Rule of thumb	Statistical baseline	Ridge	XGBoost	Best result
Demand, ordinary weather (forecast test)	Error in kW, lower is better	1.02	0.101	0.100	0.076	XGBoost
Demand, heatwave	Error in kW, lower is better	1.57	0.077	0.077	0.336	Linear models
Failure	PR-AUC, higher is better	0.123	0.487	0.492	0.893	Random Forest 0.912, XGBoost close
Failure	Failures found in the top 10%	36%	74%	73%	94%	Random Forest and XGBoost
Churn	ROC-AUC, higher is better	0.607	0.775	0.777	0.775	All four models tie
Churn	Lapsers found in the top 10%	16%	20%	20%	20%	All four models tie

The statistical baseline is ordinary linear regression for demand and logistic regression for failure and churn. The rule of thumb is the average by hour of day for demand, the most worn tool first for failure and the longest time since the last purchase for churn.

11.2 What the results tell us

First, using the weather beats ignoring it by a wide margin. For demand, every model that sees the temperature has an error around 0.1 kW or less, while the rule that ignores it is wrong by about 1 kW, which is half of the average load. The same is true for failure: a ranking built from the sensor readings finds more than twice as many failures as the most-worn-tool rule.

Second, the best model depends on the question. XGBoost is the best choice in ordinary weather and for failure. A linear model is the only safe choice beyond the training range, as the heatwave test showed. In churn, the simplest model is as good as any. A group that had chosen XGBoost for everything would have a good dashboard on most days and a wrong forecast on the hottest days, which are the days that matter. A group that had chosen Ridge for everything would have missed 40% of the accuracy on failure.

Third, engineered inputs matter most for the simple models. The absolute temperature gap halves the error of Ridge in demand and removes almost all of its heatwave error. Trees find the same shape on their own. A feature that is based on the physics of the problem is therefore cheaper than a more complex model.

Fourth, the metric decides what looks good. If we had scored the failure model by accuracy, a model that never raises an alarm would have scored 96.6%. By PR-AUC and top-10% capture, it scores almost nothing, as it should.

Fifth, small test sets give wide error bars. The churn test holds about 360 customers and the failure test 68 failures, and the five seeds disagree by several points on them. The tie rule, which treats gaps below one standard deviation as equal, kept us from reading too much into those differences.

11.3 How the product is used

From result to decision: what each tool is, and what to trust it for

Tool	Model in use and the evidence	Trust it for	Do not use it for
Demand Planner	XGBoost below 40 °C. Ridge at 40 °C and above, or outside 6.1 to 50.4 °C. Error 0.065 kW against 0.93 kW for the rule of thumb. Heatwave error 0.08 kW.	Hourly demand curve, peak hour and energy for the day, to plan staff, stock and grid load.	Planning with XGBoost alone in a heatwave. Other buildings or climates without retraining.
Failure Radar	XGBoost (Random Forest ties). PR-AUC 0.89 against 0.12 for the rule. The top 10% of the list holds 94% of failures, against 36% for the rule.	Ranking machines for inspection and working down the list as far as the team's capacity allows.	Treating the 44% alert level as exact. Trusting the scores on HVAC units before retraining on company data.
Retention List	Ridge (penalised logistic regression). ROC-AUC 0.78 against 0.61 for the rule. The top 10% holds 20% of lapsers: random 10%, rule 16%.	Choosing whom to phone first when the call budget is small.	Forecasting the exact number of customers who will lapse (the rate was off by 0.26 in a holiday window). Expecting more than twice the random hit rate.

Figure 11.1. *From the result to the decision, for each pillar*

Figure 11.1 sums up what each tool is, what it should be trusted for, and what a manager should do with it. The first column names the tool. The second gives the model in use and the evidence for it. The third states what a manager can rely on, and the fourth states what the tool should not be used for.

11.4 What the numbers would mean in practice

These examples are arithmetic on our test results, not forecasts for a company.

- Demand. On an ordinary day with an average load of 1.9 kW, an error of 0.065 kW is a miss of about 3%. A manager who plans staffing from the average-by-hour rule would be wrong by half of the load.
- Failure. A team that can inspect one machine in ten finds 94 of every 100 failures with the model and 36 with the wear rule. To find 80 in every 100, the model needs inspections of about 3% of the machines and the wear rule needs 58%.
- Retention. A team that can phone 190 of 1,900 active customers reaches 20% of those who lapse with the model, 16% with the recency rule and 10% by calling at random. The gain over the rule is four points, and over random calling it is a factor of two.

11.5 Threats to the validity of the results

Threat	Direction of the bias	What we did about it
The demand and failure data are simulated and follow clean rules	Scores are probably higher than on real data	Said so in every stage; claim only the method
One building, one year	The heatwave result may not transfer to other sites	Tested forecast and heatwave separately
The tie rule uses the same test scores that we report	A small optimistic bias in the choice of model	Five seeds; gaps below one standard deviation count as ties
Small churn test set	Wide error bars	Report the spread for every number
Final models saw all rows for the dashboard lists	The lists look better than the test	Marked as illustrations; honest scores come from the tests

Chapter 12. Conclusion and recommendation

12.1 Conclusion

We set out to show how machine learning could help an HVAC service provider such as Aire Serv with three weekly decisions: planning for demand, choosing which machines to inspect and choosing which customers to call. We built a dashboard with one tool for each decision and tested the models behind them on data they had never seen.

The work answered the question for our three public stand-in datasets. Predicting hourly power from the weather is easy for a model that sees the weather, and the question is only how far the model can be trusted when the weather leaves the range of the training data. Ranking machines by failure risk is where machine learning adds the most: the best models find 94% of failures in the 10% of machines they rank highest. Predicting which customers will go quiet is the hardest of the three, and a simple model is as good as a complex one.

We also learned some things about the process. A test that matches the real use, such as forecasting the last three months or predicting a heatwave, tells us more than a random split. A baseline from statistics and a rule of thumb keep the comparison honest. The best model changes with the question, and we should say plainly where the more complex model does not win.

12.2 Recommendation

We recommend that an HVAC service provider adopt the three tools in this order and under these conditions.

Recommendation to an HVAC service provider

Priority	Tool	What to do	Evidence from held-out tests	Condition before relying on it
1	Failure Radar	Adopt. Rank machines by risk and inspect from the top of the list down, within the team's capacity.	Top 10% of the list holds 94% of failures (rule: 36%). Inspecting 3% finds 80% (rule: 58%).	Retrain on the company's own sensor and failure records.
2	Demand Planner	Adopt with the heatwave safeguard. XGBoost in ordinary weather, Ridge at 40 °C and above or outside the training range.	Error 0.065 kW (rule: 0.93 kW). In the heatwave test 0.08 kW with Ridge against 0.34 kW with XGBoost.	Retrain on the company's buildings before each cooling season. Never use XGBoost alone.
3	Retention List	Use as a low-cost aid. Keep the model simple (Ridge).	Top 10% holds 20% of lapsers (random 10%, rule 16%). All models tie at about 0.78 ROC-AUC.	Add product and contract data before paying for a more complex model.
4	Monitoring	Run from the first day. Check input drift monthly, demand error monthly and churn rate quarterly.	Alert at PSI 0.25, error twice the tested 0.065 kW, or a rate gap of 0.20.	Log every prediction and its outcome.

Figure 12.1. *Our recommendation, with the evidence and the condition for each tool*

Figure 12.1 states the recommendation as a table. In words:

1. Start with the Failure Radar. Use XGBoost (Random Forest is equally good in our tests) to rank machines by failure risk, and inspect from the top of the list down to the capacity of the team, without relying on a single alert threshold. In testing, inspecting the top 10% finds 94% of failures against 36% for the most-worn-tool rule. Before relying on it, retrain on the company's own sensor and failure records, since our data describe machine tools.
2. Then use the Demand Planner, always with the heatwave safeguard. Use XGBoost in ordinary weather and switch to Ridge regression for hours at or above 40 °C and for temperatures outside the range seen in training. Never use XGBoost alone for planning peak load. Retrain before each cooling season on data from the company's own buildings.

3. Treat the Retention List as a low-cost aid. Use a simple penalised logistic regression (Ridge) to rank active customers, and expect about twice the hit rate of random calling. Do not spend on a more complex model until richer data, such as product categories or service contracts, are available.
4. Run the monitoring routine from the first day. Check input drift each month, compare the demand error with the tested 0.065 kW each month, and compare predicted and actual churn each quarter. Refit the demand model before and after the cooling season, the churn model each quarter and the failure model after 100 new logged failures.

This recommendation depends on a pilot. The scores in this report are from public stand-in data, and a service provider should confirm the gains on its own records before changing how it works. A reasonable pilot is a four-week side-by-side run, in which the tools produce lists and the current process continues, and the two are compared at the end.

12.3 Closing note

Of the three tools, we trust the Failure Radar and the Demand Planner the most, on the evidence of the held-out tests. We trust the Retention List the least, because its gain is small, and we say that clearly.

Chapter 13. Limitations and future work

13.1 Limitations

- None of the three datasets comes from an HVAC service company. The demand file looks simulated, the failure file is synthetic and describes machine tools, and the purchase file is from an online gift shop. The method transfers, and the scores do not.
- The demand data covers one building for one year. We cannot say how the models behave in another climate, and the heatwave test is one kind of extrapolation.
- The churn definition (no purchase in 90 days) suits a shop. A service company would define churn from contract renewals or missed visits.
- The churn test set is small, about 360 customers, so its scores have wide error bars.
- The models choose among six candidates using the same test scores that we report, so a small optimistic bias remains.
- The monitoring checks ran on stored data. We have no live system that logs predictions and outcomes.
- The dashboard is a demonstration. It has no login, no live data feed and no automatic retraining.

13.2 Future work: where a graph neural network may help

An earlier version of this project used a graph neural network (GraphSAGE) on a synthetic graph of locations, residents, devices and spare parts. We tested it on held-out months and nodes with three seeds. It never clearly beat a simple model, as the table below shows. The reason lies in the data: every label in that graph was computed from the node's own features, so neighbours carried no extra information. We therefore removed the graph model and do not claim that it helps.

Test (held-out months 3, 6, 9, 12)	GraphSAGE	Ridge with pairwise products
Demand error (MSE, lower is better)	29.6 ± 2.9	28.5 ± 5.2
Inventory error (MSE)	24.5 ± 1.7	19.2 ± 0.8
Forecast of months 10 to 12, demand error	73.5 ± 21.7	36.2 ± 6.0

The results of this project point to a short list of places where a graph model deserves a fair test.

Where a graph neural network might help, and where it would not

Gap in our results	Could a graph model help?	Why it might	Why it might not
Churn: every model ties at about 0.78 ROC-AUC	**Maybe**	Customers who buy the same products may carry signal that one customer's row does not.	The signal may not exist in purchase data. XGBoost with neighbour-averaged features may do as well.
Cross-selling a counter-seasonal appliance	**Maybe**	It is link prediction on a customer-product graph, which graph models learn directly.	Item co-occurrence and matrix factorisation are strong and much simpler.
Demand across several buildings	**Maybe**	Sites on a shared grid or weather system influence each other, and a graph can say so.	We have data for one building only, so there is nothing to test it on.
Heatwave: trees fail above 40 °C	**No**	No reason to expect it.	Trees cannot predict above the range they saw. The fix is a linear model, which the dashboard already uses.
Failure: PR-AUC 0.89 to 0.91	**Not needed**	No gap to close.	The machines in the data are independent rows with no links.

Figure 13.1. *Gaps in our results and whether a graph model could close them*

Figure 13.1 lists each gap in our results, why a graph model might help and why it might not. The first row is churn, where every model ties at about 0.78 ROC-AUC. Customers who buy the same products might carry signal that a single customer's row does not, and a customer-product graph is a natural way to add it. The second row is cross-selling a counter-seasonal appliance, which is link prediction on a customer-product graph. The third row is demand across several buildings that share a power grid. The fourth row is the heatwave, and a graph model would not fix it: trees fail there because they cannot extrapolate, and the remedy is the linear model that the dashboard already uses. The last row is failure, which is not a gap, since XGBoost reaches 0.89 PR-AUC and the machines in the data have no links between them.

How to test the idea properly

1. Build a customer-product graph from Online Retail II, with customers and products as nodes and purchases as links.
2. Define two tasks: churn as node classification, and "which product next" as link prediction on purchases held out in time.
3. Compare the graph model with XGBoost given neighbour-averaged features, such as the average recency of a customer's co-buyers, and with ridge or logistic regression and an item co-occurrence recommender.
4. Use the same split as this project, unseen customers in a later period, with at least five seeds and the same tie rule. The graph model must beat the best simple model by more than one standard deviation to be preferred.

13.3 Other future work

- Retrain and test on real HVAC data: smart-thermostat or building-management logs, technician call-out records and service contracts.
- Add a weather forecast feed to the Demand Planner and measure the error of the forecast itself, not only of the model given the true weather.
- Extend the demand model to several buildings and test whether a shared model beats one model per building.
- Add product categories to the churn features and test a cross-sell recommender.
- Log every prediction and outcome in the dashboard, so that the monitoring checks run on live data.
- Run the four-week pilot described in the recommendation, with the current process running alongside.

References and data sources

Data sources

1. felixpradoh (Kaggle user). HVAC Power Consumption (hourly, 2025). Kaggle. https://www.kaggle.com/datasets/felixpradoh/hvac-power-consumption
2. Matzka, S. (2020). AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository, dataset 601. https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset
3. Chen, D. (2019). Online Retail II. UCI Machine Learning Repository, dataset 502. https://archive.ics.uci.edu/dataset/502/online+retail+ii

Methods and software

4. Hoerl, A. E., and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. Technometrics, 12(1), 55-67.
5. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
6. Chen, T., and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
7. Pedregosa, F., et al. (2011). Scikit-learn: machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
8. Saito, T., and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432.
9. Siddiqi, N. (2006). Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Wiley. (Population Stability Index and its usual thresholds.)
10. Hamilton, W. L., Ying, R., and Leskovec, J. (2017). Inductive representation learning on large graphs. Advances in Neural Information Processing Systems 30. (GraphSAGE, discussed as future work.)
11. Streamlit documentation. https://docs.streamlit.io
12. Matplotlib, pandas, NumPy and SciPy documentation, used for figures, tables and the Kolmogorov-Smirnov test.

Course material

13. CSC-505 Machine Learning, Department of Computer Science and Engineering, IIT Roorkee. Lecture notes and project guidelines.